

FPGA Yolo Accelerator 설계

전체 커리큘럼 (총 14차시)

차시	주제	기반 자료
1차시	프로젝트 전체 그림 – "우리가 뭘 만드는 건가?"	강의1
2차시	딥러닝 기초 – YOLO와 CNN의 원리	강의1 + 배경지식
3차시	AIX 모델 구조 – 22개 레이어 완전 해부	강의1, 강의2
4차시	소프트웨어 코드 구조 – 컴파일, 실행, 디렉토리	강의1, 강의2, 코드
5차시	양자화(Quantization) – 부동소수점에서 INT8으로	강의1, 강의2, C코드
6차시	시스템 개요 – Host-FPGA 전체 구조	강의2
7차시	테스트 벡터 생성 – SW↔HW 검증의 핵심	강의2
8차시	Verilog HDL 기초 – 하드웨어 설계 언어 입문	05_HDL
9차시	MAC 유닛 설계 – 곱셈-누산기의 모든 것	강의3, 05_DSP_MAC
10차시	BRAM 버퍼 설계 – 온칩 메모리 관리	강의3, 06_BRAM
11차시	CNN 컨트롤러 – 슬라이딩 윈도우와 FSM	07_Controller
12차시	탑 모듈 통합 – 전체 가속기 엔진 조립	08_Top_Module, TB
13차시	FPGA 구현 – Vivado, Bitstream, IP 패키징	강의4
14차시	시스템 검증 – Host-FPGA 통신과 최종 테스트	강의4

1차시: 프로젝트 전체 그림 – "우리가 뭘 만드는 건가?"

1-1. 이 프로젝트의 정체

이 프로젝트의 이름은 **AIX2026**이고, 차세대반도체 혁신공유대학 사업단에서 운영하는 AI 하드웨어 설계 프로젝트입니다. 한 마디로 요약하면 이렇습니다:

"카메라로 찍은 사진 속 물체를 실시간으로 인식하는 AI를, 소프트웨어가 아니라 반도체 칩(FPGA) 위에 직접만들어 올리자."

여러분이 평소 쓰는 AI(ChatGPT, 사진 필터 등)는 모두 GPU나 CPU 같은 범용 프로세서에서 소프트웨어로 돌아갑니다. 하지만 자율주행차, 드론, 스마트 카메라처럼 **전력이 제한되고 실시간 처리가 필요한 곳**에서는 이런 방식이 비효율적이에요. 그래서 AI 연산 자체를 전용 하드웨어로 만드는 겁니다. 이것이 바로 **AI 가속기(AI Accelerator)**입니다.

1-2. 왜 YOLO인가?

이 프로젝트에서 하드웨어로 구현할 AI 모델은 **YOLOv2 (You Only Look Once)**라는 객체 검출(Object Detection) 네트워크입니다.

객체 검출이란 사진을 주면 "이 사진 안에 뭐가 있고, 어디에 있는지"를 알아내는 것입니다. 예를 들어 편의점 상품 사진을 넣으면, "여기에 코카콜라가 있고, 저기에 과자가 있다"라고 위치와 종류를 동시에 알려주는 거죠.

YOLO가 특별한 이유는, 이미지를 **한 번만 보고(You Only Look Once)** 한꺼번에 모든 객체의 위치와 종류를 판단한다는 점입니다. 다른 방식들은 이미지의 여러 영역을 반복해서 살펴봐야 하는데, YOLO는 이미지 전체를 격자(Grid)로 나눈 뒤 각 격자마다 "여기에 물체가 있나? 있다면 무엇인가?"를 동시에 예측합니다. 이 덕분에 실시간 처리에 적합하고, 하드웨어 가속기로 만들기에도 좋은 구조입니다.

1-3. AIX2024 모델의 규격

이 프로젝트에서 사용하는 모델의 구체적 사양은 다음과 같습니다:

입력: 1920×1080 Full-HD 이미지를 받아서, 내부적으로 **256×256×3 (가로×세로×RGB채널)**으로 리사이즈합니다.

네트워크: 총 **22개 레이어**로 구성됩니다. Convolution(합성곱) 11개, Max-pooling 6개, Route 2개, Upsample 1개, YOLO 2개 레이어입니다. 아직 각 레이어가 뭔지 모르셔도 괜찮습니다. 2차시와 3차시에서 자세히 다룹니다.

출력: 8×8×195 텐서와 16×16×195 텐서, 총 두 개의 출력이 나옵니다. 여기서 195라는 숫자는 **[4(위치좌표) + 60(클래스 확률) + 1(객체 존재 확률)] × 3(앵커 박스)**으로 계산됩니다. 즉 각 격자칸마다 3개의 후보 박스를 예측하고, 각 박스에 대해 위치 4개, 60개 클래스 중 어느 것인지, 그리고 정말 물체가 있는지를 나타내는 값을 출력하는 것입니다.

1-4. 전체 시스템 구조 — 큰 그림

이 프로젝트의 시스템은 크게 두 부분으로 나뉩니다:

Host PC (호스트 컴퓨터): 여러분의 노트북이나 데스크탑입니다. 이미지를 읽고, 양자화된 모델의 파라미터를 준비하고, 결과를 받아서 화면에 보여주는 역할을 합니다. C++ 프로그램으로 작성됩니다.

FPGA 보드 (Nexys A7): 실제 AI 연산을 수행하는 하드웨어입니다. Xilinx Artix-7이라는 FPGA 칩이 탑재된 보드를 사용합니다. 이 안에는 다음이 들어갑니다:

- ☐ **MicroBlaze** — FPGA 안에 들어가는 소형 CPU. 전체를 제어합니다.
- ☐ **YOLO 가속기** — 여러분이 직접 설계하는 핵심 IP. MAC 유닛, 메모리 बैं크, 컨트롤러로 구성됩니다.
- ☐ **UART** — Host PC와 통신하는 직렬 인터페이스
- ☐ **MIG (Memory Interface Generator)** — DDR2 DRAM을 제어하는 모듈

Host PC와 FPGA는 **UART (시리얼 통신)**으로 연결됩니다. Host가 이미지 데이터를 FPGA로 보내면, FPGA에서 AI 연산을 수행하고, 결과를 다시 Host로 보내는 흐름입니다.

1-5. 여러분이 해야 할 일 (과제 요약)

이 프로젝트에서 여러분이 수행해야 할 작업은 크게 세 단계입니다:

첫째, 소프트웨어 단계: 양자화(Quantization)를 수행합니다. 원래 32비트 부동소수점(float)으로 되어 있는 모델의 가중치(weight)와 입력(activation)을 **INT8/UINT8 (8비트 정수)**로 변환하는 작업입니다. 이때 정확도(mAP)가 최대한 떨어지지 않도록 multiplier 값을 찾아야 합니다.

둘째, 하드웨어 설계: Verilog HDL로 CNN 가속기 엔진을 설계합니다. MAC(곱셈-누산) 유닛, BRAM 버퍼, 컨트롤러, 탑 모듈을 만들고, RTL 시뮬레이션을 통해 소프트웨어 결과와 하드웨어 결과가 일치하는지 검증합니다.

셋째, FPGA 구현: 설계한 가속기를 IP로 패키징하고, Vivado에서 MicroBlaze 시스템과 통합하여 실제 FPGA 보드에서 동작시킵니다.

1-6. 평가 기준

최종 평가 점수는 다음 공식으로 산출됩니다:

$$\text{Score} = 10^4 / \text{Energy} \times \text{ReLU}(\text{mAP} - 0.2) \times \text{ReLU}(\text{fps} - 5)$$

여기서 mAP는 양자화 후 정확도(0.2 이상이어야 점수가 발생), fps는 초당 처리 프레임(5 이상이어야 점수가 발생), Energy는 소비 전력입니다. 즉, **정확도는 유지하면서, 빠르고, 전력을 적게 쓰는 설계가 좋은 점수를 받습니다.** 이 세 가지가 AI 가속기 설계의 핵심 트레이드오프이기도 합니다.

1-7. 이 프로젝트를 공부하는 마음가짐

이 프로젝트는 소프트웨어(C/C++), 딥러닝(CNN), 디지털 설계(Verilog), FPGA 도구(Vivado/Vitis)를 모두 아우르는 종합 프로젝트입니다. 한 번에 다 이해하려 하지 마시고, 각 차시를 따라가면서 하나씩 쌓아가면 됩니다. 특히 "소프트웨어에서 이 연산이 어떻게 이루어지는지"를 먼저 이해한 후에 "이걸 하드웨어로 어떻게 바꾸는지"를 생각하는 순서가 중요합니다.

2차시: CNN(합성곱 신경망)의 기본 원리

이번 차시에서는 이 프로젝트의 심장인 **CNN(Convolutional Neural Network)**의 핵심 연산들을 하나하나 뜯어봅니다. 하드웨어를 설계하려면 "이 연산이 정확히 어떤 숫자 계산인지"를 꿰뚫어야 하기 때문에, 수학적으로 차근차근 설명하겠습니다.

2-1. 합성곱(Convolution)이란?

합성곱은 CNN에서 가장 핵심적인 연산입니다. 쉽게 말하면 "작은 필터(커널)를 이미지 위에서 슬라이딩시키면서, 필터와 겹치는 영역의 값들을 원소별로 곱한 뒤 모두 더하는 것"입니다.

구체적으로 Layer 0을 예로 들어보겠습니다. 입력은 **256×256×3** 크기의 RGB 이미지입니다. 여기서 3은 R, G, B 세 개의 채널을 의미합니다.

이 입력에 **3×3×3 크기의 필터** 한 개를 적용한다고 생각해 봅시다. 이 필터는 가로 3, 세로 3, 채널 3이니까 총 27개의 가중치(weight) 숫자를 갖고 있습니다. 이 필터를 입력 이미지의 왼쪽 위 구석에 올려놓고, 겹치는 27개의 위치에서 각각 (입력값 × 필터값)을 계산한 후 27개를 전부 더합니다. 그 합이 출력의 한 픽셀이 됩니다.

그런 다음 필터를 오른쪽으로 한 칸 이동하고 같은 계산을 반복합니다. 한 줄이 끝나면 다음 줄로 내려갑니다. 이렇게 256×256번 슬라이딩하면, **256×256 크기의 출력 feature map** 하나가 만들어집니다.

Layer 0에는 이런 필터가 16개 있습니다. 각 필터가 서로 다른 특징을 감지하도록 학습되어 있어서, 16개의 필터를 적용하면 **256×256×16** 크기의 출력이 나옵니다.

2-2. 합성곱의 수학적 표현

강의자료에 나오는 코드를 같이 보면 이해가 쉽습니다:

```
for (k=0; k<K; k++)          // K: 출력 채널(필터 개수)
  for (p=0; p<P; p++)          // P: 출력 높이
    for (q=0; q<Q; q++)        // Q: 출력 너비
      ofm[k][p][q] = 0;
      for (r=0; r<R; r++)      // R: 필터 높이 (3)
        for (s=0; s<S; s++)    // S: 필터 너비 (3)
          for (c=0; c<C; c++)  // C: 입력 채널
            h = p * stride - pad + r;
            w = q * stride - pad + s;
            ofm[k][p][q] += ifm[c][h][w] * filter[k][c][r][s];
ofm[k][p][q] = Activation(ofm[k][p][q] + bias[k]);
```

변수들의 의미를 하나씩 짚어보면: **ifm**은 입력 피쳐 맵(Input Feature Map), **filter**는 가중치, **ofm**은 출력 피쳐 맵(Output Feature Map), **bias**는 편향, **Activation**은 활성화 함수입니다. 가장 안쪽 세 개의 루프(r, s, c)가 필터 하나를 입력과 곱해서 더하는 핵심 연산이고, 바깥쪽 세 개의 루프(k, p, q)는 이것을 모든 출력 위치와 모든 필터에 대해 반복하는 것입니다.

2-3. Stride(보폭)와 Padding(패딩)

Stride는 필터를 이동시키는 보폭입니다. Stride=1이면 한 칸씩, Stride=2이면 두 칸씩 이동합니다. Stride가 2이면 출력 크기가 입력의 절반이 됩니다.

Padding은 입력의 가장자리에 0을 덧붙이는 것입니다. 3×3 필터에 Padding=1을 주면, 입력의 위아래좌우에 0으로 된 한 줄을 추가합니다. 이렇게 하는 이유는 필터가 가장자리까지 제대로 적용되어 **입출력의 가로세로 크기가 같게 유지** 되도록 하기 위해서입니다.

이 프로젝트의 3×3 합성곱 레이어들은 모두 Stride=1, Padding=1이라서, 합성곱 자체만으로는 가로세로 크기가 변하지 않습니다. 크기가 줄어드는 것은 다음에 설명할 Max Pooling이 담당합니다.

출력 크기를 구하는 공식은 이렇습니다: **Output_Size = (Input_Size + 2×Pad - Filter_Size) / Stride + 1**. 예를 들어 256×256 입력에 3×3 필터, Stride=1, Pad=1이면: $(256 + 2 \times 1 - 3) / 1 + 1 = 256$. 크기가 그대로 유지됩니다.

2-4. Bias(편향)와 Activation(활성화 함수)

합성곱 연산이 끝나면 두 가지가 추가로 적용됩니다.

Bias 덧셈: 각 필터(출력 채널)마다 하나의 bias 값이 있습니다. 합성곱 결과의 모든 픽셀에 해당 필터의 bias를 더해줍니다. 이것은 선형대수에서 $y = Wx + b$ 의 b 에 해당합니다.

Activation (활성화 함수): 이 프로젝트에서 사용하는 활성화 함수는 **ReLU (Rectified Linear Unit)**입니다. 수식은 매우 단순합니다: **$f(x) = \max(x, 0)$** . 즉 양수는 그대로 통과시키고, 음수는 0으로 만듭니다. 왜 이런 게 필요할까요? 합성곱은 본질적으로 선형(linear) 연산입니다. 선형 연산만 아무리 반복해도 결국 하나의 선형 변환과 같습니다. ReLU 같은 비선형 함수를 중간에 끼워 넣어야 네트워크가 복잡한 패턴을 학습할 수 있게 됩니다.

하드웨어 관점에서 ReLU는 아주 구현이 간단합니다. 최상위 비트(MSB, 부호 비트)가 1이면(음수이면) 출력을 0으로, 아니면 그대로 출력하면 됩니다. 비교기(comparator) 하나면 충분하죠.

2-5. Max Pooling(최대 풀링)

Max Pooling은 feature map의 공간적 크기를 줄이는 연산입니다. 이 프로젝트에서는 2×2 커널을 사용합니다.

동작은 간단합니다: 출력 feature map을 2×2 영역씩 나누고, 각 영역에서 가장 큰 값 하나만 취합니다. 예를 들어:

입력 4×4:		Stride=2로 풀링하면 → 출력 2×2:
1 0 2 3		6 8
4 6 6 8	→	3 4
3 1 1 0		
1 2 2 4		

왼쪽 위 2×2 영역 {1, 0, 4, 6}에서 최댓값은 6, 오른쪽 위 {2, 3, 6, 8}에서 최댓값은 8, 이런 식입니다.

Stride=2 풀링은 가로세로 크기를 정확히 절반으로 줄입니다. 이 프로젝트에서 Layer 1, 3, 5, 7, 9가 Stride=2 Max Pooling입니다.

Stride=1 풀링은 Layer 11에서 사용되며, 이 경우 크기는 줄어들지 않고 각 위치에서 주변 2×2 중 최댓값을 취하는 일종의 "스무딩" 효과를 줍니다.

하드웨어 관점에서 Max Pooling은 비교기(comparator)만 있으면 됩니다. 곱셈이 필요 없어서 합성곱보다 훨씬 가볍습니다.

2-6. Route Layer(라우팅 레이어)

Route Layer는 연산을 수행하지 않습니다. 단순히 다른 레이어의 출력을 가져오거나 합치는 역할입니다.

Layer 16 (route 12): Layer 12의 출력(8×8×256)을 그대로 가져와서 Layer 17의 입력으로 사용합니다. 일종의 "분기점"입니다. 네트워크가 중간에서 갈라져서 두 가지 경로로 나뉘는 것이죠.

Layer 19 (route 18, 8): Layer 18의 출력(16×16×128)과 Layer 8의 출력(16×16×256)을 채널 방향으로 연결(concatenation)합니다. 결과는 $16 \times 16 \times (128 + 256) = 16 \times 16 \times 384$ 가 됩니다. 이것은 YOLOv2에서 다양한 해상도의 특징을 결합해서 작은 물체도 잘 잡기 위한 설계입니다.

하드웨어 관점에서 Route Layer는 메모리 주소 관리만 잘 하면 됩니다. 실제 연산은 없고, 어느 메모리에서 데이터를 읽어올지를 제어하는 것이 핵심입니다.

2-7. Upsample Layer(업샘플링)

Upsample은 Max Pooling의 반대입니다. feature map의 크기를 키웁니다. 이 프로젝트에서 Layer 18은 8×8×128을 16×16×128으로 2배 확대합니다.

방법은 간단합니다. 각 픽셀을 2×2 블록으로 복제합니다. 값 하나가 네 칸을 차지하게 되는 것이죠. 이렇게 해서 작은 feature map에서 추출한 "전체적인 특징"을 큰 해상도로 올려서, 앞서 설명한 Route Layer에서 고해상도 feature map과 합칠 수 있게 합니다.

2-8. YOLO Layer

Layer 15와 Layer 21이 YOLO 레이어입니다. 이 레이어는 합성곱의 결과를 받아서 **최종적으로 바운딩 박스 좌표, 객체 존재 확률, 클래스 확률**을 해석하는 후처리 단계입니다. 시그모이드(sigmoid), 소프트맥스(softmax) 같은 비교적 복잡한 수학 함수가 사용되는데, 이 부분은 보통 Host PC(소프트웨어)에서 처리하므로 여러분이 하드웨어로 구현할 필요는 없습니다. 하드웨어 가속기는 합성곱 연산까지만 담당하고, YOLO 후처리는 MicroBlaze CPU나 Host에서 수행합니다.

2-9. 합성곱이 왜 이미지 인식에 효과적인가? (직관적 이해)

여기서 잠깐 한 발 물러서서, 왜 이런 구조가 효과적인지 생각해 봅시다.

첫 번째 레이어의 필터들은 **가장자리(edge)**, **색상 변화**, **질감** 같은 저수준 특징을 감지합니다. 마치 사람의 눈이 먼저 윤곽선을 파악하는 것과 비슷하죠. 중간 레이어들은 이런 저수준 특징들을 조합해서 **모서리**, **원**, **패턴** 같은 중간수준 특징을 만들어냅니다. 마지막 레이어에 가면 "이것은 코카콜라 병이다", "이것은 과자 봉지이다" 같은 **고수준 의미**를 포착할 수 있게 됩니다.

이때 Max Pooling이 크기를 계속 줄여주기 때문에, 깊은 레이어로 갈수록 같은 3×3 필터가 커버하는 원본 이미지의 영역(receptive field)이 점점 넓어집니다. 처음에는 3×3 픽셀만 보던 것이, 풀링을 거치면 실질적으로 6×6 , 12×12 ... 점점 넓은 영역의 정보를 담게 되는 것입니다.

2-10. 하드웨어 설계자의 눈으로 보기

이제 이 연산들을 하드웨어 관점에서 정리해 봅시다.

합성곱이 전체 연산량의 대부분을 차지합니다. Layer 0 하나만 봐도, 출력 픽셀 하나를 만들려면 $3 \times 3 \times 3 = 27$ 번의 곱셈과 26번의 덧셈이 필요하고, 이런 픽셀이 $256 \times 256 \times 16 = 1,048,576$ 개입니다. 전체 네트워크에서의 총 연산량은 약 1.035 GFLOPS(10억 번의 부동소수점 연산)에 달합니다.

따라서 하드웨어 가속기의 핵심은 **MAC(Multiply-Accumulate, 곱셈-누산) 유닛을 얼마나 많이, 얼마나 효율적으로 병렬 배치하느냐**입니다. 나중에 9차시에서 자세히 다루겠지만, 기본 아이디어는 여러 개의 곱셈을 동시에 수행하고 결과를 더하는 것입니다.

반면 Max Pooling, Route, Upsample은 상대적으로 가벼운 연산이라 하드웨어 구현이 간단합니다. 하지만 **데이터 이동(data movement)**은 복잡할 수 있습니다. 특히 Route Layer처럼 이전 레이어의 결과를 나중에 다시 참조해야

하는 경우, 중간 결과를 어디에 저장해둘지가 설계의 중요한 과제가 됩니다.

2차시 핵심 정리:

- 1. 합성곱 = 필터를 슬라이딩하며 원소별 곱 + 누적합 → 전체 연산의 핵심
- 2. Bias + ReLU = 편향 더하기 + 음수를 0으로 처리
- 3. Max Pooling = 2×2 중 최댓값 → 크기 절반으로 축소
- 4. Route = 다른 레이어의 출력을 가져오거나 합침 (연산 없음)
- 5. Upsample = 픽셀 복제로 크기 2배 확대
- 6. YOLO = 최종 후처리 (소프트웨어 담당)

3차시: AIX2024 모델 — 22개 레이어 완전 해부

이제 실제 네트워크의 전체 구조를 레이어별로 하나씩 살펴봅시다. 이 표를 완전히 이해하면 나중에 하드웨어 설계할 때 "이 레이어에서 메모리가 얼마나 필요하지?", "MAC 유닛을 몇 번 반복해야 하지?" 같은 질문에 바로 답할 수 있게 됩니다.

3-1. 전체 레이어 테이블

강의자료에 나오는 네트워크 구조표를 정리하면 다음과 같습니다:

Layer	Type	Filters	Size	Input	Output	비고
0	conv	16	3×3/1	256×256× 3	→ 256×256× 16	BF
1	max		2×2/2	256×256× 16	→ 128×128× 16	
2	conv	32	3×3/1	128×128× 16	→ 128×128× 32	BF
3	max		2×2/2	128×128× 32	→ 64× 64× 32	
4	conv	64	3×3/1	64× 64× 32	→ 64× 64× 64	BF
5	max		2×2/2	64× 64× 64	→ 32× 32× 64	
6	conv	128	3×3/1	32× 32× 64	→ 32× 32×128	BF
7	max		2×2/2	32× 32×128	→ 16× 16×128	
8	conv	256	3×3/1	16× 16×128	→ 16× 16×256	BF

9	max		2×2/2	16× 16×256	→	8× 8×256	
10	conv	512	3×3/1	8× 8×256	→	8× 8×512	BF
11	max		2×2/1	8× 8×512	→	8× 8×512	
★Stride=1							
12	conv	256	1×1/1	8× 8×512	→	8× 8×256	BF (1×1 conv!)
13	conv	512	3×3/1	8× 8×256	→	8× 8×512	BF
14	conv	195	1×1/1	8× 8×512	→	8× 8×195	BF (1×1 conv!)
15	yolo						출력①
16	route 12			8× 8×256	→	8× 8×256	분기
17	conv	128	1×1/1	8× 8×256	→	8× 8×128	BF (1×1 conv!)
18	upsample	2×		8× 8×128	→	16× 16×128	
19	route 18, 8				→	16× 16×384	연결
20	conv	195	1×1/1	16× 16×384	→	16× 16×195	BF (1×1 conv!)
21	yolo						출력②

(BF = Batch Normalization이 합성곱에 fuse되어 있음)

3-2. 패턴을 읽어보자

이 테이블에는 명확한 패턴이 있습니다.

패턴 1: Conv-Pool 쌍. Layer 0~9까지는 "합성곱 → Max Pooling"이 반복됩니다. 합성곱은 채널 수를 늘리고(특징을 더 다양하게 추출), Max Pooling은 공간 크기를 절반으로 줄입니다. 이렇게 하면 점점 더 추상적이고 압축된 특징이 만들어집니다.

채널 수 변화를 보면: 3 → 16 → 32 → 64 → 128 → 256 → 512. 대략 2배씩 증가합니다.

공간 크기 변화를 보면: 256 → 128 → 64 → 32 → 16 → 8. 정확히 2배씩 감소합니다.

패턴 2: 1×1 합성곱. Layer 12, 14, 17, 20은 필터 크기가 1×1입니다. 1×1 합성곱은 공간적으로는 아무것도 하지 않고, 오직 **채널 간의 조합**만 수행합니다. 예를 들어 512채널을 256채널로 줄이는 것은, 512개의 특징을 256개의 새로운 조합으로 재편하는 것과 같습니다. 연산량이 3×3보다 9분의 1이라 매우 효율적입니다. 하드웨어에서는 3×3 MAC 대신 1×1 MAC만 필요하므로 제어가 다릅니다.

패턴 3: 두 갈래 출력(Multi-scale Detection). Layer 14에서 첫 번째 YOLO 출력(8×8×195)이 나오고, Layer 20

에서 두 번째 YOLO 출력($16 \times 16 \times 195$)이 나옵니다. 8×8 격자는 큰 물체를, 16×16 격자는 작은 물체를 검출하는 데 유리합니다. 이것이 YOLOv2의 핵심 아이디어 중 하나인 **다중 스케일 검출(multi-scale detection)**입니다.

3-3. 각 레이어의 연산량 감 잡기

하드웨어 설계자에게 가장 중요한 것은 어느 레이어가 가장 무거운가입니다.

합성곱의 연산량(MAC 연산 횟수)은 $\text{출력높이} \times \text{출력너비} \times \text{출력채널} \times \text{필터높이} \times \text{필터너비} \times \text{입력채널}$ 으로 계산됩니다.

Layer 0: $256 \times 256 \times 16 \times 3 \times 3 \times 3 = \mathbf{28,311,552}$ MAC 연산

Layer 8: $16 \times 16 \times 256 \times 3 \times 3 \times 128 = \mathbf{75,497,472}$ MAC 연산

Layer 10: $8 \times 8 \times 512 \times 3 \times 3 \times 256 = \mathbf{75,497,472}$ MAC 연산

Layer 0은 공간이 크지만 채널이 적고, Layer 8이나 10은 공간이 작지만 채널이 많습니다. 결과적으로 중간~후반 레이어의 연산량이 더 크거나 비슷합니다. 하지만 **데이터 크기(메모리)** 면에서는 Layer 0이 가장 큼니다 ($256 \times 256 \times 16 = 1\text{MB}$ 이상의 feature map).

이 트레이드오프가 하드웨어 설계의 핵심 도전입니다: 초반 레이어는 데이터가 크고, 후반 레이어는 연산이 무겁습니다. 하나의 가속기로 모든 레이어를 효율적으로 처리해야 합니다.

3-4. 데이터 흐름 — 네트워크를 따라가 보기

소프트웨어 코드(`yolov2_forward_network_q` 함수)에서 네트워크가 어떻게 실행되는지 보면:

```
for (i = 0; i < net.n; ++i) {           // 22개 레이어를 순서대로
    layer l = net.layers[i];

    if (l.type == CONVOLUTIONAL) forward_convolutional_layer_q(...);
    else if (l.type == MAXPOOL)   forward_maxpool_layer_cpu(...);
    else if (l.type == ROUTE)     forward_route_layer_cpu(...);
    else if (l.type == UPSAMPLE)  forward_upsample_layer_cpu(...);
    else if (l.type == YOLO)      forward_yolo_layer_cpu(...);

    state.input = l.output; // 이번 레이어의 출력이 다음 레이어의 입력이 됨
}
```

핵심은 마지막 줄입니다: 이번 레이어의 출력이 다음 레이어의 입력이 됩니다. 이것이 "순전파(forward propagation)"입니다. 데이터가 물 흐르듯 앞으로만 흘러갑니다. 하드웨어에서도 이 순서를 정확히 따라야 합니다.

단, Route Layer에서 주의할 점이 있습니다. Layer 19는 Layer 18과 Layer 8의 결과를 합쳐야 하므로, **Layer 8의 출력을 나중에 다시 사용하기 위해 메모리에 보관해** 두어야 합니다. 하드웨어에서는 이 중간 결과를 외부 DDR 메모리에 저장했다가 필요할 때 다시 읽어오는 방식을 사용합니다.

3-5. 하드웨어가 처리할 범위와 소프트웨어가 처리할 범위

정리하면, 여러분이 설계할 하드웨어 가속기는 주로 **합성곱(Conv) + Bias + ReLU + Max Pooling**을 담당합니다. 이것이 전체 연산량의 99% 이상을 차지합니다.

Route, Upsample은 하드웨어에서도 구현할 수 있지만, 본질적으로는 데이터 이동 문제이므로 컨트롤러(FSM)의 메모리 주소 제어로 해결됩니다.

YOLO 레이어의 후처리(sigmoid, softmax, NMS 등)는 MicroBlaze CPU나 Host PC에서 소프트웨어로 처리합니다.

3차시 핵심 정리:

1. 22개 레이어는 Conv-Pool 반복 → 1×1 Conv → Multi-scale 출력의 구조
 2. 채널 수는 점점 늘어나고(3→512), 공간 크기는 점점 줄어듦(256→8)
 3. 1×1 합성곱은 채널 조합용, 3×3 합성곱은 공간 특징 추출용
 4. 연산량의 대부분은 합성곱이 차지 → 하드웨어 가속의 핵심 타겟
 5. Route Layer 때문에 중간 결과를 메모리에 보관해야 함
-

보충 1: ReLU가 왜 필요한가? — 진짜 이유

선형 연산의 한계부터 이해하자

합성곱 연산의 본질은 곱하고 더하기(Multiply and Add)입니다. 이것은 수학적으로 선형 변환(Linear Transformation)에 해당합니다.

선형 변환의 특징은 무엇일까요? 아무리 여러 번 반복해도, 결과는 항상 하나의 선형 변환으로 압축됩니다. 예를 들어볼게요.

$$\begin{aligned} y &= 3x && \text{(첫 번째 선형 변환)} \\ z &= 5y = 5 \times 3x = 15x && \text{(두 번째 선형 변환)} \end{aligned}$$

3을 곱하고 나서 5를 곱한 것은, 처음부터 15를 곱한 것과 완전히 동일합니다. 100개의 레이어를 쌓아도 마찬가지입니다. 선형 변환 100번은 선형 변환 1번과 수학적으로 같은 표현력밖에 없습니다.

이것을 이미지 인식에 적용해 보면, 선형 변환만으로는 직선 하나로 구분할 수 있는 패턴만 인식할 수 있습니다. "이 픽셀이 밝으면 고양이, 어두우면 개" 같은 단순한 판단은 가능하지만, 현실의 이미지는 그렇게 단순하지 않죠.

비선형이 필요한 이유 — 구체적 예시

실제 객체 인식을 생각해 봅시다. 코카콜라 병을 인식하려면 다음과 같은 복합적 판단이 필요합니다:

☐ "빨간색 영역이 있다" 그리고 "특정 곡선 형태가 있다" 그리고 "흰색 글자 패턴이 있다"

이런 "그리고(AND)" 조건의 조합은 본질적으로 비선형입니다. 두 가지 특징이 동시에 존재할 때만 반응하고, 하나만 있으면 반응하지 않아야 하는데, 이건 직선 하나로는 표현이 불가능합니다.

ReLU를 넣으면 어떻게 될까요? $f(x) = \max(0, x)$ 는 이런 효과를 냅니다:

특징 A의 값: +5 → ReLU 통과 후: 5 (살아남음)
특징 B의 값: -3 → ReLU 통과 후: 0 (죽음)

음수 값을 0으로 만든다는 것은, "이 필터가 감지하려는 패턴이 없다"는 신호를 깔끔하게 차단하는 것입니다. 이 덕분에 다음 레이어에서는 "존재하는 특징들만"을 조합할 수 있게 됩니다. 없는 특징(음수값)이 다음 레이어의 결과를 엉뚱하게 왜곡하는 것을 방지하는 거죠.

시각적으로 이해하기

2차원 평면에 빨간 점과 파란 점이 섞여 있다고 상상해 보세요.

선형 변환만 있는 경우: 직선 하나로만 빨간 점과 파란 점을 구분해야 합니다. 만약 점들이 원형으로 분포되어 있으면(빨간 점이 가운데, 파란 점이 바깥) 직선으로는 절대 구분할 수 없습니다.

ReLU가 들어가면: 공간을 "접을" 수 있게 됩니다. 음수 영역을 0으로 눌러서 공간 자체를 비틀 수 있고, 이렇게 비틀린 공간에서는 직선으로 구분이 가능해집니다. 레이어를 여러 개 쌓으면 공간을 점점 더 복잡하게 접고 펼 수 있어서, 아무리 복잡한 패턴도 분리할 수 있게 됩니다.

왜 하필 ReLU인가?

비선형 함수는 sigmoid, tanh 등 여러 가지가 있는데, ReLU가 가장 널리 쓰이는 이유는 세 가지입니다.

첫째, 계산이 극도로 단순합니다. $\max(0, x)$ 는 비교 한 번이면 끝입니다. sigmoid는 지수함수(exp)를 계산해야 하는데, 이건 하드웨어로 구현하기 매우 비쌉니다.

둘째, 기울기 소실(gradient vanishing) 문제가 적습니다. 학습(training) 과정에서 sigmoid의 기울기는 0~0.25 사이인데, 이걸 여러 레이어에 걸쳐 곱하면 기울기가 0에 가까워져서 학습이 안 됩니다. ReLU는 양수 영역에서 기울기가 항상 1이라 이 문제가 없습니다.

셋째, 하드웨어 구현이 극도로 간단합니다. 이진수에서 최상위 비트(MSB)는 부호 비트입니다. MSB가 1이면 음수이므로 출력을 0으로, MSB가 0이면 양수이므로 그대로 통과시키면 됩니다. Verilog로는 이렇게 됩니다:

```
assign relu_out = (data_in[MSB] == 1'b1) ? 0 : data_in;
```

딱 한 줄입니다. 추가 하드웨어 자원이 거의 필요 없습니다.

보충 2: Route Layer와 Upsample Layer가 왜 필요한가?

문제의 출발점 — "작은 물체를 놓친다"

이 부분을 이해하려면, 먼저 네트워크에서 벌어지는 일을 시각적으로 따라가 봐야 합니다.

입력 이미지가 256×256에서 시작해서 Layer 9(Max Pooling)을 거치면 8×8까지 줄어듭니다. 8×8이라는 것은 원본 이미지를 **32×32 픽셀 크기의 블록 64개로 쪼갠 것과 같습니다.**

만약 편의점 선반 사진에 작은 사탕이 있다고 합시다. 이 사탕이 원본 이미지에서 20×20 픽셀 정도의 크기라면, 8×8 격자에서는 **하나의 셀 안에 완전히 묻혀버립니다.** 32×32 블록 안에 20×20 사탕이 들어가면, 그 블록의 대부분은 배경이고 사탕 정보는 희미해지죠. 이러면 검출이 어렵습니다.

반대로 16×16 격자라면? 원본 이미지를 16×16 픽셀 크기의 블록 256개로 쪼갠 것이고, 20×20 사탕은 여러 셀에 걸쳐서 나타납니다. 이 해상도에서는 사탕을 잡아낼 확률이 높아집니다.

그래서 YOLOv2는 **두 가지 해상도에서 동시에 검출합니다:**

- ☐ 8×8 격자 → 큰 물체 검출 (Layer 14 → YOLO Layer 15)
- ☐ 16×16 격자 → 작은 물체 검출 (Layer 20 → YOLO Layer 21)

Route Layer의 역할 — "과거의 기억을 불러오기"

그런데 여기서 문제가 생깁니다. 네트워크가 Layer 0부터 Layer 14까지 쪽 내려오면서 8×8까지 줄었는데, 16×16 격자에서의 검출을 하려면 어떻게 해야 할까요? 이미 줄어든 feature map을 다시 키워야 합니다.

단순히 8×8을 16×16으로 키우면(Upsample) 정보가 매우 뭉뚱해집니다. 1개의 값을 4개로 복사한 것에 불과하니까요. 여기에 **예전에 16×16이었을 때의 세밀한 정보를 합쳐주면 훨씬 풍부해집니다.**

이것이 Route Layer의 역할입니다. 구체적으로 보면:

Layer 16 (route 12): Layer 12의 출력(8×8×256)을 가져옵니다. 왜 하필 Layer 12일까요? Layer 12는 1×1 합성곱으로 512채널을 256채널로 압축한 것인데, 이것을 별도의 경로로 빼서 "16×16 검출 경로"의 시작점으로 사용합니다.

Layer 17 (conv 1×1): 이 256채널을 128채널로 더 압축합니다.

Layer 18 (upsample): 8×8×128을 16×16×128으로 키웁니다.

Layer 19 (route 18, 8): 여기가 핵심입니다. Layer 18의 업샘플된 결과(16×16×128)와 **Layer 8의 출력**

(16×16×256)을 합칩니다. Layer 8은 네트워크 초반부에서 아직 16×16 해상도를 유지하고 있었을 때의 출력입니다. 이 출력에는 세밀한 공간 정보(edge, texture 등)가 살아있습니다.

이 둘을 합치면 $16 \times 16 \times (128 + 256) = 16 \times 16 \times 384$ 가 됩니다. 이 안에는:

- ☐ 깊은 레이어에서 온 **의미적(semantic) 정보** (128채널): "이 영역은 음료수 같다"
- ☐ 얇은 레이어에서 온 **공간적(spatial) 정보** (256채널): "여기에 둥근 윤곽이 있다"

이 두 정보가 결합되어 작은 물체도 "무엇이 어디에 있는지" 정확하게 판단할 수 있게 됩니다.

Upsample의 역할 – "해상도 맞추기"

Upsample이 필요한 이유는 단순합니다: 두 **feature map**을 합치려면 크기가 같아야 하기 때문입니다.

Layer 17의 출력은 8×8이고, Layer 8의 출력은 16×16입니다. 이 두 개를 채널 방향으로 연결하려면 가로세로 크기가 일치해야 합니다. 그래서 8×8을 16×16으로 키워주는 것이 Upsample의 역할입니다.

Upsample 방식 자체는 단순히 각 픽셀을 2×2로 복사하는 것이라 정보 손실이 있지만, 어차피 Route에서 Layer 8의 세밀한 정보를 합칠 것이기 때문에 괜찮습니다. Upsample은 "대략적인 의미 정보를 넓은 해상도로 펼치는" 역할이고, 세밀한 디테일은 Layer 8에서 가져옵니다.

비유로 정리

이것을 비유하면 이렇습니다:

사진을 아주 작게 축소해서 보면(8×8) 전체적인 구성은 파악되지만 디테일은 사라집니다. "왼쪽에 뭔가 빨간 게 있네" 정도만 알 수 있죠.

원본 크기에 가까운 사진을 보면(16×16) 디테일은 보이지만, 그것이 무엇인지 맥락적으로 판단하기는 어려울 수 있습니다. "여기 둥그란 것이 있는데, 이게 사과인지 야구공인지?"

Route Layer는 이 두 관점을 합칩니다: 축소판에서 얻은 "이건 음료수야"라는 이해와 원본에 가까운 해상도에서 얻은 "여기에 이런 모양이 있어"라는 디테일을 결합하는 것입니다.

보충 3: 합성곱이 이미지 인식에 왜 효과적인가? (2-9 재설명)

핵심 아이디어: 세 가지 원리

합성곱이 이미지 인식에 강력한 이유는 세 가지 근본적인 원리에 있습니다.

원리 1: 지역적 연결성 (Local Connectivity)

사람이 사진을 볼 때, 이미지의 모든 픽셀을 동시에 하나하나 비교하지 않습니다. 먼저 특정 영역의 윤곽선을 보고, 색상 변화를 감지하고, 질감을 파악합니다. 이 모든 것은 **작은 영역(local region)** 안에서의 패턴입니다.

합성곱의 3×3 필터가 정확히 이것을 합니다. 전체 이미지를 한 번에 보는 것이 아니라, **한 번에 $3 \times 3 = 9$ 개 픽셀의 관계**만 살핍니다. "이 9개 픽셀이 수직선 패턴인가?", "수평 그라데이션인가?", "모서리인가?" 같은 질문을 던지는 겁니다.

만약 이미지의 모든 픽셀이 서로 연결되어 있는 **Fully Connected(완전 연결)** 방식을 쓴다면 어떨까요? $256 \times 256 \times 3 = 196,608$ 개 입력 $\times$ $196,608$ 개 출력이면 약 **386억 개의 가중치**가 필요합니다. 반면 $3 \times 3 \times 3$ 필터 16개는 **432개의 가중치**면 충분합니다. 차이가 8천만 배입니다. 적은 파라미터로도 의미 있는 패턴을 찾을 수 있다는 것이 지역적 연결성의 위력입니다.

원리 2: 가중치 공유 (Weight Sharing)

합성곱의 또 다른 강력한 특성은, **같은 필터를 이미지의 모든 위치에 동일하게 적용**한다는 것입니다.

왜 이게 좋을까요? "수직 윤곽선을 감지하는 필터"가 하나 있다고 합시다. 이 필터는 이미지의 왼쪽 위에서도, 오른쪽 아래에서도, 한가운데에서도 동일하게 수직 윤곽선을 감지합니다. 수직 윤곽선이라는 개념은 위치에 상관없이 같은 것이니까, 필터도 하나만 있으면 충분합니다.

이것은 **위치 불변성(Translation Invariance)**이라고 합니다. 코카콜라 병이 이미지의 왼쪽에 있든 오른쪽에 있든, 같은 필터가 같은 특징을 감지합니다. 만약 위치마다 다른 가중치를 사용한다면, "왼쪽 위의 코카콜라"와 "오른쪽 아래의 코카콜라"를 따로따로 학습해야 합니다. 데이터도 훨씬 많이 필요하고, 효율도 나쁩니다.

원리 3: 계층적 특징 추출 (Hierarchical Feature Extraction)

이것이 가장 강력한 원리입니다. CNN은 레이어를 쌓으면서 **점점 더 복잡한 특징**을 자동으로 학습합니다.

구체적으로 이 프로젝트의 네트워크에서 어떤 일이 벌어지는지 따라가 봅시다:

Layer 0 (첫 번째 합성곱, 필터 16개): 각 필터는 $3 \times 3 \times 3 = 27$ 개의 가중치를 갖습니다. 학습이 끝나면, 이 16개 필터는 각각 다른 기본 패턴을 감지하게 됩니다. 예를 들어:

- ☐ 필터 0: 수평 엣지 (위는 밝고 아래는 어두운 경계)
- ☐ 필터 1: 수직 엣지
- ☐ 필터 2: 45도 대각선 엣지
- ☐ 필터 3: 빨간색→초록색 변화
- ☐ 필터 4: 밝은 점(blob)
- ☐ ... 등등

이 16개의 출력 feature map 각각은 "이 위치에 해당 패턴이 얼마나 강하게 존재하는가"를 나타내는 256×256 크기의 지도입니다.

Layer 1 (Max Pooling): 256×256 을 128×128 로 줄입니다. 이 과정에서 정확한 위치 정보는 조금 잃지만, "이 영역 어딘가에 수평 엣지가 있다"라는 정보는 보존됩니다.

Layer 2 (두 번째 합성곱, 필터 32개): 이제 입력이 $128 \times 128 \times 16$, 즉 **16개의 기본 패턴 지도**입니다. 여기에 $3 \times 3 \times 16$ 필터를 적용하면, 이 필터는 **기본 패턴들의 조합**을 학습합니다. 예를 들어:

- ☐ 필터 0: "수직 엣지 + 수평 엣지가 만나는 곳" = **모서리(corner)**
- ☐ 필터 1: "대각선 엣지가 연속되는 곳" = **곡선(curve)**
- ☐ 필터 2: "밝은 점 주위에 빨간색 변화가 있는 곳" = **빨간 하이라이트**

즉, 첫 번째 레이어의 "선(edge)" 조합이 두 번째 레이어에서 "도형(shape)"이 되는 겁니다.

Layer 4~8 (더 깊은 합성곱): 이 과정이 반복되면서:

- ☐ 도형의 조합 → **텍스처(질감)**나 **부분 구조(part)**
- ☐ 부분 구조의 조합 → **물체의 일부** (예: 병뚜껑, 라벨)
- ☐ 물체 일부의 조합 → **완전한 물체** (예: 코카콜라 병)

Layer 10~14 (최종 레이어): 이 시점에서 각 채널은 매우 추상적인 개념을 담고 있습니다. 예를 들어 512개 채널 중 하나는 "음료수 병 형태가 존재하는 정도", 다른 하나는 "과자 봉지 패턴이 존재하는 정도"를 나타낼 수 있습니다.

Receptive Field(수용 영역)의 확장

이 계층적 구조에서 중요한 개념이 **Receptive Field**입니다. 각 레이어에서 필터는 3×3 만 보지만, Max Pooling으로 크기가 줄었기 때문에, 실제로 원본 이미지에서 커버하는 영역은 점점 넓어집니다.

구체적으로 계산해 볼게요:

- ☐ **Layer 0의 출력 픽셀 하나:** 원본 이미지의 3×3 영역을 봅니다.
- ☐ **Layer 1(Pooling) 후:** 2×2를 하나로 합치므로, 이 하나의 값은 원본의 대략 6×6 영역 정보를 담습니다.
- ☐ **Layer 2의 출력 픽셀 하나:** Layer 1 결과의 3×3을 보는데, Layer 1의 각 값이 원본 6×6을 대표하므로, 결과적으로 원본의 약 14×14 영역을 봅니다.
- ☐ **이것이 계속 반복되면...**
- ☐ **Layer 10의 출력 픽셀 하나:** 원본 이미지의 거의 전체 영역의 정보가 응축되어 있습니다.

즉, 필터 크기는 항상 3×3이지만, **깊은 레이어로 갈수록 원본 이미지의 더 넓은 맥락을 고려한 판단**을 하게 됩니다. 초반에는 "이 3개 픽셀이 밝기 변화가 있나?"를 보고, 후반에는 "이 영역 전체가 코카콜라 병 패턴인가?"를 보는 것입니다.

Fully Connected와의 비교 — 왜 CNN이 이겼나

과거에는 이미지 인식에 Fully Connected Network(완전 연결 신경망)를 사용했습니다. 모든 입력 픽셀이 모든 출력 뉴런에 연결되는 방식이죠. 하지만 이 방식은 세 가지 치명적 문제가 있었습니다:

파라미터가 너무 많습니다. 256×256×3 입력을 1000개 뉴런에 연결하면 약 2억 개의 가중치가 필요합니다. 이렇게 많은 파라미터를 학습하려면 데이터도 엄청나게 많아야 하고, 과적합(overfitting) 위험도 큼니다.

공간 구조를 무시합니다. 이미지를 1차원 벡터로 펼쳐서 입력하기 때문에, "인접한 픽셀은 비슷한 정보를 담고 있다"는 이미지의 본질적 특성을 전혀 활용하지 못합니다.

위치가 바뀌면 인식을 못합니다. 왼쪽 위에 있는 고양이를 학습한 것과 오른쪽 아래에 있는 고양이를 학습한 것이 완전히 별개의 패턴으로 취급됩니다.

CNN은 이 세 가지를 모두 해결합니다. 가중치 공유로 파라미터가 극적으로 줄고, 3×3 필터로 공간 구조를 활용하고, 같은 필터를 모든 위치에 적용하므로 위치 불변성이 자연스럽게 확보됩니다.

하드웨어 설계자의 관점에서 다시 보면

이 세 가지 원리가 하드웨어에 주는 의미도 있습니다:

지역적 연결성 → 3×3 = 9개의 곱셈만 하면 됩니다. Fully Connected라면 수십만 개의 곱셈이 필요하지만, CNN은 한 출력 채널에 대해 9×입력채널 수 정도만 하면 됩니다. MAC 유닛 설계가 현실적으로 가능해집니다.

가중치 공유 → 필터의 가중치를 한 번만 메모리에 로드하면, 모든 위치에서 재사용할 수 있습니다. 이것은 하드웨어에서 **데이터 재사용(data reuse)**이라 불리며, 메모리 접근을 크게 줄여줍니다. 메모리 접근은 하드웨어에서 전력 소모의 주범이므로, 이 특성이 에너지 효율을 높이는 핵심입니다.

계층적 구조 → 모든 레이어가 같은 종류의 연산(3×3 합성곱)을 하므로, **하나의 가속기 엔진을 레이어마다 재사용**할 수 있습니다. 레이어마다 다른 하드웨어를 만들 필요가 없고, 하나의 엔진에 다른 가중치와 설정값만 넣어주면 됩니다.

4차시: 소프트웨어 코드 구조 — 컴파일, 실행, 디렉토리 이해

이번 차시에서는 실제 프로젝트의 파일들이 어떻게 구성되어 있고, 어떻게 빌드하고 실행하는지, 그리고 각 소스 파일이 어떤 역할을 하는지를 살펴봅니다. "코드를 손대기 전에 지도부터 그리자"는 차시입니다.

4-1. 프로젝트의 두 개의 큰 덩어리

이 프로젝트는 크게 **소프트웨어(skeleton)**와 **하드웨어(yolohw)** 두 개의 폴더로 나뉩니다. 이번 차시에서는 소프트웨어 쪽을 집중적으로 보고, 하드웨어 쪽은 8차시 이후에 다룹니다.

프로젝트 전체

```
├─ skeleton/      ← 소프트웨어 (C/C++, 양자화, 추론)
└─ yolohw/        ← 하드웨어 (Verilog HDL, 테스트벤치, 시뮬레이션 데이터)
```

4-2. 소프트웨어 디렉토리 상세 구조

`skeleton/` 폴더 안의 구조를 하나씩 뜯어보겠습니다.

src/ — 소스 코드 (핵심)

여기가 여러분이 실제로 읽고 수정할 파일들이 모인 곳입니다.

main.c — 프로그램의 진입점입니다. 커맨드라인 인자를 파싱하고, 모델을 로드하고, 이미지를 읽어서 추론을 실행하고, 결과를 화면에 출력합니다. `-quantized` 옵션이 있으면 양자화 버전을 실행하고, `-save_params` 옵션이 있으면 양자화된 파라미터를 hex 파일로 저장합니다. 이 파일 자체를 수정할 일은 거의 없지만, **프로그램 실행 흐름을 이해하려면 반드시 한번 읽어봐야** 합니다.

yolov2_forward_network.c — **FP32(32비트 부동소수점)** 버전의 추론 코드입니다. 원본 모델 그대로, 아무런 양자화 없이 추론을 수행합니다. 이것이 **기준선(baseline)**입니다. 이 코드를 실행하면 mAP = 81.76%가 나옵니다. 여러분이 직접 수정할 일은 없지만, "양자화 전의 정상 동작"을 확인하는 참조용입니다.

yolov2_forward_network_quantized.c — 이 파일이 **여러분이 가장 많이 다루게 될 핵심 파일**입니다. INT8 양자화 버전의 추론 코드이며, 세 가지 중요한 함수가 들어있습니다:

- ☐ `do_quantization()` — 양자화 multiplier를 설정하고, 가중치와 bias를 정수로 변환
- ☐ `forward_convolutional_layer_q()` — 양자화된 합성곱 연산을 수행
- ☐ `save_quantized_model()` — 양자화된 파라미터를 hex 파일로 저장 (하드웨어 검증용)

additionally.c / additionally.h — 유틸리티 함수 모음입니다. im2col, 각종 레이어의 forward 함수(maxpool, route, upsample, yolo 등), 이미지 로딩/저장 등이 포함됩니다. 이 파일은 건드릴 필요 없이 "라이브러리"처럼 사용합니다.

box.c / box.h — 바운딩 박스 관련 연산입니다. IoU(Intersection over Union) 계산, NMS(Non-Maximum Suppression) 등 후처리 로직이 담겨있습니다. 역시 수정할 필요 없습니다.

stb_image.h / stb_image_write.h — 이미지 읽기/쓰기 라이브러리입니다. 단일 헤더 파일로 된 유명한 오픈소스 라이브러리이고, 수정 불필요합니다.

bin/ — 실행 파일과 데이터

빌드된 실행 파일, 모델 파라미터, 테스트 이미지, 실행 스크립트가 모두 여기에 있습니다.

aix2024.cfg — 모델의 네트워크 구조를 정의하는 설정 파일입니다. 각 레이어의 타입, 필터 수, 크기, stride, padding 등이 텍스트로 기록되어 있습니다. 앞서 3차시에서 본 22개 레이어 테이블이 이 파일에서 나옵니다.

aix2024.weights — 학습이 완료된 모델의 가중치(weight) 파일입니다. 바이너리 형식이고, 모든 합성곱 레이어의

필터 값, bias 값이 float32로 저장되어 있습니다. 약 100,000장의 상품 이미지로 학습된 결과물입니다.

yolohw.names — 60개 클래스(상품 종류)의 이름 목록입니다. `aunt_jemima_original_syrup`, `band_aid_clear_strips`, `pringles_bbq`, `cheeze_it`, `redbull` 같은 편의점 상품명들이 한 줄에 하나씩 나열되어 있습니다.

yolohw.data — 데이터셋 경로 설정 파일입니다. 테스트 이미지 목록, 클래스 이름 파일의 위치를 지정합니다.

dataset/ — 테스트 이미지 232장이 들어있습니다. 두 종류가 있는데, "long" (상품들 사이 거리가 먼 사진)과 "close" (상품들이 뽕뽕하게 모여있는 사진)입니다. 각 이미지에는 같은 이름의 `.txt` 파일이 짝으로 있어서, 정답 (ground truth)이 [클래스번호 x좌표 y좌표 너비 높이] 형식으로 기록되어 있습니다.

실행 스크립트들:

- ☐ `script-wins-aix2024-test-one.cmd` — Windows에서 이미지 한 장 테스트
- ☐ `script-wins-aix2024-test-all.cmd` — Windows에서 전체 이미지 테스트 (mAP 측정)
- ☐ `script-wins-aix2024-test-one-quantized.cmd` — 양자화 버전으로 한 장 테스트
- ☐ `script-wins-aix2024-test-all-quantized.cmd` — 양자화 버전 전체 테스트
- ☐ Linux/Mac용은 같은 이름에 `.sh` 확장자입니다.

log_feamap/ — 소프트웨어 실행 시 생성되는 **feature map 데이터**입니다. 각 합성곱 레이어의 입력과 출력을 hex 형식으로 저장합니다. 예를 들어 `CONV00_input.hex` 는 Layer 0의 입력, `CONV00_output.hex` 는 Layer 0의 출력입니다. 이 파일들이 나중에 하드웨어 검증의 핵심 역할을 합니다.

log_param/ — 양자화된 파라미터 파일입니다. 각 합성곱 레이어의 양자화된 가중치 (`CONV00_param_weight.hex`), bias (`CONV00_param_biases.hex`), scale (`CONV00_param_scales.hex`)이 저장됩니다. 이 hex 파일들이 하드웨어의 BRAM에 초기값으로 로드됩니다.

3rdparty/ — 서드파티 라이브러리

pthread — 멀티스레딩 라이브러리입니다. 합성곱의 각 출력 채널을 병렬로 계산하기 위해 사용됩니다(코드의 `#pragma omp parallel for` 부분). 직접 다룰 일은 없습니다.

CLBlast — OpenCL 기반 BLAS(Basic Linear Algebra Subprograms) 라이브러리입니다. GPU 가속이 가능한 환경에서 행렬 연산을 빠르게 하기 위한 것인데, 이 프로젝트에서는 CPU만 사용하므로 실질적으로 쓰이지 않습니다.

프로젝트 파일

yolo_cpu.sln / yolo_cpu.vcxproj — Visual Studio 프로젝트 파일입니다. Windows에서 Visual Studio로 열면 바로 빌드할 수 있습니다.

Makefile — Linux/Mac에서 **make** 명령으로 컴파일할 때 사용하는 빌드 설정 파일입니다.

4-3. 컴파일과 실행 방법

Windows에서 (Visual Studio)

1. `skeleton/yolo_cpu.sln`을 Visual Studio 2019 이상에서 엽니다.
2. Build → Clean Solution 클릭
3. Build → Build Solution 클릭 → `bin/` 폴더에 `yolo_cpu.exe`가 생성됩니다.
4. 명령 프롬프트를 열고 `cd C:\skeleton\bin\`으로 이동합니다.
5. 스크립트를 실행합니다.

Linux/Mac에서 (gcc + make)

```
cd skeleton/
make                # 컴파일, 실행파일이 bin/에 생성
cd bin/dataset/
python make_list_cur.py    # 테스트 이미지 목록 생성
cd ..
sh script-unix-aix2024-test-one.sh    # 이미지 한 장 테스트
sh script-unix-aix2024-test-all.sh    # 전체 테스트 (mAP 측정)
```

실행 명령어 해부

스크립트 안의 실제 명령어를 분석해 봅시다:

```
yolo_cpu.exe detector test yolohw.names aix2024.cfg aix2024.weights -thresh
0.24 test01.jpg -out_filename test01-det
```

각 인자의 의미:

- ☐ `detector test` — 탐지 모드, 한 장 테스트
- ☐ `yolohw.names` — 60개 클래스 이름 파일
- ☐ `aix2024.cfg` — 네트워크 구조 파일
- ☐ `aix2024.weights` — 학습된 가중치 파일
- ☐ `-thresh 0.24` — 검출 신뢰도 임계값 (24% 이상만 표시)
- ☐ `test01.jpg` — 테스트할 이미지
- ☐ `-out_filename test01-det` — 결과 이미지 저장 이름

양자화 버전을 실행할 때는 두 가지 옵션이 추가됩니다:

```
yolo_cpu.exe detector test yolohw.names aix2024.cfg aix2024.weights -thresh 0.24 test01.jpg -out_filename test01-det-quantized -quantized -save_params
```

- ☐ `-quantized` — 양자화 추론 모드로 실행
- ☐ `-save_params` — 양자화된 파라미터를 hex 파일로 저장

4-4. 실행 결과 이해하기

FP32 모드 실행 결과

전체 테스트(`test-all`)를 돌리면 이런 결과가 나옵니다:

```
mean average precision (mAP) = 0.817559, or 81.76%
Total Detection Time: 7.000000 Seconds
```

이것이 기준선입니다. 32비트 부동소수점으로 계산했을 때의 정확도가 81.76%라는 뜻입니다.

INT8 양자화 모드 실행 결과

기본 제공되는 multiplier 값으로 양자화 버전을 돌리면:

mean average precision (mAP) = 0.550382, or 55.04%

정확도가 81.76%에서 55.04%로 크게 떨어졌습니다. 이것은 제공된 multiplier 값이 "naive(단순한)" 버전이기 때문입니다. 여러분의 과제는 이 multiplier 값을 최적화해서 **mAP를 최대한 81.76%에 가깝게 복원**하는 것입니다.

한 장 테스트 결과

한 장 테스트를 돌리면 원본 이미지(`test01.jpg`)에 바운딩 박스가 그려진 결과 이미지(`test01-det.png`)가 생성됩니다. 양자화 버전도 마찬가지로 `test01-det-quantized.png`이 생성됩니다. 두 이미지를 비교하면 양자화로 인한 검출 성능 차이를 눈으로 확인할 수 있습니다.

4-5. 핵심 소스 코드 흐름 — 프로그램이 실행될 때 무슨 일이 벌어지나

프로그램이 시작되면 대략 이런 순서로 실행됩니다:

1단계: 모델 로딩 — `aix2024.cfg`를 파싱해서 네트워크 구조(레이어 타입, 크기 등)를 메모리에 구성합니다. 그런 다음 `aix2024.weights`를 읽어서 각 레이어의 가중치를 채웁니다.

2단계: 이미지 전처리 — 테스트 이미지를 읽어서 256×256×3으로 리사이즈하고, 픽셀 값을 0~1 사이의 float으로 정규화합니다.

3단계: 양자화 (양자화 모드일 때) — `do_quantization()` 함수가 호출됩니다. 각 합성곱 레이어에 대해 `input_quant_multiplier`와 `weights_quant_multiplier`를 설정하고, float 가중치를 INT8로, float bias를 INT16으로 변환합니다.

4단계: 추론 (Forward Pass) — `yolo2_forward_network_q()` 함수가 22개 레이어를 순서대로 실행합니다. 각 합성곱 레이어에서는 `forward_convolutional_layer_q()`가 호출되어 INT8 연산을 수행합니다. 이 과정에서 `log_feamap/` 폴더에 각 레이어의 입출력이 hex로 저장됩니다.

5단계: 후처리 — YOLO 레이어의 출력을 해석해서 바운딩 박스를 생성하고, NMS로 중복을 제거하고, 결과를 이미지에 그립니다.

6단계: 평가 — 전체 테스트 모드에서는 모든 이미지의 검출 결과를 ground truth와 비교하여 mAP를 계산합니다.

4-6. 하드웨어와의 연결 고리 — hex 파일의 의미

이 부분이 매우 중요합니다. 소프트웨어가 생성하는 hex 파일들이 **소프트웨어와 하드웨어를 연결하는 다리** 역할을 합니다.

log_param/ 폴더의 파일들 — 하드웨어의 ROM/BRAM에 초기값으로 로드됩니다.

- ☐ CONV00_param_weight.hex → 가중치 메모리에 로드
- ☐ CONV00_param_biases.hex → bias 레지스터에 로드
- ☐ CONV00_param_scales.hex → descaling 레지스터에 로드

log_feamap/ 폴더의 파일들 — 하드웨어 시뮬레이션의 입력과 정답(golden reference)으로 사용됩니다.

- ☐ CONV00_input.hex → 테스트벤치가 이 데이터를 외부 메모리 모델에 로드
- ☐ CONV00_output.hex → 하드웨어 출력과 비교해서 정확성을 검증

즉, 소프트웨어를 먼저 실행해서 "이 입력을 넣으면 이 출력이 나와야 한다"는 정답지를 만들고, 하드웨어 시뮬레이션에서 같은 입력을 넣어봤을 때 같은 출력이 나오는지 비교하는 것입니다. 이것이 **테스트 벡터(test vector) 기반 검증**이며, 다음 차시에서 더 자세히 다룹니다.

4-7. 하드웨어 디렉토리 미리보기

yolohw/ 폴더의 구조도 간단히 훑어보겠습니다. 상세한 내용은 나중 차시에서 다루지만, 전체 지도를 머릿속에 그려두는 것이 좋습니다.

```
yolohw/
├── src/                ← Verilog 소스 코드
│   ├── yolo_engine.v  ← 최상위 모듈 (가속기 엔진)
│   ├── yolo_engine_axi.v ← AXI 인터페이스 래퍼
│   ├── axi_dma_read.v  ← DMA 읽기 모듈
│   ├── axi_dma_write.v ← DMA 쓰기 모듈
│   └── axi_dma_ctrl.v  ← DMA 제어 모듈
├── sim/               ← 테스트벤치와 시뮬레이션 데이터
│   ├── yolo_engine_tb.v ← 탑 모듈 테스트벤치
│   ├── conv_kern_tb.v  ← 합성곱 커널 테스트벤치
│   ├── conv_layer_tb.v ← 합성곱 레이어 테스트벤치
│   └── cnn_ctrl_tb.v   ← 컨트롤러 테스트벤치
```

```

|   |─ inout_data_sw/      ← SW에서 생성한 hex 데이터 (정답)
|   |   |─ log_feamap/    ← feature map 데이터
|   |   |   └─ log_param/ ← 양자화 파라미터
|   |─ inout_data_hw/     ← HW 시뮬레이션 출력 (검증용)
|   └─ sim_dram_model/    ← 외부 메모리 시뮬레이션 모델
└─ fpga/                  ← FPGA 구현 관련
    |─ yolo_design_1_wrapper.xsa ← 하드웨어 export 파일
    |─ ip_repo/            ← 사용자 IP 저장소
    └─ vitis/              ← MicroBlaze 펌웨어
        └─ memorytest.c    ← 기본 테스트 코드
└─ arxiv/                  ← 시뮬레이션 결과 스크린캡처

```

sim/inout_data_sw/ 안의 데이터가 바로 소프트웨어에서 생성한 hex 파일들을 복사해 넣은 것입니다. 이것을 하드웨어 테스트벤치가 읽어서 검증에 사용합니다.

4-8. 개발 환경 요약

정리하면 이 프로젝트에서 사용하는 도구는 다음과 같습니다:

소프트웨어 개발: Visual Studio 2019 (Windows) 또는 gcc (Linux/Mac)로 C 코드를 컴파일합니다. Python은 데이터셋 목록 생성과 양자화 분석용으로 보조적으로 사용됩니다.

하드웨어 개발: Xilinx Vivado 2021.1을 사용합니다. RTL 시뮬레이션, IP 패키징, 합성(Synthesis), FPGA 구현까지 모두 Vivado에서 수행합니다. Vivado는 Windows와 Linux만 지원하므로, Mac 사용자는 가상 머신이 필요합니다.

시스템 통합: Xilinx Vitis 2021.1을 사용해서 MicroBlaze 펌웨어를 작성하고 FPGA에 다운로드합니다. Host PC 쪽 통신 코드는 C/C++ 또는 Python으로 작성합니다.

4차시 핵심 정리:

1. src/ 폴더의 핵심 파일은 yolov2_forward_network_quantized.c — 양자화 코드의 모든 것
2. bin/ 폴더에 모델(cfg, weights), 데이터셋, 스크립트, hex 출력이 있음
3. FP32 기준선은 mAP 81.76%, 기본 양자화는 55.04% — multiplier 최적화가 과제
4. SW가 생성하는 hex 파일(log_feamap, log_param)이 HW 검증의 핵심 다리 역할
5. 실행 흐름: 모델 로딩 → 이미지 전처리 → 양자화 → 추론 → 후처리 → 평가

5차시: 양자화(Quantization) — float에서 INT8로

이번 차시는 소프트웨어 파트의 핵심입니다. "왜 양자화가 필요하고, 수학적으로 정확히 어떻게 동작하며, 코드에서 각 줄이 무엇을 하는지"를 완전히 해부합니다.

5-1. 왜 양자화가 필요한가?

원래 학습된 모델(`aix2024.weights`)의 가중치와 연산은 모두 **float32(32비트 부동소수점)**로 되어 있습니다. float32 하나는 4바이트이고, 소수점 아래 약 7자리까지 정밀하게 표현할 수 있습니다.

하지만 하드웨어(FPGA)에서 float32 곱셈기를 구현하면 문제가 큼니다.

면적 문제: float32 곱셈기 하나에 DSP 슬라이스 여러 개와 많은 LUT가 필요합니다. Artix-7 FPGA에 DSP가 240 개밖에 없는데, float32 곱셈기를 많이 넣으면 병렬도가 크게 떨어집니다.

속도 문제: float32 연산은 지수부(exponent)와 가수부(mantissa)를 따로 처리해야 하므로 파이프라인이 길어지고, 클럭 주파수를 높이기 어렵습니다.

전력 문제: 넓은 비트 연산은 전력을 더 소모합니다.

반면 **INT8(8비트 정수)** 곱셈기는 매우 작고 빠릅니다. Xilinx DSP48E 블록 하나가 25×18비트 곱셈을 지원하므로, 8×8비트 곱셈은 DSP 하나로 충분하며, 경우에 따라서는 LUT만으로도 구현 가능합니다. 같은 FPGA 자원으로 **훨씬 더 많은 곱셈기를 병렬로 배치**할 수 있습니다.

정리하면, 양자화의 목적은 **정확도를 최대한 유지하면서, 하드웨어 구현이 쉬운 정수 연산으로 변환**하는 것입니다.

5-2. 양자화의 기본 원리 — Scale and Clip

양자화의 핵심 아이디어는 단순합니다: **실수(float) 값에 적절한 배율(multiplier)을 곱해서 정수 범위로 매핑**하는 것입니다.

예를 들어 어떤 가중치가 float으로 0.0523 이라고 합시다. 이 값에 multiplier 100 을 곱하면 5.23 이 되고, 이것을 반올림하면 정수 5 가 됩니다. multiplier를 1000 으로 하면 52.3 → 52 가 됩니다. multiplier가 클수록 정밀도는 높아지지만, INT8의 범위(-127 ~ +127)를 넘어갈 위험도 커집니다.

이 프로젝트에서의 양자화 공식은 두 단계입니다:

- ① Scale (스케일링): 정수값 = float값 × multiplier
- ② Clip (클리핑): 정수값이 범위를 넘으면 잘라냄

코드에서 이것을 구현하는 함수가 max_abs() 입니다:

```
int max_abs(int src, int max_val) {  
    if (abs(src) > abs(max_val))  
        src = (src > 0) ? max_val : -max_val - 1;  
    return src;  
}
```

값이 max_val (예: 127)을 넘으면 127로, -max_val 보다 작으면 -128로 잘라냅니다. 이것이 클리핑 (saturation)입니다.

5-3. 가중치 양자화 (Weight Quantization) — 한 번만 하면 된다

가중치 양자화는 추론 전에 한 번만 수행됩니다. 가중치는 학습 후 고정된 값이니까요.

do_quantization() 함수에서 하는 일:

```
// 각 합성곱 레이어에 대해  
for (fil = 0; fil < l->n; ++fil) {           // 각 필터(출력 채널)  
    for (i = 0; i < filter_size; ++i) {      // 필터 내 각 원소  
        float w = l->weights[fil*filter_size + i] * l->  
weights_quant_multiplier; // Scale  
        l->weights_int8[fil*filter_size + i] = max_abs(w,  
MAX_VAL_8); // Clip to [-127, 127]  
    }  
}
```

```
}
```

구체적 예시로 Layer 0을 봅시다. Layer 0의 필터 크기는 $3 \times 3 \times 3 = 27$ 이고, 필터 16개입니다.

`weights_quant_multiplier`가 21이라고 하면:

원래 가중치: 0.0523 (float32)
스케일링: $0.0523 \times 21 = 1.098$
클리핑: 1 (INT8 범위 안이므로 그대로)

원래 가중치: 6.064 (float32) ← 이 레이어에서 가장 큰 값
스케일링: $6.064 \times 21 = 127.34$
클리핑: 127 (MAX_VAL_8로 잘림)

5-4. Bias 양자화

Bias도 정수로 변환해야 합니다. 핵심은 **bias의 multiplier가 가중치와 입력의 multiplier 곱**이라는 점입니다.

왜 그런지 생각해 봅시다. 합성곱의 결과는 `output =  $\Sigma(\text{input} \times \text{weight}) + \text{bias}$` 입니다. 양자화된 세계에서:

$$\text{output_q} = \Sigma(\text{input_q} \times \text{weight_q}) + \text{bias_q}$$

여기서 `input_q = input × I_mult`이고 `weight_q = weight × W_mult`이므로:

$$\begin{aligned} \text{output_q} &= \Sigma((\text{input} \times I_mult) \times (\text{weight} \times W_mult)) + \text{bias_q} \\ &= (I_mult \times W_mult) \times \Sigma(\text{input} \times \text{weight}) + \text{bias_q} \end{aligned}$$

`bias_q`가 `output_q`와 같은 스케일에서 더해지려면, `bias_q = bias × (I_mult × W_mult)` 이어야 합니다.

코드에서:

```
float biases_multiplier = (l->weights_quant_multiplier * l->input_quant_multiplier);
for (fil = 0; fil < l->n; ++fil) {
    float b = l->biases[fil] * biases_multiplier;    // Scale
    l->biases_quant[fil] = max_abs(b, MAX_VAL_16);    // Clip to [-32767, 32767]
}
```

Bias는 **INT16(16비트)**으로 양자화됩니다. INT8이 아니라 INT16인 이유는, bias 값 자체가 가중치보다 크거나 multiplier 곱이 커서 8비트로 표현 범위가 부족할 수 있기 때문입니다.

5-5. 입력(Activation) 양자화 — 매 레이어마다 실시간으로

가중치와 달리, 입력 feature map은 이미지마다 값이 다릅니다. 따라서 **추론 실행 중에 매 레이어마다** 양자화를 수행해야 합니다.

forward_convolutional_layer_q() 함수의 첫 부분:

```
for (z = 0; z < l.inputs; ++z) {
    int16_t src = state.input[z] * l.input_quant_multiplier;    // Scale
    state.input_uint8[z] = max_abs(src, MAX_VAL_UINT_8);        // Clip to [0, 255]
}
```

여기서 주목할 점은 입력이 **UINT8(0~255)**로 변환된다는 것입니다. INT8(-127~127)이 아닙니다. 왜냐하면 이전 레이어의 출력이 ReLU를 거쳐서 나왔기 때문에, 값이 항상 0 이상입니다. 음수가 없으므로 0~255 범위의 unsigned 8비트를 사용하는 것이 표현 범위를 더 효율적으로 쓰는 방법입니다.

5-6. 양자화된 합성곱의 전체 파이프라인

한 레이어의 양자화된 합성곱이 실행되는 전체 순서를 정리하면:

```
① 입력 양자화:    input_uint8 = clip(input_float × I_mult, 255)
                  → float을 UINT8로 변환
```

- ② im2col 변환: 입력 feature map을 행렬 형태로 재배치
 → 합성곱을 행렬곱으로 변환하는 트릭 (건드릴 필요 없음)
- ③ 행렬 곱셈: $\text{output_q}(\text{INT32}) = \text{weights_int8}(\text{INT8}) \times \text{input_uint8}(\text{UINT8})$
 → 8비트 × 8비트 = 최대 16비트, 이것을 수백~수천번 누적하므로 INT32로 저장
- ④ Bias 덧셈: $\text{output_q} += \text{biases_quant}(\text{INT16})$
 → 각 출력 채널별로 bias 추가
- ⑤ ReLU: $\text{output_q} = \max(\text{output_q}, 0)$
 → 음수를 0으로
- ⑥ De-scaling: $\text{output_float} = \text{output_q} / (\text{I_mult} \times \text{W_mult})$
 → 정수 결과를 다시 float 스케일로 복원
- ⑦ 다음 레이어로: 이 output_float이 다음 레이어의 input_float이 됨
 → 다음 레이어에서 다시 ①부터 반복

하드웨어에서는 ⑥의 de-scaling이 약간 다르게 처리됩니다. float 나눗셈 대신, 다음 레이어의 `I_mult`를 이용한 정수 나눗셈(비트 시프트)으로 처리합니다. 구체적으로는:

```
scale = (I_mult_현재 × W_mult_현재) / I_mult_다음
output_for_next_layer = output_q / scale
```

이 `scale` 값이 바로 `save_quantized_model()`에서 `CONV00_param_scales.hex`로 저장되는 값입니다. 하드웨어에서는 이 `scale` 값을 레지스터에 넣어두고, 합성곱 결과를 이 값으로 나누어 다음 레이어 입력용 8비트 값을 만듭니다.

5-7. Multiplier의 선택이 정확도에 미치는 영향

multiplier 선택에는 근본적인 딜레마가 있습니다.

Multiplier가 너무 작으면: float 값이 작은 정수로 매핑됩니다. 예를 들어 0.05에 multiplier 10을 곱하면 0.5 → 반올림하면 0 또는 1. 원래 서로 다른 값들이 양자화 후에 같은 정수가 되어버립니다. 이것을 해상도 손실(resolution loss) 또는 라운딩 에러(rounding error)라고 합니다.

5-9. Multiplier 최적화 전략 — 실전 팁

이론적 최적값을 출발점으로 삼아서, 다음과 같은 전략으로 최적화할 수 있습니다.

전략 1: Max 기반 (가장 단순)

`multiplier = 127 / max_abs_value`. 위에서 설명한 방법입니다. 클리핑 에러는 없지만 outlier에 취약합니다.

전략 2: Percentile 기반

최댓값 대신 99.9 percentile 값을 사용합니다. 즉 상위 0.1%의 극단값은 잘려나가도록 허용하고, 나머지 99.9%의 해상도를 높입니다. `multiplier = 127 / percentile_99.9_value`.

전략 3: 수동 탐색 (Grid Search)

각 레이어의 multiplier를 일정 범위에서 바꿔가며 mAP를 측정합니다. 22개 값을 동시에 탐색하면 조합이 폭발하므로, 한 레이어씩 고정하고 하나만 바꾸는 방식(coordinate descent)을 사용하면 현실적입니다.

전략 4: 민감도 분석

각 레이어의 multiplier를 하나씩 바꿔보고, mAP에 미치는 영향을 측정합니다. mAP 변화가 큰 레이어는 "민감한" 레이어이므로 더 신중하게 multiplier를 선택하고, 영향이 적은 레이어는 대략적으로 설정해도 됩니다.

5-10. De-scaling과 하드웨어의 관계

De-scaling 단계를 다시 자세히 봅시다. 소프트웨어에서는 이렇게 합니다:

```
float ALPHA1 = 1 / (l.input_quant_multiplier * l.weights_quant_multiplier);
for (i = 0; i < l.outputs; ++i) {
    l.output[i] = output_q[i] * ALPHA1; // INT32 → float
}
```

그런데 하드웨어에는 float 나눗셈이 없습니다. 대신 `save_quantized_model()` 에서 저장하는 정수 **scale** 값을 사용합니다:

```
int scale = (l->input_quant_multiplier * l->weights_quant_multiplier)
            / next_input_quant_multiplier;
```

하드웨어에서는 합성곱 결과(INT32)를 이 `scale`로 나누고 8비트로 잘라서, 바로 다음 레이어의 **UINT8** 입력으로 사용합니다. 즉 소프트웨어처럼 float으로 복원했다가 다시 양자화하는 것이 아니라, 정수 세계 안에서 스케일만 조정하는 것입니다. 이것이 하드웨어에서 양자화의 핵심입니다 — float 연산이 단 한 번도 필요 없습니다.

소프트웨어: INT32 결과 → float 복원 → 다시 UINT8 양자화 (float 연산 2번)
하드웨어: INT32 결과 → scale로 나눗셈 → UINT8 (정수 연산만)

하드웨어에서의 나눗셈도 실제로는 나눗셈기가 아니라 **비트 시프트(bit shift)**로 구현하는 경우가 많습니다. scale 값이 2의 거듭제곱이면 오른쪽 시프트 한 번으로 나눗셈이 끝납니다. 아니더라도 "곱셈 + 시프트"로 나눗셈을 근사할 수 있습니다.

5-11. 데이터 타입 정리

이 시점에서 전체 데이터 흐름에서의 비트 폭을 정리하면:

데이터	소프트웨어 타입	비트 폭	비고
입력 feature map	UINT8	8비트	ReLU 이후라 0 이상
가중치 (weight)	INT8	8비트	양/음 모두 가능
곱셈 결과 하나	INT16	16비트	$8 \times 8 =$ 최대 16비트
누적합 (여러 곱셈의 합)	INT32	32비트	수백~수천 번 누적
Bias	INT16	16비트	
ReLU 이후	INT32	32비트	음수가 0으로 변환됨
De-scale 이후	UINT8	8비트	다음 레이어 입력으로
Scale 값	UINT16	16비트	레이어별 하나

하드웨어 설계할 때 이 비트 폭이 매우 중요합니다. MAC 유닛에서 곱셈기는 $8 \times 8 = 16$ 비트, 누산기(accumulator)는 최소 32비트가 필요하고, 최종 출력은 scale로 나눈 뒤 8비트로 잘라야 합니다.

5-12. hex 파일 생성 — 하드웨어로의 다리

양자화 코드를 실행하면 세 종류의 hex 파일이 생성됩니다:

weight hex (CONV00_param_weight.hex): 양자화된 가중치를 8비트 16진수로 한 줄에 하나씩 기록합니다. Layer 0의 경우 16개 필터 $\times$ $3 \times 3 \times 3 = 432$ 개의 값이 순서대로 나열됩니다.

bias hex (CONV00_param_biases.hex): 양자화된 bias를 16비트 16진수로 기록합니다. Layer 0은 16개 (필터 수만큼).

scale hex (CONV00_param_scales.hex): de-scaling 값을 16비트 16진수로 기록합니다. Layer 0은 1개 (레이어당 하나).

이 파일들이 나중에 Verilog 테스트벤치에서 \$readmemh() 로 읽혀서 BRAM에 초기값으로 로드됩니다. 하드웨어가 합성곱을 수행할 때 이 BRAM에서 가중치와 bias를 읽어 사용합니다.

5차시 핵심 정리:

1. 양자화 = float $\times$ multiplier $\rightarrow$ 정수 변환 + 범위 초과시 클리핑
2. 가중치는 INT8(-127~127), bias는 INT16, 입력은 UINT8(0~255)
3. Multiplier가 작으면 해상도 손실, 크면 클리핑 에러 $\rightarrow$ 균형점 찾기가 과제
4. $127/\text{max_value}$ 가 이론적 출발점이지만, outlier와 레이어 간 전파 때문에 실험적 조정 필요
5. 하드웨어에서는 de-scaling을 정수 나눗셈(scale)으로 처리 $\rightarrow$ float 연산 완전 제거
6. 생성되는 hex 파일(weight, bias, scale)이 하드웨어 BRAM의 초기값이 됨

질문 1: save_quantized_model() 의 정수 scale 값이 뭔가요?

코드를 직접 봅시다:

```
int next_input_quant_multiplier = 1;
for (int z = 1->index + 1; z < net.n; ++z) {
    if (net.layers[z].type == CONVOLUTIONAL) {
        next_input_quant_multiplier = net.layers[z].input_quant_multiplier;
        break;
    }
}
```

```

}

int scale = (l->input_quant_multiplier * l->weights_quant_multiplier)
            / next_input_quant_multiplier;

```

이것이 무엇을 하는지 수학적으로 따라가 봅시다.

문제 상황

Layer 0의 합성곱 결과 `output_q`는 이런 스케일을 갖고 있습니다:

$$\text{output_q} = \Sigma(\text{input_uint8} \times \text{weight_int8}) + \text{bias_quant}$$

여기서 $\text{input_uint8} = \text{input_float} \times I_mult_0$ 이고, $\text{weight_int8} = \text{weight_float} \times W_mult_0$ 이므로:

$$\text{output_q} \approx (I_mult_0 \times W_mult_0) \times \text{원래_합성곱_결과_float}$$

즉 `output_q`는 원래 **float** 결과에 $I_mult_0 \times W_mult_0$ 이 곱해진 상태입니다.

다음 레이어가 원하는 것

이 `output_q`가 ReLU를 거친 뒤, 다음 합성곱 레이어(Layer 2)의 입력이 됩니다. Layer 2는 자기 입력을 I_mult_2 로 곱해서 UINT8로 만들기를 기대합니다:

$$\text{Layer 2의 입력_uint8} = \text{원래_float_값} \times I_mult_2$$

연결하기

현재 `output_q` 안에는 $I_mult_0 \times W_mult_0$ 이 곱해져 있고, 다음 레이어는 I_mult_2 만 곱해진 값을 원합니다. 따라서:

다음_레이어_입력 = output_q / scale

여기서 $scale = (I_mult_0 \times W_mult_0) / I_mult_2$

이렇게 나누면:

$$\begin{aligned} output_q / scale &= (float결과 \times I_mult_0 \times W_mult_0) / (I_mult_0 \times W_mult_0 / I_mult_2) \\ &= float결과 \times I_mult_2 \end{aligned}$$

정확히 다음 레이어가 원하는 스케일이 됩니다.

구체적 숫자 예시

Layer 0의 multiplier가 $I_mult_0 = 97$, $W_mult_0 = 21$ 이고, 다음 합성곱인 Layer 2의 $I_mult_2 = 10$ 이라고 하면:

$$scale = (97 \times 21) / 10 = 2037 / 10 = 203$$

합성곱 결과(INT32)를 203으로 나누면, 바로 Layer 2에 넣을 수 있는 UINT8 값이 나옵니다. **소프트웨어처럼 float으로 복원했다가 다시 양자화할 필요 없이, 정수 나눗셈 한 번으로 끝나는 것입니다.**

이 203이라는 숫자가 CONV00_param_scales.hex 에 16진수로 저장됩니다 (00cb). 하드웨어는 이 값을 레지스터에 넣어두고, 합성곱이 끝날 때마다 결과를 이 값으로 나눕니다.

왜 "레이어당 하나"인가?

scale 값을 구성하는 세 요소(I_mult_0 , W_mult_0 , I_mult_2)가 모두 **레이어 단위로 고정된 상수**이기 때문입니다. 같은 레이어 안의 모든 픽셀, 모든 채널에 대해 동일한 scale이 적용됩니다. 그래서 하드웨어에서도 레지스터 하나에 넣어두면 됩니다.

질문 2: 입력이 매번 다른데 어떻게 양자화를 하나?

이것은 정말 중요한 통찰입니다. 핵심을 짚어보면:

multiplier는 "고정"이다

`input_quant_multiplier`라는 이름이 혼동을 줄 수 있는데, 이 값은 이미지마다 계산하는 게 아니라, 미리 정해 놓은 상수입니다. 코드를 보면 명확합니다:

```
float input_quant_multiplier[11] = { 97, 10, 10, 9, 14, 2, 7, 7, 10, 17, 11 };
```

이 배열은 프로그램 시작 시 한 번 설정되면, 어떤 이미지가 들어오든 똑같은 값이 사용됩니다. test01.jpg를 넣든 test02.jpg를 넣든, Layer 0의 `input_quant_multiplier`는 항상 97입니다.

그러면 이미지마다 입력 값의 범위가 다른데 괜찮은가?

여기서 "왜 괜찮은지"를 이해하려면, 각 레이어 입력의 특성을 생각해야 합니다.

Layer 0 (첫 번째 레이어): 입력은 원본 이미지를 0.1 사이의 float으로 정규화한 값입니다. 어떤 이미지를 넣든 값의 범위는 항상 0.0 ~ 1.0입니다. 따라서 $I_mult = 97$ 이면 $0 \sim 1 \times 97 = 0 \sim 97$ 범위가 되어, `UINT8(0~255)` 안에 안전하게 들어갑니다. 이미지가 바뀌어도 이 범위는 변하지 않죠.

Layer 2 이후: 입력은 이전 레이어의 합성곱 → ReLU → de-scaling 결과입니다. 여기서 핵심은 **de-scaling**이 다음 레이어의 `I_mult`를 기준으로 맞춰준다는 점입니다. 5-10에서 설명했듯이:

```
다음_레이어_입력 = output_q / scale = float결과 × I_mult_다음
```

즉, de-scaling 단계에서 이미 다음 레이어의 multiplier에 맞게 스케일이 조정됩니다. 따라서 다음 레이어에서는 추가 양자화가 필요 없이 그 값을 바로 `UINT8`로 쓸 수 있습니다.

하지만 값이 255를 넘으면?

물론 넘을 수 있습니다. 그래서 `max_abs()` 함수가 클리핑을 합니다. 이미지에 따라 특정 레이어의 활성화 값이 유독 클 수 있고, 그 경우 255에서 잘립니다.

이것이 바로 **양자화의 정확도 손실이 발생하는 근본 원인** 중 하나입니다. `multiplier`가 고정되어 있으니까, 어떤 이미지에서는 잘 맞지만 다른 이미지에서는 클리핑이 많이 발생할 수 있는 거죠.

그러면 `multiplier`는 어떻게 정하는 게 좋은가?

`multiplier`를 정할 때는 **한 장이 아니라 여러 장의 대표적인 이미지를 기준으로** 합니다. 이것을 **캘리브레이션 (calibration)**이라고 합니다.

실전적인 접근은 이렇습니다:

1. 대표 이미지 여러 장(예: 10~50장)을 FP32로 추론
2. 각 레이어의 입력 값 분포(최솟값, 최댓값, 평균, 분산 등)를 기록
3. 이 분포를 기반으로 "대부분의 이미지에서 클리핑이 최소화되는" `multiplier`를 선택
4. 전체 테스트셋으로 mAP를 측정해서 최종 확인

코드에 이미 이 분석을 위한 프로파일링 코드가 주석 처리되어 들어있습니다:

```
//// Profiling range
//float xmin = 0, xmax = 0, qmin = 0, qmax = 0;
//for (int i = 0; i < l.inputs; i++) {
//    xmin = (xmin > state.input[i]) ? state.input[i] : xmin;
//    xmax = (xmax < state.input[i]) ? state.input[i] : xmax;
//}
```

이 주석을 풀면 각 레이어에서 입력 값의 실제 범위를 측정할 수 있습니다. 여러 이미지에 대해 이것을 수집하면 적절한 `multiplier`의 힌트를 얻을 수 있죠.

요약: 소프트웨어 vs 하드웨어에서의 양자화

	소프트웨어 (추론 시)	하드웨어 (추론 시)
multiplier	코드에 상수 배열로 고정	레지스터에 고정값 저장
입력 양자화	$\text{input} \times I_mult \rightarrow \text{clip}(255)$	이전 레이어의 de-scale 결과가 이미 UINT8이므로 추가 양자화 불필요
합성곱	$\text{INT8} \times \text{UINT8} \rightarrow \text{INT32}$ 누적	동일
de-scaling	$\text{output} / (I \times W) \rightarrow \text{float} \rightarrow$ 다시 양자화 (float 경유)	$\text{output} / \text{scale} \rightarrow$ 바로 UINT8 (정수만 사용)

하드웨어에서의 큰 차이점은, 첫 번째 레이어만 제외하면 명시적인 "입력 양자화" 단계가 없다는 것입니다. de-scaling 이 곧 다음 레이어의 양자화를 겸하기 때문입니다. 이것이 가능한 이유가 바로 scale 값에 `next_input_quant_multiplier`가 포함되어 있기 때문입니다.

6차시: 시스템 개요 — Host-FPGA 전체 구조

이번 차시에서는 하드웨어 설계에 들어가기 전에, 전체 시스템이 어떻게 생겼고 데이터가 어떻게 흘러가는지 큰 그림을 잡습니다. 이 그림을 모르면 나중에 Verilog를 짤 때 "내가 만드는 모듈이 전체에서 어디에 위치하는 건지" 감을 잃게 됩니다.

6-1. 시스템의 두 세계: Host PC와 FPGA 보드

전체 시스템은 물리적으로 두 개의 장치로 구성됩니다.

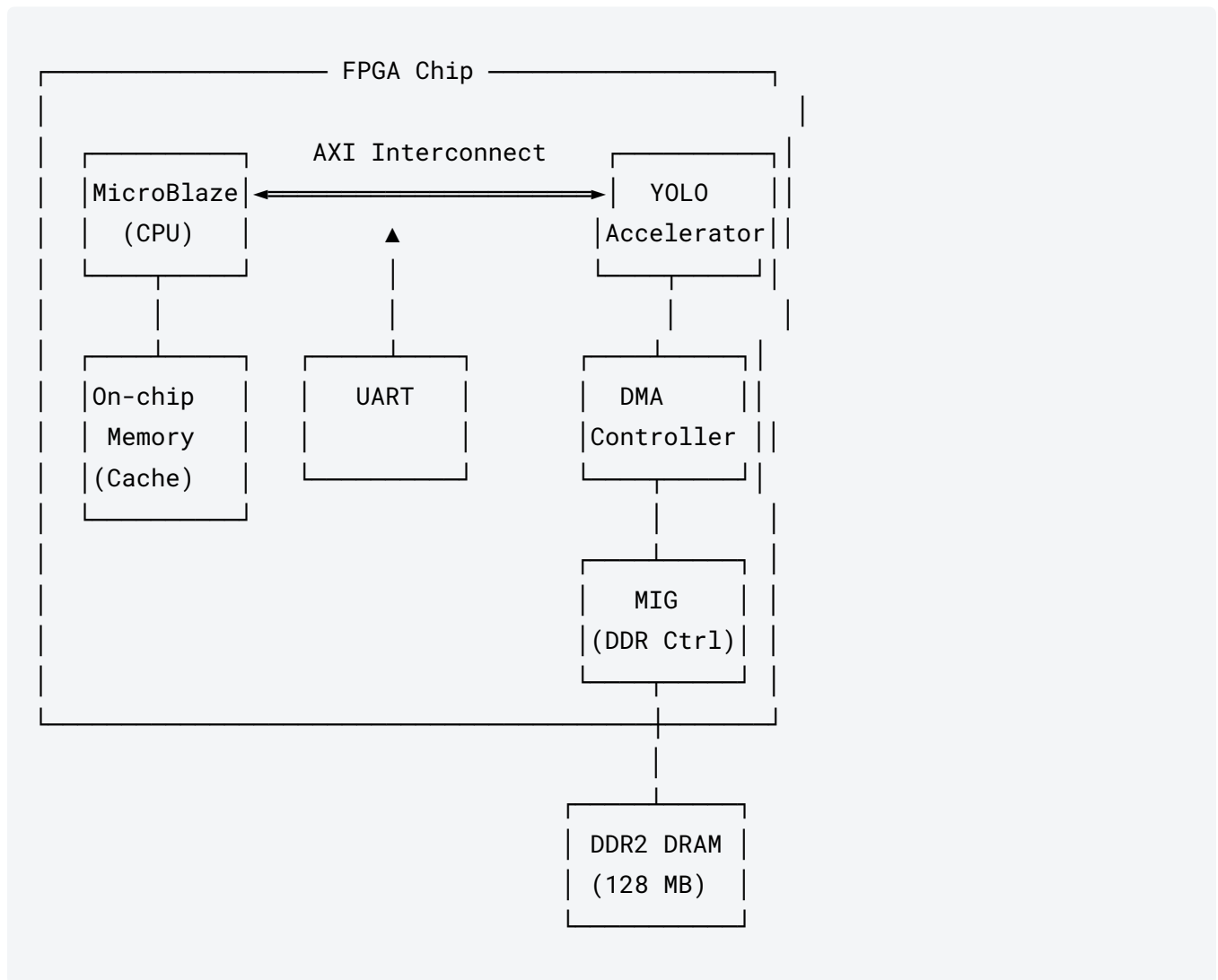
Host PC — 여러분의 노트북이나 데스크탑입니다. 이미지를 읽고, 양자화된 모델 파라미터를 준비하고, FPGA에 데이터를 보내고, 결과를 받아서 후처리(YOLO 디코딩, NMS, 바운딩 박스 그리기)를 합니다. C/C++ 프로그램(Visual Studio)으로 동작합니다.

FPGA 보드 (Nexys A7-100T) — Xilinx Artix-7 FPGA 칩이 탑재된 개발 보드입니다. 이 보드 위에서 실제 CNN 연산이 하드웨어로 수행됩니다. FPGA 칩 안에 소프트 프로세서(MicroBlaze)와 여러분이 설계한 가속기(YOLO 엔진)가 함께 들어갑니다. 보드에는 DDR2 DRAM도 탑재되어 있어서, 이미지와 파라미터를 저장할 수 있습니다.

이 두 장치는 **USB 케이블을 통한 UART(시리얼 통신)**로 연결됩니다. Host PC의 USB 포트에 FPGA 보드를 연결하면, 운영체제에서 COM 포트(시리얼 포트)로 인식됩니다. 통신 속도(baud rate)는 230,400 bps로 설정합니다.

6-2. FPGA 보드 내부 구조

FPGA 칩 안에 들어가는 모듈들과 그 연결을 살펴봅시다.



각 모듈의 역할:

MicroBlaze — FPGA 안에 구현된 **소프트 프로세서(CPU)**입니다. Intel의 x86이나 ARM처럼 물리적 칩이 아니라, FPGA의 논리 게이트로 만든 프로세서입니다. C 프로그램을 실행할 수 있고, 이 프로젝트에서는 "총 사령관" 역할을 합니다. Host PC에서 UART로 명령을 받으면, YOLO 가속기에 설정값을 써주고, 시작 신호를 보내고, 완료 여부를 확인하는 등의 제어를 담당합니다.

UART — Host PC와의 시리얼 통신 인터페이스입니다. MicroBlaze가 UART를 통해 Host와 데이터를 주고받습니

다.

MIG (Memory Interface Generator) — DDR2 DRAM을 제어하는 모듈입니다. Xilinx가 제공하는 IP로, DDR2의 복잡한 타이밍 프로토콜(CAS latency, refresh 등)을 처리해줍니다. MicroBlaze와 YOLO 가속기 모두 MIG를 통해 DRAM에 접근합니다.

DDR2 DRAM — FPGA 보드에 장착된 외부 메모리입니다. 128MB 용량으로, 입력 이미지, 모델 파라미터, 중간 결과, 최종 출력을 모두 저장합니다.

YOLO Accelerator — 여러분이 설계하는 핵심 모듈입니다. MAC 유닛, 온칩 버퍼(BRAM), 컨트롤러, DMA가 포함됩니다. 뒤에서 자세히 다룹니다.

AXI Interconnect — 이 모든 모듈을 연결하는 **버스(bus)**입니다. AXI(Advanced eXtensible Interface)는 ARM에서 설계한 고성능 버스 프로토콜로, Xilinx FPGA에서 표준으로 사용됩니다.

6-3. AXI 버스 — 모듈 간 통신의 언어

AXI는 모듈들이 서로 데이터를 주고받는 규칙(프로토콜)입니다. 두 가지 관계가 있습니다:

Master (마스터) — 데이터 전송을 주도하는 쪽입니다. "이 주소에서 데이터를 읽어와라" 또는 "이 주소에 이 데이터를 써라"라고 요청합니다.

Slave (슬레이브) — 요청에 응답하는 쪽입니다. 마스터가 요청한 주소의 데이터를 제공하거나, 마스터가 보낸 데이터를 받아 저장합니다.

이 프로젝트에서의 Master-Slave 관계:

```
MicroBlaze (Master) → UART (Slave)           : CPU가 UART 레지스터를 읽고 씀
MicroBlaze (Master) → YOLO Accelerator (Slave) : CPU가 가속기의 제어 레지스터를 설정
MicroBlaze (Master) → MIG/DRAM (Slave)         : CPU가 DRAM을 읽고 씀
YOLO Accel (Master) → MIG/DRAM (Slave)         : 가속기가 DRAM에서 데이터를 직접 읽고 씀
```

YOLO 가속기가 **두 가지 역할**을 동시에 하는 점이 중요합니다. MicroBlaze에서 설정을 받을 때는 **Slave**로 동작하고, DRAM에서 이미지/파라미터를 읽어올 때는 **Master**로 동작합니다. 이것이 코드에서 `yolo_engine_axi.v`가

AXI Slave 인터페이스와 AXI Master 인터페이스를 모두 가지고 있는 이유입니다.

6-4. 메모리 맵 — 모든 것은 주소로 접근한다

FPGA 시스템에서 모든 모듈은 **메모리 주소**로 접근됩니다. MicroBlaze 입장에서 보면:

주소 범위	매핑된 장치
0x0000_0000	On-chip Memory (I-Cache, D-Cache)
0x4060_0000	UART 레지스터
0x44A0_0000	YOLO Accelerator 제어 레지스터
0x8000_0000	DDR2 DRAM (128MB)

MicroBlaze가 주소 `0x44A0_0000`에 값을 쓰면, 그것은 YOLO 가속기의 첫 번째 제어 레지스터(`i_ctrl_reg0`)에 기록됩니다. 이 레지스터에 특정 값을 쓰면 가속기가 동작을 시작하는 식입니다.

6-5. DRAM 메모리 할당 — 무엇이 어디에 저장되나

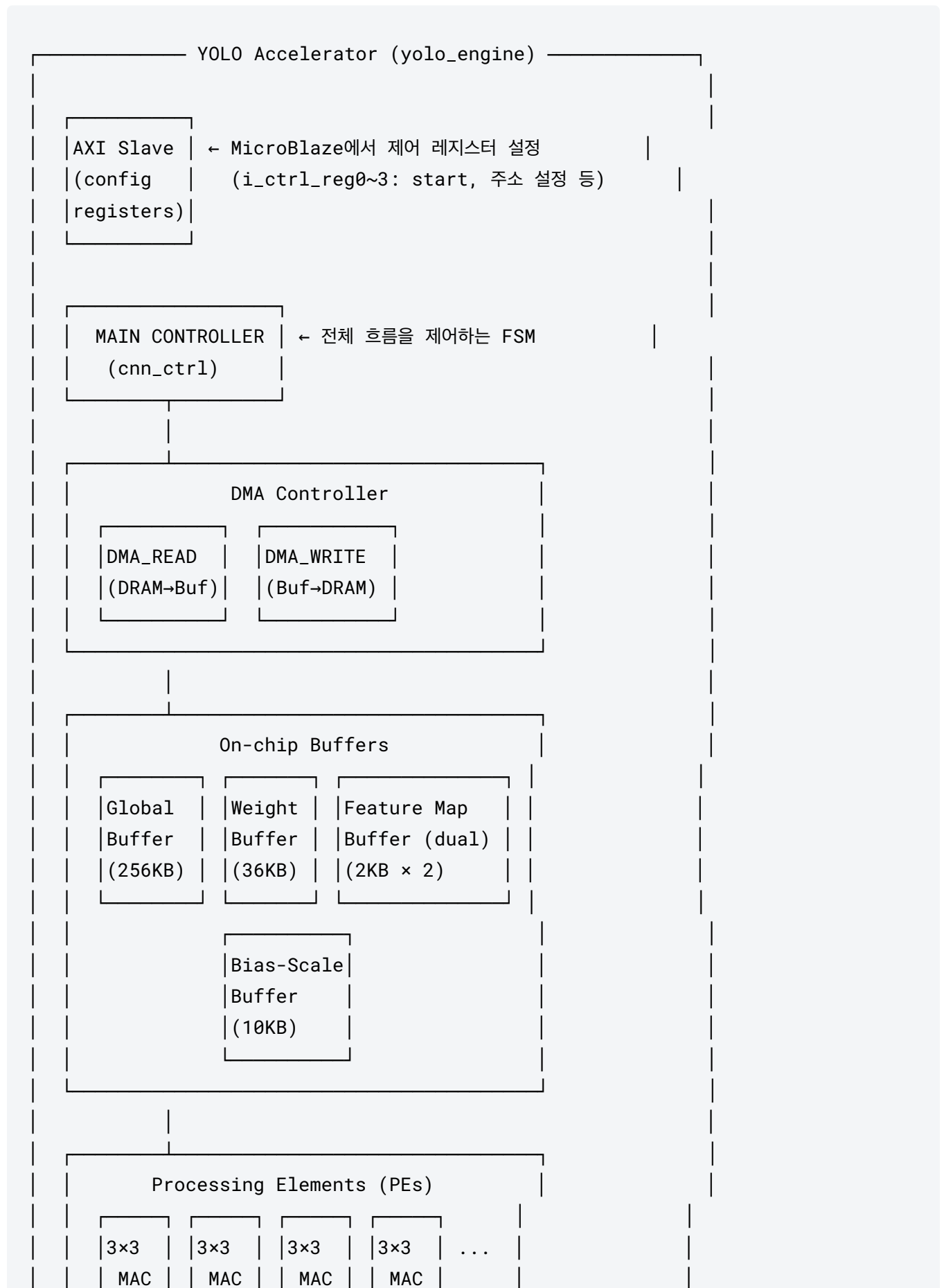
DDR2 DRAM은 주소 `0x8000_0000`부터 시작하며, 다음과 같이 영역이 나뉩니다:

DRAM 영역	크기	내용
입력 이미지	$256 \times 256 \times 4 = 256\text{KB}$	양자화된 입력 (UINT8, 4바이트 정렬)
모델 파라미터	~3.4MB	11개 합성곱 레이어의 weight, bias, scale
출력 1 (Layer 14)	$8 \times 8 \times 195 = \sim 12.5\text{KB}$	YOLO 출력 텐서 1
출력 2 (Layer 20)	$16 \times 16 \times 195 = \sim 50\text{KB}$	YOLO 출력 텐서 2
중간 결과 저장	필요에 따라	Route Layer용 중간 feature map

Host PC가 UART를 통해 이미지와 파라미터를 이 DRAM 영역에 써넣고, 가속기가 연산을 끝내면 출력 영역에서 결과를 읽어옵니다.

6-6. YOLO 가속기 내부 구조

여러분이 설계하는 가속기의 내부를 봅시다:



Host PC가 이미지 데이터와 모델 파라미터를 UART를 통해 FPGA 보드의 DRAM에 저장합니다. UART는 느리지만 (230,400 bps), 이 전송은 추론 전에 한 번만 하면 됩니다. 통신 흐름은: Host → UART → MicroBlaze → MIG → DRAM.

채널 ②: DRAM → L2 Global Buffer (DMA 경유)

가속기가 연산에 필요한 데이터를 DRAM에서 온칩 Global Buffer로 가져옵니다. 예를 들어 Layer 0을 처리하려면, 입력 이미지(256×256×3)와 Layer 0의 가중치(16×3×3×3)를 DRAM에서 읽어와야 합니다. DMA가 AXI Master로 동작하여 DRAM에서 데이터를 burst 방식으로 빠르게 읽어옵니다.

채널 ③: L2 Global Buffer → L1 Local Buffers → PEs

Global Buffer에서 더 작은 L1 버퍼(Weight Buffer, Feature Map Buffer)로 데이터를 쪼개서 옮기고, 여기서 MAC 유닛(PE)으로 공급합니다. 이 단계가 가장 빈번하게 일어나며, 데이터 재사용(data reuse)을 극대화하는 것이 성능의 핵심입니다.

연산 결과는 역순으로: PE → Partial Sum Buffer → Global Buffer → DMA → DRAM으로 저장됩니다.

6-8. 전체 동작 시나리오 — 이미지 한 장을 처리하는 흐름

Host PC에서 이미지를 하나 넣고 추론 결과를 받기까지의 전체 흐름을 시간순으로 정리하면:

1. Host: COM 포트 열기 (open_port)
2. Host: FPGA와 연결 확인 (test_hello, test_echo)
3. Host: 모델 파라미터를 FPGA DRAM에 전송 (write_from_file_to_fpga)
→ weight, bias, scale hex 파일의 내용을 UART로 보냄
4. Host: 입력 이미지를 FPGA DRAM에 전송
5. Host: 가속기 설정 (write_config)
→ 입력/출력 DRAM 주소 등을 레지스터에 설정
6. Host: 가속기 시작 명령 (start_engine)
→ MicroBlaze가 i_ctrl_reg0에 1을 씀
7. FPGA: 가속기가 22개 레이어를 순서대로 처리
→ 각 레이어마다: DMA로 데이터 로드 → 합성곱 → 결과 DMA로 저장
8. FPGA: 처리 완료 신호 (network_done)

9. Host: FPGA DRAM에서 결과 읽기 (`read_from_fpga_to_file`)
 - Layer 14 출력($8 \times 8 \times 195$)과 Layer 20 출력($16 \times 16 \times 195$)을 UART로 수신
10. Host: 결과 검증 및 후처리
 - 소프트웨어 결과와 비교 (`verify_result`)
 - YOLO 디코딩, NMS, 바운딩 박스 그리기 (`write_image`)
11. Host: COM 포트 닫기 (`close_port`)

6-9. 왜 MicroBlaze가 필요한가?

"YOLO 가속기가 직접 Host PC와 통신하면 안 되나?"라고 생각할 수 있는데, MicroBlaze가 있으면 좋은 이유가 있습니다.

유연한 제어: UART 프로토콜 처리, DRAM 초기화, 주소 설정 등의 복잡한 제어 로직을 C 코드로 작성할 수 있습니다. 이것을 하드웨어(Verilog)로 구현하면 매우 복잡하고 디버깅도 어렵습니다.

디버깅: MicroBlaze에서 `printf()`를 사용해 UART로 디버그 메시지를 출력할 수 있습니다. 하드웨어 개발 중에 "지금 어떤 상태인지"를 확인하는 데 매우 유용합니다.

후처리 보조: YOLO 레이어의 sigmoid, softmax 같은 복잡한 수학 함수를 MicroBlaze에서 소프트웨어로 처리할 수 있습니다.

6-10. UART 통신의 기본

UART가 어떻게 동작하는지 간단히 알아봅시다. UART는 데이터를 **한 비트씩 순서대로** 보내는 시리얼 통신입니다.

한 바이트(8비트)를 보내는 과정:

1. **Idle 상태:** 신호선이 High(1)를 유지
2. **Start bit:** Low(0)으로 떨어짐 — "지금부터 데이터 시작"
3. **Data bits:** LSB부터 MSB까지 8비트를 순서대로 전송
4. **Stop bit:** High(1)로 돌아감 — "데이터 끝"

230,400 bps라는 것은 1초에 230,400비트를 보낼 수 있다는 뜻이고, 한 바이트에 $\text{start} + 8\text{data} + \text{stop} = 10$ 비트가 필요하므로, 초당 약 23,040 바이트(약 22.5 KB/s)를 전송할 수 있습니다. $256 \times 256 \times 3 =$ 약 192KB 이미지를 보내려면 약 8.5초가 걸립니다. 느리지만, 이것은 "데이터 전송 시간"이고 "연산 시간"과는 별개입니다. 연산 속도를 측정할 때는 FPGA 내부 처리 시간만 봅니다.

6-11. RTL 시뮬레이션 vs FPGA 구현

이 시스템에는 두 가지 검증 방법이 있습니다:

RTL 시뮬레이션 — Vivado의 시뮬레이터에서 Verilog 코드를 실행합니다. 실제 FPGA 보드 없이도 할 수 있습니다. 테스트벤치(`yolo_engine_tb.v`)가 DRAM 대신 SRAM 모델을 사용하고, UART 대신 직접 제어 신호를 주입합니다. 클럭 단위로 파형을 확인할 수 있어서 **디버깅에 최적**이지만, 실행 속도가 매우 느립니다(한 레이어 시뮬레이션에 수 분~수십 분).

FPGA 구현 — 실제 FPGA 보드에 설계를 다운로드해서 실행합니다. 실제 클럭 속도(~200MHz)로 동작하므로 빠르지만, 내부 상태를 직접 관찰하기 어려워서 디버깅이 까다롭습니다.

일반적인 개발 순서: 먼저 RTL 시뮬레이션으로 기능을 검증한 뒤, 문제가 없으면 FPGA에 구현합니다.

6차시 핵심 정리:

1. 시스템은 Host PC + FPGA 보드(Nexys A7)로 구성, UART로 연결
 2. FPGA 안에는 MicroBlaze(CPU), YOLO 가속기, UART, MIG, DDR2 DRAM이 AXI 버스로 연결
 3. YOLO 가속기는 AXI Slave(설정 수신)와 AXI Master(DRAM 접근)를 모두 가짐
 4. 데이터 이동 3단계: Host→DRAM(UART), DRAM→Global Buffer(DMA), Buffer→PE(내부)
 5. 전체 흐름: Host가 데이터 전송 → 가속기 시작 → 연산 → 결과 회수 → 후처리
 6. RTL 시뮬레이션으로 먼저 검증, 그 다음 FPGA 구현
-

질문 1: MicroBlaze는 어떻게 프로그램이 돌아가는 건가?

네, 맞는 방향입니다. MicroBlaze는 범용 **CPU**의 구조를 갖추고 있어서 C 프로그램을 실행할 수 있습니다.

다만 ARM이나 x86과는 출발점이 다릅니다. ARM이나 x86은 **실리콘 칩으로 미리 제작된** 프로세서입니다. 반면 MicroBlaze는 Xilinx가 설계한 **프로세서의 설계도(HDL 코드)**이고, 이것을 FPGA의 LUT, 플립플롭, BRAM 같은 범용 논리 자원으로 **구현(synthesize)**하는 것입니다.

하지만 구현 방식만 다를 뿐, MicroBlaze 안에는 진짜 CPU가 갖춰야 할 핵심 요소가 모두 들어있습니다:

프로그램 카운터(PC) — 다음에 실행할 명령어의 주소를 추적합니다.

레지스터 파일 — 32개의 32비트 범용 레지스터가 있습니다. ARM의 r0~r15와 비슷한 역할입니다.

ALU — 덧셈, 뺄셈, 논리 연산, 시프트 등을 수행합니다.

명령어 디코더 — 메모리에서 읽어온 명령어(machine code)를 해석해서 ALU에 무슨 연산을 시킬지 결정합니다.

메모리 인터페이스 — 명령어를 읽고(Instruction Fetch), 데이터를 읽고 쓰는(Load/Store) 인터페이스가 있습니다.

MicroBlaze의 명령어 세트(ISA)는 ARM이나 x86과는 다른 **고유한 RISC 아키텍처**입니다. 하지만 Xilinx가 제공하는 **GCC 기반 컴파일러**가 C 코드를 MicroBlaze 기계어로 변환해주므로, 여러분은 그냥 C로 프로그래밍하면 됩니다. Vitis IDE에서 C 코드를 작성하고 빌드하면, MicroBlaze용 바이너리가 생성되어 FPGA 내부의 On-chip Memory에 로드됩니다.

성능 면에서는 ARM이나 x86에 비해 훨씬 느립니다. MicroBlaze는 단일 파이프라인의 단순한 구조이고, 클럭도 100~200MHz 정도입니다. 하지만 이 프로젝트에서 MicroBlaze가 하는 일은 UART 통신 제어, 레지스터 설정, 완료 대기 같은 **가벼운 제어 작업**이라 충분합니다. 무거운 연산은 YOLO 가속기(전용 하드웨어)가 담당하니까요.

비유하자면, MicroBlaze는 공장의 **관리자**이고 YOLO 가속기는 **생산 라인**입니다. 관리자가 "이 재료로 이 제품을 만들어라"라고 지시하면, 생산 라인이 고속으로 작업합니다. 관리자 자체는 빠를 필요가 없고, 지시만 정확히 내리면 됩니다.

질문 2: Layer 15, 21 (YOLO 레이어)은 FPGA에서 돌아가지 않나?

정확합니다. YOLO 레이어(Layer 15, 21)의 후처리는 FPGA의 하드웨어 가속기에서 실행되지 않습니다.

그 이유를 구체적으로 봅시다. YOLO 레이어가 하는 일은:

Sigmoid 함수: $\sigma(x) = 1 / (1 + e^{(-x)})$. 바운딩 박스의 중심 좌표(x, y)와 객체 존재 확률(objectness)에 적용됩니다. 지수함수(exp)가 필요합니다.

Softmax 함수: 60개 클래스의 확률 분포를 계산합니다. 역시 지수함수와 나눗셈이 필요합니다.

바운딩 박스 디코딩: 격자 좌표를 실제 이미지 좌표로 변환합니다. 부동소수점 곱셈과 덧셈이 필요합니다.

NMS (Non-Maximum Suppression): 겹치는 바운딩 박스들 중 가장 신뢰도가 높은 것만 남기는 작업입니다. IoU 계산(면적 비율), 정렬, 조건 분기 등 복잡한 로직이 필요합니다.

이것들을 하드웨어로 구현하지 않는 이유:

첫째, 연산량이 매우 적습니다. Layer 14의 출력은 $8 \times 8 \times 195 = 12,480$ 개 값이고, Layer 20은 $16 \times 16 \times 195 = 49,920$ 개 값입니다. 전체 네트워크의 합성곱 연산량(약 10억 MAC)에 비하면 무시할 수 있는 수준입니다. 하드웨어로 가속할 가치가 없습니다.

둘째, 구현이 복잡합니다. 지수함수, 나눗셈, 정렬 같은 연산은 하드웨어로 구현하기 까다롭고, 많은 FPGA 자원을 소모합니다. 그 자원을 MAC 유닛에 투입하는 것이 훨씬 효율적입니다.

셋째, 부동소수점이 필요합니다. YOLO 후처리는 좌표 변환이나 확률 계산에서 float 정밀도가 필요합니다. 양자화(INT8)로는 정확한 처리가 어렵습니다.

따라서 이 후처리는 **MicroBlaze에서 소프트웨어로 처리하거나, Host PC에서 처리합니다.** 이 프로젝트에서는 주로 Host PC 쪽에서 처리하는 방식을 사용합니다. FPGA에서 합성곱 결과(Layer 14, 20의 출력)를 DRAM에 저장하면, Host가 UART로 읽어와서 후처리를 수행합니다.

질문 3: YOLO 가속기의 역할은 Layer 14, 20의 출력까지이고 나머지는 Host인가?

네, 정확하게 이해하셨습니다. 역할 분담을 명확히 정리하면:

YOLO 가속기(하드웨어)가 하는 일:

- ☐ Layer 0 ~ Layer 14: 합성곱, Bias 덧셈, ReLU, Max Pooling, Route, Upsample
- ☐ Layer 16 ~ Layer 20: 합성곱, Route, Upsample (두 번째 YOLO 출력 경로)
- ☐ 결과적으로 $8 \times 8 \times 195$ 텐서와 $16 \times 16 \times 195$ 텐서를 DRAM에 저장

Host PC(소프트웨어)가 하는 일:

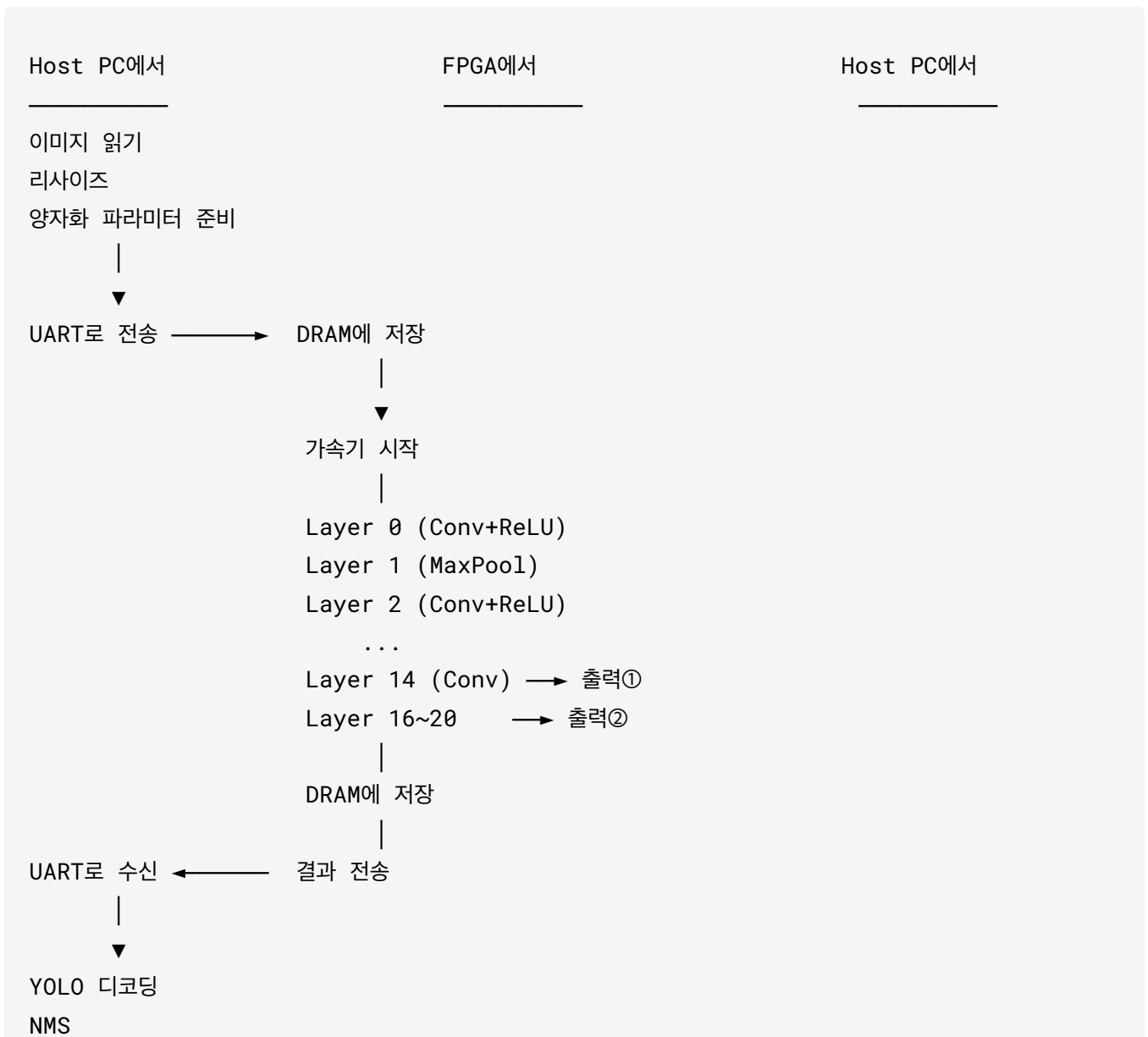
추론 전:

- ☐ 이미지 읽기, 리사이즈(1920×1080 → 256×256), 정규화
- ☐ 양자화된 파라미터(weight, bias, scale)를 FPGA DRAM에 전송
- ☐ 입력 이미지를 FPGA DRAM에 전송
- ☐ 가속기 시작 명령

추론 후:

- ☐ FPGA DRAM에서 Layer 14, 20 출력을 읽어옴
- ☐ YOLO 후처리 (sigmoid, softmax, 바운딩 박스 디코딩)
- ☐ NMS (중복 박스 제거)
- ☐ 결과 이미지 생성 (바운딩 박스 그리기)
- ☐ mAP 계산 (정확도 평가)

이 구조를 그림으로 보면:



이 설계 철학은 AI 가속기 분야에서 매우 일반적입니다. Google의 TPU, NVIDIA의 GPU 가속도 마찬가지로, 연산량의 99%를 차지하는 행렬 연산(합성곱)은 전용 하드웨어에서, 복잡하지만 가벼운 후처리는 범용 CPU에서 처리합니다. 이것이 가장 효율적인 분업입니다.

7차시: 테스트 벡터 생성과 HW 검증 방법론

이번 차시에서는 소프트웨어와 하드웨어를 연결하는 검증(verification) 방법을 다룹니다. "내가 설계한 하드웨어가 올바르게 동작하는지 어떻게 확인하는가?"에 대한 답이자, 실제 개발 과정에서 가장 많은 시간을 쓰게 되는 단계입니다.

7-1. 검증의 기본 철학 — Golden Reference

하드웨어 설계에서 검증의 핵심 원리는 단순합니다:

"같은 입력을 넣었을 때, 하드웨어의 출력이 소프트웨어의 출력과 일치하면 하드웨어가 맞다."

소프트웨어는 이미 검증이 끝난 상태입니다 — FP32 모드에서 mAP 81.76%, 양자화 모드에서 55.04%(또는 여러분이 최적화한 값)가 나오는 것을 확인했으니까요. 이 소프트웨어의 결과를 **Golden Reference(황금 기준)** 라고 부릅니다.

하드웨어 설계 과정은 이렇습니다:

1. 소프트웨어를 실행해서 각 레이어의 **입력과 출력을 hex 파일로 저장** (테스트 벡터 생성)
2. 하드웨어 시뮬레이션에서 **같은 입력을 넣고** 출력을 확인
3. 하드웨어 출력과 소프트웨어 출력을 **바이트 단위로 비교**
4. 불일치가 있으면 → 하드웨어 버그를 찾아서 수정
5. 완전히 일치하면 → 다음 레이어로 넘어감

7-2. 테스트 벡터가 무엇인가?

테스트 벡터(test vector)는 "이 입력을 넣으면 이 출력이 나와야 한다"는 입출력 쌍입니다. 이 프로젝트에서 테스트 벡

터는 소프트웨어가 생성하는 hex 파일들입니다.

5차시에서 이미 봤듯이, `forward_convolutional_layer_q()` 함수 안에 `// Write data for the HW verification` 블록이 있습니다. 이 코드가 각 합성곱 레이어의 입력과 출력을 hex 파일로 저장합니다.

각 레이어에 대해 생성되는 파일들:

입력 테스트 벡터:

`log_feamap/CONV00_input.hex` ← Layer 0의 입력 (UINT8, 256×256×3개 값)

출력 테스트 벡터 (Golden Reference):

`log_feamap/CONV00_output.hex` ← Layer 0의 출력 (UINT8, 256×256×16개 값)

파라미터:

`log_param/CONV00_param_weight.hex` ← 양자화된 가중치 (INT8)

`log_param/CONV00_param_biases.hex` ← 양자화된 bias (INT16)

`log_param/CONV00_param_scales.hex` ← de-scaling 값 (UINT16)

하드웨어 검증 시나리오: 입력 hex와 파라미터 hex를 하드웨어에 넣고, 하드웨어가 출력한 결과를 출력 hex와 비교합니다. 모든 바이트가 일치하면 통과입니다.

7-3. hex 파일의 형식

hex 파일의 내부를 들여다봅시다. `CONV00_input.hex`의 내용은 이런 형태입니다:

```
61
61
61
5e
5d
5c
...
```

한 줄에 하나의 값이 **16진수(hexadecimal)**로 기록되어 있습니다. `61`은 10진수로 97입니다. 이것이 입력 이미지의 첫 번째 픽셀의 양자화된 값(UINT8)입니다.

데이터 순서는 코드에 나와 있듯이 **[Channel, Width, Height]** 순서입니다:

```
for (int chn = 0; chn < l.c; chn++) {           // 채널 루프 (바깥)
    for (int idx = 0; idx < l.h * l.w; idx++) {   // 픽셀 루프 (안쪽)
        // chn번째 채널의 idx번째 픽셀
    }
}
```

Layer 0의 경우 채널이 3(R, G, B)이고 크기가 256×256이므로, R 채널의 65,536개 픽셀이 먼저 나오고, 그 다음 G 채널 65,536개, 마지막으로 B 채널 65,536개가 순서대로 나열됩니다. 총 196,608줄입니다.

가중치 hex(CONV00_param_weight.hex)도 비슷한 구조입니다. Layer 0의 경우 필터 16개 × 3×3×3 = 432개 값이 순서대로 나열됩니다. 첫 번째 필터의 27개 가중치가 먼저 나오고, 그 다음 두 번째 필터의 27개, ... 이런 식입니다.

7-4. 테스트벤치에서 hex 파일을 읽는 방법

Verilog 테스트벤치에서 hex 파일을 메모리에 로드하는 방법을 봅시다. 강의자료의 예시:

```
module tb_layer_00_no_pool;
parameter CLK_PERIOD      = 10;           // 100MHz

parameter LAYER_IFM_FILE   = "C:/skeleton/bin/log_feamap/CONV00_input.hex";
parameter LAYER_OFM_FILE   = "C:/skeleton/bin/log_feamap/
CONV00_output.hex";
parameter LAYER_WGT_FILE   = "C:/skeleton/bin/log_param/
CONV00_param_weight.hex";
parameter LAYER_BIASES_FILE = "C:/skeleton/bin/log_param/
CONV00_param_biases.hex";
parameter LAYER_SCALES_FILE = "C:/skeleton/bin/log_param/
CONV00_param_scales.hex";

parameter IFM_WIDTH        = 256;
parameter IFM_HEIGHT       = 256;
parameter IFM_CHANNEL       = 3;
parameter OFM_CHANNEL       = 16;
```

```

parameter FX          = 3;      // 필터 가로
parameter FY          = 3;      // 필터 세로
parameter ACT_RELU_ENA = 1;      // ReLU 활성화
parameter SCALE_SHIFT = 7;      // de-scaling 시프트 값

```

이 파라미터들이 레이어의 스펙을 완전히 정의합니다. Layer 0을 테스트하려면 위와 같이 설정하고, Layer 2를 테스트하려면 파일 경로와 크기를 Layer 2에 맞게 바꿔주면 됩니다:

```

// Layer 2 설정 예시
parameter LAYER_IFM_FILE = "C:/skeleton/bin/log_feamap/CONV02_input.hex";
parameter LAYER_OFM_FILE = "C:/skeleton/bin/log_feamap/CONV02_output.hex";
parameter LAYER_WGT_FILE = "C:/skeleton/bin/log_param/
CONV02_param_weight.hex";
parameter IFM_WIDTH      = 128;
parameter IFM_HEIGHT     = 128;
parameter IFM_CHANNEL    = 16;
parameter OFM_CHANNEL    = 32;

```

7-5. 외부 메모리 모델 — DRAM을 시뮬레이션으로 대체

실제 FPGA에서는 DDR2 DRAM을 사용하지만, RTL 시뮬레이션에서는 DDR2의 복잡한 타이밍을 모델링하기 어렵습니다. 그래서 간단한 **SRAM 모델**로 대체합니다.

yolo_engine_tb.v를 보면:

```

sram #(
    .FILE_NAME(IFM_FILE),
    .SIZE(2**MEM_ADDRW),
    .WL_ADDR(MEM_ADDRW),
    .WL_DATA(MEM_DW)
) u_ext_mem_input (
    .clk    (clk),
    .addr   (mem_addr),
    .wdata  (mem_di),
    .rdata  (mem_do),
    .ena    (1'b0)    // Read only

```



```
);
```

이 SRAM 모델은 시뮬레이션 시작 시 hex 파일에서 데이터를 읽어서 메모리를 초기화합니다. 가속기의 DMA가 이 메모리에서 데이터를 읽으면, 마치 진짜 DRAM에서 읽는 것처럼 동작합니다.

중요한 제약이 하나 있습니다: 이 SRAM 모델은 **16비트 워드**만 지원합니다. 소프트웨어의 hex 파일은 8비트(한 바이트) 단위인데, 하드웨어 메모리 모델은 16비트 단위이므로, **8비트 데이터를 16비트로 패킹**해야 합니다.

예를 들어 8비트 데이터 {64, 64, 62, 5d, 5c, 58, 58, 56} 이 있으면:

```
8비트 원본:    64  64  62  5d  5c  58  58  56
16비트 패킹:   7064  0070  6e62  006e  6a5d  0067
```

두 바이트를 합쳐서 16비트 한 줄로 만드는 것입니다. 데이터의 순서와 바이트 배치(endianness)는 여러분의 하드웨어 설계에 맞게 결정해야 합니다.

7-6. 테스트벤치의 동작 흐름

yolo_engine_tb.v의 `initial` 블록을 보면 시뮬레이션이 어떻게 진행되는지 알 수 있습니다:

```
initial begin
    rstn = 1'b0;           // ① 리셋 활성화
    i_0 = 0;
    i_1 = 0;
    i_2 = (4*256*256)*4;    // 출력 저장 주소 설정

    #(4*CLK_PERIOD) rstn = 1'b1;    // ② 리셋 해제
    #(100*CLK_PERIOD)
        @(posedge clk)
            i_0 = 32'd1;           // ③ 가속기 시작 (i_ctrl_reg0 = 1)

    #(100*CLK_PERIOD)
        @(posedge clk)
            i_0 = 32'd0;           // ④ 시작 신호 해제
```

```

while(!network_done) begin          // ⑤ 완료 대기
    #(128*CLK_PERIOD) @(posedge clk);
end

#(100*CLK_PERIOD)
@(posedge clk) $stop;              // ⑥ 시뮬레이션 종료
end

```

단계별로:

1. 리셋으로 모든 모듈을 초기 상태로
2. 리셋을 풀어서 정상 동작 시작
3. `i_ctrl_reg0`에 1을 써서 가속기에 "시작하라"고 명령
4. 시작 신호를 다시 0으로 (펄스 형태)
5. `network_done` 신호가 1이 될 때까지 기다림
6. 완료되면 `$stop`으로 시뮬레이션을 멈추고 파형을 확인

실제 FPGA에서는 MicroBlaze가 `i_ctrl_reg0`에 값을 쓰지만, 시뮬레이션에서는 테스트벤치가 직접 값을 넣어줍니다. MicroBlaze 없이도 가속기만 독립적으로 테스트할 수 있는 것이 큰 장점입니다.

7-7. PRELOAD 기능 — 특정 레이어만 테스트하기

테스트벤치에는 `PRELOAD`라는 유용한 기능이 있습니다:

```

`ifdef PRELOAD
    preload = 1'b0;
    preload_layer_idx = 4;    // Layer 4부터 시작
`endif

```

전체 네트워크를 Layer 0부터 끝까지 시뮬레이션하면 매우 오래 걸립니다. `PRELOAD`를 사용하면 **특정 레이어의 입력 데이터를 외부 메모리에 미리 로드해두고, 그 레이어부터 시뮬레이션을 시작할 수** 있습니다.

예를 들어 Layer 4를 디버깅 중이라면, Layer 0~3의 시뮬레이션을 건너뛰고 Layer 4의 입력 (`CONV04_input.hex`)을 직접 메모리에 넣은 뒤 Layer 4부터 실행합니다. 이렇게 하면 디버깅 시간이 크게 줄어듭니다.

7-8. 결과 확인 방법 — 시각적 검증과 수치적 검증

하드웨어 시뮬레이션 결과를 확인하는 방법은 두 가지입니다.

시각적 검증 (Visual Verification)

테스트벤치에는 `bmp_image_writer` 모듈이 포함되어 있습니다:

```
bmp_image_writer #(
    .OUTFILE(CONV_OUTPUT_IMG00),
    .WIDTH(IFM_WIDTH),
    .HEIGHT(IFM_HEIGHT)
) u_bmp_image_writer_00 (
    .clk  (clk),
    .rstn (rstn),
    .din  (i_WDATA[7:0]),
    .vld  (i_WVALID),
    .frame_done()
);
```

이 모듈은 가속기가 DMA로 출력을 쓸 때, 그 데이터를 BMP 이미지 파일로 저장합니다. 출력 채널별로 하나의 이미지가 생성됩니다(CONV00_output_ch00.bmp, ch01.bmp, ...). 이 이미지를 눈으로 보면 "대략 맞는 것 같다"는 직관적 확인이 가능합니다.

하지만 강의자료가 명확히 경고합니다:

"It is only for visual verification. You must write data in hex files and compare them with the outputs from the software."

눈으로 보는 것만으로는 부족하고, 반드시 수치적으로 비교해야 합니다.

수치적 검증 (Numerical Verification)

하드웨어 출력을 hex 파일로 저장한 뒤, 소프트웨어가 생성한 golden reference hex와 **바이트 단위로 비교**합니다. 이것은 스크립트나 간단한 프로그램으로 자동화할 수 있습니다.

FPGA 구현 단계에서는 `verify_result()` 함수가 이 비교를 수행합니다:

```
verify_result(  
    "C:/skeleton/bin/log_feamap/CONV00_input_32b.hex",    // SW 결과 (Golden)  
    "C:/skeleton/bin/log_result_fpga/CONV00_input_32b.hex", // HW 결과  
    256 * 256                                             // 비교할 데이터 수  
);
```

모든 값이 일치하면 "PASS", 하나라도 다르면 "FAIL"과 함께 어느 위치에서 불일치가 발생했는지 보고합니다.

7-9. 단계적 검증 전략

실제 개발에서는 한 번에 전체 네트워크를 검증하지 않습니다. **작은 단위부터 점진적으로** 검증합니다.

1단계: 단위 모듈 테스트

개별 모듈(곱셈기, MAC 유닛, BRAM)을 각각의 테스트벤치로 검증합니다. 예를 들어 `mac_tb.v` 는 MAC 유닛 하나에 알려진 입력을 넣고 출력을 확인합니다.

2단계: 합성곱 커널 테스트

`conv_kern_tb.v` 는 여러 MAC을 조합한 합성곱 커널이 한 출력 픽셀을 올바르게 계산하는지 검증합니다. 소프트웨어에서 생성한 가중치와 입력을 넣고, 출력이 기대값과 일치하는지 확인합니다.

3단계: 단일 레이어 테스트

`conv_layer_tb.v` 는 컨트롤러가 합성곱 커널을 반복 호출해서 한 레이어 전체를 처리하는지 검증합니다. Layer 0의 입력($256 \times 256 \times 3$)을 넣고 출력($256 \times 256 \times 16$)이 golden reference와 일치하는지 비교합니다.

4단계: 다중 레이어 테스트

`cnn_ctrl_multi_layer_tb.v` 는 컨트롤러가 여러 레이어를 순서대로 처리하는지 검증합니다. Layer 0 → (Pooling) → Layer 2 → (Pooling) → Layer 4처럼 연속 실행이 올바른지 확인합니다.

5단계: 전체 네트워크 테스트

`yolo_engine_tb.v` 는 최상위 모듈을 DMA와 외부 메모리까지 포함해서 테스트합니다. Layer 0부터 Layer 20까지 전체를 실행하고, 최종 출력이 소프트웨어 결과와 일치하는지 확인합니다.

이 단계를 순서대로 밟아야 합니다. 1단계에서 MAC이 틀리면, 3단계에서 레이어 전체가 틀릴 수밖에 없습니다. 작은

단위에서 버그를 잡은 뒤 큰 단위로 넘어가는 것이 디버깅의 핵심입니다.

7-10. 흔한 불일치 원인과 디버깅 팁

하드웨어와 소프트웨어의 결과가 불일치할 때 자주 나타나는 원인들:

데이터 순서 불일치: 소프트웨어는 [Channel, Height, Width] 순서로 저장하는데 하드웨어는 [Height, Width, Channel] 순서로 읽는 경우. hex 파일의 데이터 배치를 하드웨어의 메모리 접근 패턴과 정확히 맞춰야 합니다.

비트 폭 문제: 누산기가 32비트여야 하는데 16비트로 설계하면 오버플로우가 발생합니다. 특히 채널 수가 많은 후반 레이어에서 문제가 됩니다.

패딩 처리 오류: 가장자리 픽셀에서 패딩(0 삽입)이 제대로 안 되면, 가장자리의 출력만 틀어집니다. 가운데는 맞는데 테두리만 다르다면 패딩을 의심해야 합니다.

De-scaling 오류: 나눗셈이나 비트 시프트의 반올림 방식이 소프트웨어와 다를 수 있습니다. 소프트웨어는 floor(내림)인데 하드웨어는 truncation(절삭)일 수 있어서, 1 차이가 발생할 수 있습니다. 이 경우 ± 1 오차는 허용하는 것이 일반적입니다.

signed vs unsigned 혼동: 가중치는 signed(INT8), 입력은 unsigned(UINT8)인데 둘 다 unsigned로 처리하면 음수 가중치가 엉뚱한 큰 양수로 해석됩니다.

7차시 핵심 정리:

1. 검증의 핵심 = 같은 입력에 대해 HW 출력이 SW 출력(Golden Reference)과 일치하는지 비교
 2. 소프트웨어가 생성하는 hex 파일(log_feamap, log_param)이 테스트 벡터
 3. 테스트벤치는 SRAM 모델로 DRAM을 대체하고, hex 파일로 초기화
 4. PRELOAD 기능으로 특정 레이어만 독립적으로 테스트 가능
 5. 검증은 단위 모듈 → 커널 → 레이어 → 다중 레이어 → 전체 네트워크 순서로 단계적으로
 6. 시각적 검증(BMP)은 참고용이고, 반드시 수치적 비교(hex 비교)를 해야 함
-

8차시: Verilog HDL 기초

8-1. 왜 지금 Verilog를 배워야 하는가?

지금까지 1~7차시에서 우리는 "무엇을 만들 것인지"를 배웠습니다. CNN이 뭔지, 22개 레이어가 어떻게 생겼는지, 양자화가 뭔지, 테스트 벡터는 어떻게 쓰는지를 알게 됐죠.

이제부터는 "어떻게 만드는지"를 배울 차례입니다. FPGA 위에 돌아갈 하드웨어 가속기를 설계하려면, 그 하드웨어를 "기술(describe)"할 수 있는 언어가 필요합니다. 그 언어가 바로 **Verilog HDL**입니다.

C언어가 소프트웨어를 만드는 언어라면, Verilog는 하드웨어를 만드는 언어입니다. 우리 프로젝트의 `yolohw/src/` 폴더에 있는 `yolo_engine.v`, `mac.v`, `adder_tree.v`, `cnn_ctrl.v` 같은 파일들이 전부 Verilog로 작성되어 있습니다.

8-2. HDL (Hardware Description Language)이란?

HDL은 말 그대로 **하드웨어를 기술하는 언어**입니다. 강의 자료에 따르면, 가장 널리 쓰이는 HDL 두 가지가 있습니다:

- **Verilog HDL** (보통 줄여서 "Verilog"라 부름) — 우리 프로젝트에서 사용
- **VHDL** (Very High-Speed Integrated Circuits HDL) — 유럽 쪽에서 많이 사용

HDL의 핵심 특징은 이렇습니다:

- 설계를 매우 추상적인 수준에서 기술할 수 있음
- 설계 초기 단계에서 기능 검증(시뮬레이션)이 가능
- 프로그래밍과 비슷하게 작성하지만, **결과물은 소프트웨어가 아니라 하드웨어**

C언어 vs Verilog: 근본적인 차이

이것은 초보자가 가장 혼동하기 쉬운 부분이라 확실히 짚고 넘어가겠습니다.

C언어는 명령어가 **순차적으로(한 줄씩)** 실행됩니다. `a = 1;` 다음에 `b = 2;` 가 오면, a가 먼저 설정되고 나서 b가

설정됩니다. CPU가 한 줄씩 읽어서 처리하니까요.

Verilog는 하드웨어를 기술합니다. 하드웨어에서는 전선(wire)과 게이트(gate)가 **동시에(병렬로) 동작**합니다. 예를 들어 AND 게이트와 OR 게이트가 나란히 있으면, 입력 신호가 들어오는 순간 둘 다 **동시에** 결과를 냅니다. 하나가 먼저 계산되고 다른 하나를 기다리는 게 아닙니다.

비유하자면: C언어는 **요리 레시피**(1단계 하고 → 2단계 하고 → 3단계), Verilog는 **회로 설계도**(전선 연결하면 전기가 동시에 흐름)입니다.

8-3. Verilog의 기본 단위: Module

Verilog에서 가장 기본이 되는 빌딩 블록은 **module**입니다. 하나의 module은 하나의 하드웨어 블록을 나타냅니다. IC 칩 하나라고 생각해도 좋습니다.

강의 슬라이드에 나온 module의 기본 구조를 보겠습니다:

```
module 모듈이름 (  
    // 포트 리스트, 포트 선언  
);  
    // 파라미터 (있다면)  
  
    // wire, reg 등 변수 선언  
  
    // 하위 모듈 인스턴스화 (다른 모듈 가져다 쓰기)  
  
    // 데이터 흐름 문 (assign)  
  
    // always 블록, initial 블록 (동작 기술)  
  
    // task, function (있다면)  
  
endmodule
```

실제 예를 들어보겠습니다. 가장 간단한 곱셈기를 생각해 봅시다:

```

module mul (
    input      clk,          // 클럭 입력
    input  [7:0] w,          // 가중치 8비트
    input  [7:0] x,          // 입력값 8비트
    output [15:0] y          // 결과 16비트
);
    // ... 내부 구현 ...
endmodule

```

이게 무슨 뜻이냐면: "나는 `mul` 이라는 하드웨어 블록을 만들 건데, 이 블록은 클럭(`clk`) 하나, 8비트 입력 2개(`w`, `x`), 16비트 출력 1개(`y`)를 가진다"라고 선언하는 겁니다. 마치 IC 칩의 핀 배치를 정의하는 것과 같습니다.

왜 `module` 과 `endmodule` 로 감싸는가? 하드웨어의 경계를 명확히 하기 위해서입니다. 실제 회로에서도 칩 하나의 경계가 명확하듯이, Verilog에서도 모듈 하나의 경계를 명확히 정합니다.

8-4. 숫자 표현법 (Lexical Conventions)

하드웨어를 다루다 보면 "이 신호는 몇 비트짜리인가?"가 매우 중요합니다. Verilog에서 숫자를 표현하는 방법을 알아봅시다.

기본 형식: `<비트수> ' <진법> <값>`

<code>4'b1001</code>	→ 4비트 이진수 (= 십진수 9)
<code>16'habcd</code>	→ 16비트 16진수 (= 십진수 43981)
<code>8'd255</code>	→ 8비트 십진수 255

여기서 진법 코드는:

- `b` = binary (2진수)
- `h` = hexadecimal (16진수)
- `d` = decimal (10진수)
- `o` = octal (8진수)

크기를 안 쓰면?

2007 → 그냥 32비트 십진수 (기본값이 32비트)
'habc → 32비트 16진수

크기를 안 쓰면 자동으로 32비트로 처리됩니다.

특수한 값들

x → 알 수 없는 값 (unknown). 시뮬레이션에서 빨간색으로 보임
z → 하이 임피던스 (아무것도 연결 안 된 상태)

x는 디버깅할 때 자주 보게 됩니다. 시뮬레이션 파형에서 빨간색으로 나타나는 신호가 있으면 "이 값이 아직 결정되지 않았다"는 뜻입니다. 초기화를 안 했거나 연결이 안 됐을 때 나옵니다.

밑줄(_)을 구분자로 사용 가능

16'b0101_1001_1110_0000 → 가독성을 위한 밑줄 (의미 없음)

긴 이진수를 4비트씩 끊어 읽을 수 있어 편합니다.

왜 이게 중요한가?

우리 프로젝트에서 양자화된 가중치는 INT8(-127~+127), 입력은 UINT8(0~255)이라고 5차시에서 배웠습니다. 이걸 Verilog에서 표현할 때 8'b, 8'd 같은 표기를 사용하게 됩니다. 비트 폭을 잘못 쓰면 값이 잘리거나 넘치는 버그가 생기는데, 7차시에서 배운 "비트 폭 부족" 버그가 바로 이것입니다.

8-5. 데이터 타입: wire vs reg

이것이 Verilog 초보자의 **최대 혼동 포인트**입니다. 하나씩 설명하겠습니다.

wire (와이어)

말 그대로 **전선**입니다. 하드웨어에서 두 부품을 연결하는 전선처럼, 값을 **전달**만 합니다. 자체적으로 값을 저장하지 않습니다. 연결된 곳에서 신호가 오면 그걸 그대로 전달합니다.

```
wire [7:0] data_bus;      // 8비트 와이어 선언
wire      enable;        // 1비트 와이어
```

wire는 `assign` 문이나 **모듈의 출력**에 연결합니다:

```
assign data_bus = a + b;  // a+b 결과가 data_bus로 흐름
```

reg (레지스터)

이름이 "레지스터"라서 혼동을 일으키는데, 반드시 하드웨어 레지스터(플립플롭)를 뜻하는 건 아닙니다. Verilog에서 `reg`는 "**값을 저장(hold)할 수 있는 변수**"를 의미합니다. `always` 블록이나 `initial` 블록 안에서 값을 대입할 수 있습니다.

```
reg [7:0] counter;       // 8비트 reg
reg      flag;           // 1비트 reg
```

핵심 구분법

| `assign` 문에서 쓸 것이면 `wire`, `always` / `initial` 블록 안에서 값을 넣을 것이면 `reg`

테스트벤치에서 실제 예를 볼까요? 우리 프로젝트의 `yolo_engine_tb.v`를 보면:

```
reg clk;                  // 클럭 - 테스트벤치가 직접 값을 바꿔야 하므로 reg
```

```

reg rstn;                                // 리셋 - 마찬가지로

wire [31:0] M_AWADDR;                    // AXI 주소 - 가속기 모듈이 출력하는 것을 받으므로 wire
wire [31:0] M_WDATA;                     // AXI 데이터 - 마찬가지로

```

클럭은 테스트벤치가 `initial` 블록에서 직접 토글시키는 신호이므로 `reg`, AXI 주소나 데이터는 테스트 대상 모듈 (`yolo_engine`)이 생성하는 출력을 받아오는 전선이므로 `wire`입니다.

8-6. 포트(Port) 선언

모듈의 "핀"에 해당하는 것이 포트입니다. 세 종류가 있습니다:

- `input`: 외부에서 이 모듈로 들어오는 신호
- `output`: 이 모듈에서 외부로 나가는 신호
- `inout`: 양방향 (거의 안 씀, I2C 같은 특수한 경우에만)

```

module mac (
    input          clk,          // 클럭 (1비트)
    input          rstn,         // 리셋 (1비트, active low)
    input          vld_i,        // 입력 유효 신호
    input [127:0]  win,          // 가중치 128비트 = 16개 × 8비트
    input [127:0]  din,          // 입력 128비트 = 16개 × 8비트
    output [19:0]  acc_o,        // 누적합 출력 20비트
    output         vld_o         // 출력 유효 신호
);

```

이 예는 강의 자료의 MAC 모듈에서 가져온 것입니다. 이 MAC이 바로 우리 가속기의 핵심 연산부인데, 16개의 8비트 곱셈을 수행하고 결과를 더해서 20비트 누적합을 출력합니다.

포트 연결 방식 — 모듈을 "인스턴스화"할 때(즉 가져다 쓸 때) 두 가지 방식이 있습니다:

```

// 1) 이름 연결 (Named Association) - 권장!
mul u_mul_0 (

```

```

        .clk(clk),
        .w(weight_data),
        .x(input_data),
        .y(product)
    );

// 2) 위치 연결 (Positional Association) - 비추천, 실수 유발
mul u_mul_0 (clk, weight_data, input_data, product);

```

이름 연결 방식은 `.포트이름(연결할신호)` 형태로 쓰는 건데, 어떤 포트에 뭘 연결하는지 명확해서 실무에서 거의 항상 이 방식을 씁니다. 우리 프로젝트의 `yolo_engine_tb.v` 를 보면 AXI SRAM 인터페이스를 이 방식으로 연결하고 있습니다:

```

axi_sram_if #(.MEM_ADDRW(MEM_ADDRW), .MEM_DW(MEM_DW), ...)
u_axi_ext_mem_if_input(
    .ACLK(clk),
    .ARESETn(rstn),
    .AWID(M_AWID),
    .AWADDR(M_AWADDR),
    ...
);

```

8-7. Parameter와 Localparam

모듈을 설계할 때, 비트 폭이나 크기를 하드코딩하지 않고 **파라미터**로 만들어 두면 재사용하기 좋습니다.

```

parameter CLK_PERIOD = 10;      // 클럭 주기 10ns (100MHz)
localparam MEM_ADDRW = 22;      // 메모리 주소 폭 22비트
localparam MEM_DW = 16;         // 메모리 데이터 폭 16비트

```

`parameter` 와 `localparam` 의 차이는: `parameter` 는 모듈을 인스턴스화할 때 외부에서 바꿀 수 있고, `localparam` 은 내부에서만 쓰이는 상수로 외부에서 변경 불가능합니다.

우리 테스트벤치에서 `MEM_ADDRW = 22` 라고 한 것은 SRAM 모델의 주소가 22비트라는 뜻이고, 이는 $2^{22} = \text{약 } 4 \text{ 백만}$ 워드까지 접근 가능하다는 의미입니다. 6차시에서 배운 DRAM 할당(입력 256KB + 파라미터 3.4MB + 출력)을 담기에 충분한 크기죠.

8-8. 네 가지 모델링 스타일

Verilog로 하드웨어를 기술하는 스타일은 크게 네 가지입니다. 강의 슬라이드에서 중요하게 다루는 내용입니다.

(1) 구조적 모델링 (Structural Modeling)

게이트를 직접 연결해서 회로를 만드는 방식입니다. 제일 저수준(low-level)이죠.

```
// 반가산기 (Half Adder) - 구조적 모델링
module half_adder (input x, y, output s, c);
    xor (s, x, y);      // XOR 게이트: s = x ^ y
    and (c, x, y);      // AND 게이트: c = x & y
endmodule
```

이걸 이용해서 전가산기(Full Adder)도 만들 수 있습니다:

```
module full_adder (input x, y, cin, output s, cout);
    wire s1, c1, c2;
    half_adder ha_1 (x, y, s1, c1);    // 반가산기 1 인스턴스화
    half_adder ha_2 (cin, s1, s, c2);  // 반가산기 2 인스턴스화
    or (cout, c1, c2);                // OR 게이트
endmodule
```

강의 슬라이드에서 이것을 "**계층적 설계(Hierarchical Design)**"라고 부릅니다. 작은 모듈을 만들고, 그걸 조합해서 더 큰 모듈을 만드는 방식이죠. 우리 프로젝트도 정확히 이렇게 되어 있습니다: `mul.v` (곱셈기) → `mac.v` (MAC) → `adder_tree.v` (adder 트리) → `cnn_ctrl.v` (컨트롤러) → `yolo_engine.v` (전체 엔진).

(2) 데이터플로우 모델링 (Dataflow Modeling)

`assign` 문을 사용해서 데이터가 어떻게 흐르는지를 기술합니다. 게이트 하나하나를 쓰지 않아도 되어 편합니다.

```
// 반가산기 - 데이터플로우 모델링
module half_adder (input x, y, output s, c);
    assign s = x ^ y;      // XOR 연산
    assign c = x & y;      // AND 연산
endmodule
```

핵심 포인트: `assign` 문은 "항상" 활성 상태입니다. C에서 `s = x ^ y;` 는 실행될 때 한 번만 계산되지만, Verilog 에서 `assign s = x ^ y;` 는 x나 y가 변할 때마다 **자동으로** s가 재계산됩니다. 전선을 연결해 놓은 거라서, 입력이 바뀌면 출력도 바뀌는 것이 자연스럽습니다.

(3) 행위적 모델링 (Behavioral Modeling)

`always` 블록과 `initial` 블록을 사용합니다. 프로그래밍에 가장 가까운 스타일이라서 초보자에게 친숙합니다.

```
// initial 블록: 시뮬레이션 시작 시 딱 한 번만 실행
initial begin
    clk = 1'b1;
    forever #5 clk = ~clk;    // 5ns마다 클럭 토글 → 주기 10ns (100MHz)
end

// always 블록: 조건이 충족될 때마다 반복 실행
always @(posedge clk, negedge rstn) begin
    if (~rstn) begin
        counter <= 8'd0;      // 리셋이면 카운터 0
    end else begin
        counter <= counter + 1; // 아니면 1 증가
    end
end
```

`initial`: 시뮬레이션이 시작될 때(시간 0에서) 딱 한 번 실행됩니다. 주로 테스트벤치에서 클럭 생성이나 초기값 설정에 사용합니다. 합성(synthesis)은 안 됩니다 — 즉 실제 하드웨어로 만들어지지 않고, 시뮬레이션용입니다.

`always @(posedge clk)` : 클럭의 상승 엣지(0→1)마다 실행됩니다. 이것은 합성이 됩니다 — 실제 플립플롭(레지스터)이 됩니다. `posedge` 는 positive edge(상승), `negedge` 는 negative edge(하강)를 뜻합니다.

우리 프로젝트의 `yolo_engine_tb.v` 에서 클럭을 만드는 부분이 정확히 이 패턴입니다:

```
parameter CLK_PERIOD = 10;    // 100MHz
reg clk;

initial begin
    clk = 1'b1;
    forever #(CLK_PERIOD/2) clk = ~clk;    // 5ns마다 토글
end
```

(4) 혼합 모델링 (Mixed-Style Modeling)

실제 프로젝트에서는 위 세 가지를 섞어 씁니다. 이것이 가장 현실적인 스타일이고, 우리 프로젝트도 이렇게 되어 있습니다.

강의 자료에서는 이를 **RTL (Register-Transfer Level)**이라고 부릅니다. RTL이란 "합성 가능한 behavioral + dataflow 구조의 조합"을 의미합니다. 실무에서 가장 많이 쓰이는 설계 수준입니다.

8-9. `assign` vs `<=` vs `=` — 대입 연산의 세 가지

이것도 초보자가 많이 헷갈리는 부분입니다:

① `assign` (연속 대입) — `wire` 에 사용. 항상 활성. `always` 블록 바깥에서 사용.

```
assign y = a & b;    // a나 b가 바뀌면 y도 즉시 바뀜
```

② `<=` (비블로킹 대입) — `always` 블록 안에서 사용. 순차 회로(레지스터) 기술 시 사용.

```
always @(posedge clk) begin
    q <= d;    // 클럭 엣지에서 d의 값을 q에 저장 (동시에 모두 업데이트)
end
```

③ = (블로킹 대입) — always 블록 안에서 사용. 조합 회로 또는 테스트벤치에서 사용.

```
always @(*) begin
    sum = a + b;    // 조합 로직: a, b 바뀌면 즉시 계산
end
```

실무 규칙: 순차 회로(클럭 엣지 트리거)에서는 항상 <= 를 쓰고, 조합 회로(always @(*))에서는 = 를 씁니다. 이걸 섞어 쓰면 예측 불가능한 동작이 발생할 수 있습니다.

8-10. Timescale

시뮬레이션에서 시간 단위를 정의합니다:

```
`timescale 1ns / 1ps
```

이 뜻은: "시간 단위는 1ns이고, 정밀도는 1ps이다." 따라서 #10 이라 쓰면 10ns 지연을 의미합니다.

우리 프로젝트의 yolo_engine_tb.v 맨 위에 timescale 1ns / 1ns 이라고 되어 있고, 클럭 주기가 10(= 10ns = 100MHz)으로 설정되어 있습니다. Nexys A7 보드가 100MHz로 동작하기 때문입니다.

8-11. 이 프로젝트와의 연결: 실제 코드로 보기

지금까지 배운 개념들이 실제 프로젝트 코드에서 어떻게 쓰이는지 보겠습니다.

yolo_engine.v의 일부 (탑 모듈)

```
always @(posedge clk, negedge rstn) begin
    if (~rstn) begin
        dram_base_addr_rd <= 0;
        dram_base_addr_wr <= 0;
    end
    else begin
        if (!ap_start && i_ctrl_reg0[0]) begin
            dram_base_addr_rd <= i_ctrl_reg1; // DRAM Read 시작 주소
            dram_base_addr_wr <= i_ctrl_reg2; // DRAM Write 시작 주소
        end
        else if (ap_done) begin
            dram_base_addr_rd <= 0;
            dram_base_addr_wr <= 0;
        end
    end
end
end
```

이 코드를 해석해 보면:

- `always @(posedge clk, negedge rstn)`: 클럭 상승 엣지 또는 리셋 하강 엣지에 반응
- `if (~rstn)`: 리셋이 활성화(low)이면 주소를 0으로 초기화
- `if (!ap_start && i_ctrl_reg0[0])`: 가속기가 아직 시작 안 했고, 제어 레지스터의 0번 비트가 1이면 → DRAM 베이스 주소를 설정. 이것은 6차시에서 배운 "Host가 제어 레지스터에 설정값을 쓰면 가속기가 읽어간다"는 동작 그 자체입니다.
- `else if (ap_done)`: 네트워크 처리가 완료되면 주소를 0으로 리셋

여기서 `<=` (비블로킹 대입)를 쓰고 있는 것도 보이시죠? 이것은 순차 회로(플립플롭)이기 때문입니다.

테스트벤치에서의 모듈 인스턴스화

yolo_engine_tb.v에서 가속기 본체를 인스턴스화하는 부분:

```
yolo_engine u_yolo_engine (
```

```
.clk(clk),  
.rstn(rstn),  
.M_AWID(M_AWID),  
.M_AWADDR(M_AWADDR),  
// ... 수많은 AXI 신호들 ...  
.i_ctrl_reg0(i_ctrl_reg0),  
.network_done(network_done)  
);
```

7차시에서 배운 "테스트벤치가 가속기를 DUT(Design Under Test)로 감싸서 시뮬레이션한다"는 구조가 바로 이것입니다. 테스트벤치 안에서 `yolo_engine` 모듈을 인스턴스화하고, 클럭과 리셋을 넣어주고, 출력 신호를 관찰하는 겁니다.

8-12. 이번 차시 핵심 정리

이번 8차시에서 배운 핵심을 정리하면:

Verilog의 핵심 특성: 하드웨어 기술 언어. C와 달리 모든 것이 병렬로 동작함.

Module: Verilog의 기본 빌딩 블록. `module` ~ `endmodule` 로 감쌈. 포트(input/output)를 통해 외부와 통신.

숫자 표현: 비트수'진법코드값' 형식. 예: `8'hFF`, `4'b1010`. 비트 폭 명시가 매우 중요.

wire vs reg: `assign`에서 쓰면 wire, `always` / `initial`에서 쓰면 reg.

모델링 스타일 4가지: Structural(게이트 연결), Dataflow(assign), Behavioral(always/initial), Mixed(실무에서 주로 사용 = RTL).

대입 연산: `assign` (연속), `<=` (비블로킹, 순차회로), `=` (블로킹, 조합/테스트벤치).

Timescale: 시뮬레이션 시간 단위. 우리 프로젝트는 1ns 단위, 100MHz 클럭.

9차시: MAC 유닛 설계 (DSP, 곱셈-누산기)

9-1. 이번 차시의 위치: 왜 MAC인가?

8차시에서 Verilog 문법을 배웠으니, 이제 실제 하드웨어 블록을 설계할 차례입니다. 가장 먼저 만들어야 할 것은 **MAC (Multiply-Accumulate, 곱셈-누산기)**입니다.

2차시에서 배운 것을 떠올려 보면, CNN의 핵심 연산은 합성곱(Convolution)이었습니다. 3x3 필터가 슬라이딩하면서 원소별 곱셈 → 누적합을 하는 연산이었죠. 3차시에서 확인했듯이 전체 연산량 ~1.035 GFLOPS 중 99% 이상이 이 합성곱 연산입니다.

따라서 합성곱의 "곱하고 더하는" 핵심 연산을 하드웨어로 구현하는 것, 그것이 MAC 유닛의 역할입니다. MAC은 우리 가속기의 **ALU(산술논리장치)**에 해당합니다.

이번 차시의 전체 구조는 바닥부터 위로 올라가는 **bottom-up 설계**입니다:

Step 1: DSP 블록 이해	→ FPGA에 내장된 곱셈기 하드웨어
Step 2: mul.v	→ DSP를 감싼 8비트 곱셈기 모듈
Step 3: adder_tree.v	→ 곱셈 결과들을 트리 구조로 더하기
Step 4: mac.v	→ 곱셈기 + 애더 트리 = 기본 MAC
Step 5: mul_dual.v	→ DSP 효율을 높이는 듀얼 곱셈기
Step 6: mac_stack.v	→ 36개 곱셈기 + 애더 트리 (실전용)
Step 7: mac_kern.v	→ 누적기(Accumulator) 포함한 완성형

9-2. DSP 블록이란? — FPGA 안의 "전용 곱셈기"

왜 DSP를 알아야 하는가?

곱셈이라는 연산을 하드웨어로 만드는 방법은 두 가지입니다:

(a) LUT(Look-Up Table)로 만들기: FPGA의 범용 로직 자원을 조합해서 곱셈 회로를 구성합니다. 말하자면 AND 게이트, XOR 게이트 등을 조합해서 곱셈기를 "직접 조립"하는 것입니다.

(b) DSP 블록 사용하기: FPGA 칩 안에 미리 만들어져 있는 전용 곱셈기 하드웨어를 사용합니다. Xilinx FPGA에서는 이것을 DSP48E 블록이라고 부릅니다.

우리 보드(Nexys A7-100T)에는 **DSP 슬라이스가 240개** 들어 있습니다. 한편 LUT는 63,400개(= 15,850 슬라이스 × 4개)입니다.

강의 매뉴얼에 아주 중요한 수치가 나옵니다: 만약 240개의 곱셈기를 DSP를 안 쓰고 전부 LUT로 만들면, **14,640개의 LUT를 소모**합니다. 이것은 전체 LUT의 **23.09%**에 해당합니다. 반면 DSP를 쓰면 그 23%의 LUT를 다른 모듈(컨트롤러, 버퍼 제어 로직 등)에 활용할 수 있습니다.

비유하자면: DSP는 건물에 기본으로 설치된 **엘리베이터**입니다. 짐을 나르려면 계단으로도 가능하지만(LUT), 엘리베이터가 있는데 굳이 계단을 쓸 이유가 없죠. 특히 엘리베이터가 240대나 있으니까요.

DSP48E 블록의 내부 구조

DSP48E 블록 하나의 핵심은 **25×18비트 곱셈기**입니다. 이것은 두 개의 정수를 곱할 수 있는 전용 하드웨어이고, 거기에 48비트 덧셈기와 파이프라인 레지스터가 붙어 있습니다. 수식으로 표현하면 $P = A \times B + C$ (곱하고 더하기)를 한 번에 처리할 수 있습니다.

우리는 8비트 × 8비트 곱셈을 하면 되니까(가중치 INT8 × 입력 UINT8), 25×18비트 곱셈기로 충분하고도 남습니다.

DSP 생성 방법: Vivado IP Catalog

Vivado에서 DSP 블록을 사용하려면:

1. **IP Catalog**에서 "DSP48 Macro"를 검색
2. **Instructions** 탭에서 수식 선택 (예: $A*B+C$)
3. **Pipeline Options** 탭에서 파이프라인 레지스터 설정

여기서 **파이프라인 레지스터**라는 중요한 개념이 나옵니다. 기본 설정(Automatic)에서 DSP는 **4단계 파이프라인**으로 동작합니다. 즉 입력을 넣으면 결과가 **4클럭 후**에 나옵니다.

"4클럭이나 기다려야 한다고? 느린 거 아닌가?" — 좋은 질문입니다. 파이프라인의 핵심은 **한 번에 하나만 처리하는** 게

아니라는 것입니다. 1번 데이터를 넣고, 다음 클럭에 2번 데이터, 그 다음에 3번... 이렇게 매 클럭마다 새 데이터를 넣을 수 있습니다. 4클럭의 지연(latency)은 있지만, 한번 파이프라인이 채워지면 **매 클럭마다 하나의 결과**가 나옵니다. 세탁기-건조기와 비슷합니다 — 첫 빨래가 마르기까지는 시간이 걸리지만, 일단 돌아가기 시작하면 계속 빨래가 나옵니다.

매뉴얼에서는 파이프라인 레지스터를 전부 제거하면 0사이클(조합회로)로도 가능하다고 언급하지만, 그러면 클럭 주파수가 내려가기 때문에 보통은 파이프라인을 유지합니다.

9-3. mul.v — 8비트 곱셈기 모듈

DSP 블록을 직접 호출해도 되지만, 매번 DSP의 복잡한 포트를 다루기 불편합니다. 그래서 **wrapper 모듈**인 `mul.v`를 만듭니다.

```
`timescale 1ns / 1ps

module mul(
    input      clk,
    input  [7:0] w,      // Weight (가중치), 8비트
    input  [7:0] x,      // Input pixel (입력), 8비트
    output [15:0] y      // Product (결과), 16비트
);

    wire [17:0] dsp_A, dsp_B;    // DSP 입력용 18비트
    wire [47:0] dsp_P;          // DSP 출력 48비트

    // 부호 확장 (Sign Extension)
    assign dsp_A = w[7] ? {10'b11_1111_1111, w} : {10'b00_0000_0000, w};
    assign dsp_B = x[7] ? {10'b11_1111_1111, x} : {10'b00_0000_0000, x};

    // 출력은 48비트 중 하위 16비트만 사용
    assign y = dsp_P[15:0];

    // DSP 인스턴스화
    xbp_dsp48_macro_0
    u_dsp(.CLK(clk), .A(dsp_A), .B(dsp_B), .C(48'b0), .P(dsp_P));

endmodule
```

부호 확장 (Sign Extension) — 왜 필요한가?

이 부분이 매우 중요합니다. DSP 블록의 입력은 18비트인데, 우리 데이터는 8비트입니다. 남은 10비트를 어떻게 채울까요?

5차시에서 배웠듯이 가중치는 **INT8** (2의 보수법, -127~+127)입니다. 2의 보수법에서 음수를 더 넓은 비트 폭으로 확장할 때는 **최상위 비트(부호 비트)**를 복사해서 채워야 합니다. 이것을 부호 확장(Sign Extension)이라 합니다.

예를 들어 볼까요:

```
-3 을 8비트로 표현:  1111_1101  (MSB가 1 → 음수)
-3 을 18비트로 확장: 11_1111_1111_1111_1101  (앞을 전부 1로 채움)

+5 를 8비트로 표현:  0000_0101  (MSB가 0 → 양수)
+5 를 18비트로 확장: 00_0000_0000_0000_0101  (앞을 전부 0으로 채움)
```

코드에서 `w[7]` 은 `w`의 MSB(부호 비트)를 확인합니다. 1이면(음수이면) 앞에 1을 채우고, 0이면(양수이면) 앞에 0을 채웁니다. 이것을 삼항 연산자(`?:`)로 한 줄에 처리하고 있습니다.

7차시에서 흔한 버그로 "signed/unsigned 혼동"을 언급했었죠? 바로 이 부호 확장을 잘못 하면 발생하는 버그입니다. 음수 가중치 -3을 부호 확장 없이 단순히 앞에 0을 붙이면 253이라는 전혀 다른 양수가 되어 버립니다.

출력 비트 폭은 왜 16비트인가?

8비트 × 8비트 = 최대 16비트가 필요합니다. 가장 큰 경우를 계산하면: $127 \times 255 = 32,385$ 인데, 이것은 16비트(최대 65,535)로 충분히 표현 가능합니다. DSP의 48비트 출력 중 하위 16비트만 취하면 됩니다.

테스트벤치로 확인하기

강의 자료의 `mul_tb.v`에서는 가중치 `w`를 4로 고정하고, 입력 `x`를 0부터 15까지 바꿔가며 결과를 확인합니다:

```
w=4, x=0 → y=0  (4사이클 후 출력)
w=4, x=1 → y=4
```

```
w=4, x=2 → y=8
...
w=4, x=15 → y=60
```

파형(waveform)에서 보면, 입력을 넣고 **4클럭 후에** 결과가 나오는 것이 확인됩니다. 이것이 DSP의 4단 파이프라인 지연입니다.

9-4. Adder Tree — 곱셈 결과들을 더하는 트리

곱셈기 하나만으로는 합성곱 연산을 못 합니다. 합성곱은 "곱해서 **다 더하는**" 연산이니까, 여러 곱셈 결과를 누적해야 합니다.

왜 트리 구조인가?

16개의 곱셈 결과를 더해야 한다고 합시다. 순차적으로 하면:

```
result = a0 + a1 + a2 + ... + a15
→ 15번의 덧셈을 차례로 = 15클럭
```

트리 구조로 하면:

```
Level 1: (a0+a1), (a2+a3), ..., (a14+a15) → 8개의 덧셈, 1클럭
Level 2: 8개 결과를 2개씩 묶어 더하기 → 4개의 덧셈, 1클럭
Level 3: 4개 → 2개 → 2개의 덧셈, 1클럭
Level 4: 2개 → 1개 → 1개의 덧셈, 1클럭
총 4클럭! ( $\log_2(16) = 4$ )
```

순차적으로 하면 15클럭, 트리로 하면 4클럭. 게다가 덧셈기의 수는 $8+4+2+1 = 15$ 개로 같습니다. 즉 같은 하드웨어 양으로 **3.75배 빠릅니다**.

비트 폭이 레벨마다 늘어나는 이유

각 레벨에서 두 수를 더하면 비트 폭이 1비트씩 늘어야 합니다. 오버플로우를 방지하기 위해서입니다:

```
Level 0 (곱셈 출력): 16비트 × 16개
Level 1: 17비트 × 8개    (16+16 → 최대 17비트)
Level 2: 19비트 × 4개    (여기서는 3개씩 더하므로 +2비트)
Level 3: 21비트 × 2개
Level 4: 22비트 × 1개 → 최종 출력
```

이것은 5차원에서 "누적합은 INT32로 해야 한다"고 배운 것과 연결됩니다. 곱셈 결과를 수백~수천 번 누적하면 비트 폭이 점점 커지는데, 중간에 잘리지 않도록 충분한 비트 폭을 유지해야 합니다.

Valid 신호의 지연(Delay Line)

Adder Tree에서 중요한 또 하나의 개념이 **valid 신호 처리**입니다.

데이터가 트리를 통과하는 데 4클럭이 걸리므로, "지금 출력이 유효하다"라고 알려주는 **vld_o** 신호도 4클럭 지연시켜야 합니다. 이것을 **시프트 레지스터(shift register)**로 구현합니다:

```
always @(posedge clk, negedge rstn) begin
    if (!rstn) begin
        vld_i_d1 <= 0;
        vld_i_d2 <= 0;
        vld_i_d3 <= 0;
        vld_i_d4 <= 0;
    end
    else begin
        vld_i_d1 <= vld_i;      // 1클럭 지연
        vld_i_d2 <= vld_i_d1;  // 2클럭 지연
        vld_i_d3 <= vld_i_d2;  // 3클럭 지연
        vld_i_d4 <= vld_i_d3;  // 4클럭 지연
    end
end

assign vld_o = vld_i_d4;      // 4클럭 후의 valid가 출력 valid
```

이게 왜 필요하냐면: **vld_i**가 HIGH인 순간에 입력한 데이터의 결과는 4클럭 후에 **acc_o**에 나옵니다. 그러니까

vld_o도 정확히 그 시점에 HIGH가 되어야, 외부 모듈이 "아, 지금 acc_o에 있는 값이 유효한 결과구나"라고 알 수 있습니다.

이 패턴은 하드웨어 설계에서 매우 자주 등장합니다. 데이터를 파이프라인으로 처리하면, 그 데이터에 동반하는 제어 신호도 같은 만큼 지연시켜야 합니다.

9-5. mac.v — 기본 MAC 모듈 (16-MAC)

이제 곱셈기 16개와 애더 트리를 합쳐서 기본 MAC 모듈을 만듭니다.

구조와 포트

```
module mac (  
    input          clk,  
    input          rstn,      // Active-low 리셋  
    input          vld_i,     // 입력 유효 신호  
    input [127:0]  win,       // 가중치 128비트 = 16개 × 8비트  
    input [127:0]  din,       // 입력 128비트 = 16개 × 8비트  
    output [19:0]  acc_o,     // 누적합 출력 20비트  
    output         vld_o      // 출력 유효 신호  
);
```

왜 128비트인가? 16개의 8비트 값을 하나의 버스로 묶으면 $16 \times 8 = 128$ 비트입니다. 이렇게 한 번에 전달하면 한 클럭에 16개의 곱셈을 동시에 시작할 수 있습니다.

내부 동작

```
win[7:0] × din[7:0]   → 곱셈기 0 → product[0]  
win[15:8] × din[15:8] → 곱셈기 1 → product[1]  
...  
win[127:120] × din[127:120] → 곱셈기 15 → product[15]  
  
product[0~15] → Adder Tree → acc_o (20비트)
```

16개의 곱셈이 동시에 실행됩니다! C 코드에서 `for (j=0; j<16; j++) sum += w[j]*x[j];`로 16번 반복해야 할 것을 하드웨어에서는 한 사이클에 합니다. 이것이 하드웨어 가속의 핵심입니다.

총 레이턴시

DSP 곱셈기 4클럭 + Adder Tree 4클럭 = **총 8클럭의 레이턴시**. 하지만 파이프라인이므로, 한번 채워지면 **매 클럭마다 하나의 결과**가 나옵니다.

CNN 연산과의 대응

2차원에서 배운 합성곱 연산을 떠올려 봅시다. 3×3 필터가 3채널 입력에 적용되면:

한 출력 픽셀 = $\sum (3 \times 3 \times 3 = 27 \text{개의 곱셈-덧셈})$

16-MAC 모듈은 한 사이클에 16개를 처리하니까, 27개를 처리하려면 약 2사이클이 필요합니다(첫째 사이클에 16개, 둘째에 11개, 나머지는 0 패딩). 완벽하게 나누어 떨어지지 않지만, 이것은 시작점일 뿐이고 실전에서는 이 숫자를 최적화하게 됩니다.

9-6. DSP 효율 높이기: Dual Multiplier (mul_dual.v)

문제: DSP가 모자란다

Nexys A7에는 DSP가 240개밖에 없습니다. 단순한 곱셈기 하나에 DSP 하나를 쓰면, 240개의 곱셈기가 한계입니다. 더 많은 곱셈을 병렬로 하고 싶으면 어떻게 할까요?

아이디어: 하나의 DSP로 두 번 곱하기

강의 자료에서 소개하는 **Dual Multiplier**는 하나의 DSP로 **두 개의 곱셈**을 수행합니다:

$w \times x1$ 과 $w \times x2$ 를 동시에 계산

핵심은 **가중치(w)를 재사용(reuse)**하는 것입니다. 합성곱에서 같은 필터 가중치가 여러 입력 픽셀에 반복 적용되니까, w 를 한 번 읽어서 두 개의 다른 입력에 곱하면 DSP 사용량을 절반으로 줄일 수 있습니다.

이것은 CNN 연산의 특성을 잘 활용한 것입니다. 2차원에서 배운 "**가중치 공유**" 원리를 떠올려 보세요. 같은 필터가 슬라이딩하면서 여러 위치에 적용되므로, 가중치는 재사용 가능하고, 바뀌는 것은 입력 위치입니다.

DSP 자원 계산

Dual Multiplier를 쓰면: DSP 1개 = 곱셈 2개. 따라서 DSP 240개 $\times$ 2 = **480개의 곱셈**이 가능해집니다. 싱글로는 240개밖에 안 되던 것이 두 배가 됩니다.

9-7. mac_stack.v — 실전 MAC (36개 곱셈기)

기본 mac.v는 16-MAC이었지만, 실제 프로젝트에서는 **36개의 곱셈기**를 묶은 `mac_stack.v`를 사용합니다.

왜 36개인가?

우리 네트워크의 주요 연산은 **3×3 Conv**입니다. 3×3 필터에 4채널을 동시 처리하면:

$3 \times 3 \times 4 = 36$ 개의 곱셈

즉 3×3 필터의 한 "조각"(4채널분)을 한 사이클에 처리할 수 있는 크기입니다.

mac_stack의 구조

```
module mac_stack (  
    input                clk,
```

```

input          rstn,
input          vld_i,
input [36*8-1:0] wgt,          // 가중치 36개 × 8비트 = 288비트
input [36*8-1:0] ifm_00,       // 입력 픽셀 세트 0
input [36*8-1:0] ifm_01,       // 입력 픽셀 세트 1
input [36*8-1:0] ifm_10,       // 입력 픽셀 세트 2
input [36*8-1:0] ifm_11,       // 입력 픽셀 세트 3
output [21:0]   acc_00,         // 누적합 출력 0
output [21:0]   acc_01,         // 누적합 출력 1
output [21:0]   acc_10,         // 누적합 출력 2
output [21:0]   acc_11,         // 누적합 출력 3
output          vld_o           // 출력 유효
);

```

주목할 점: **가중치(wgt)**는 하나인데, **입력은 네 세트(ifm_00~11)**입니다. 하나의 가중치 세트를 네 개의 서로 다른 입력 위치에 적용해서, 한 번에 **4개의 출력 픽셀**을 계산합니다. 이것이 Dual Multiplier의 가중치 재사용 원리가 확장된 것입니다.

4개의 입력 세트에 대해 각각 36개의 곱셈을 수행하면 총 **144개의 곱셈**이 한 사이클에 일어납니다.

Adder Tree 구조 (36개 입력용)

36개의 곱셈 결과를 더하는 트리는 `add_tree_36in.v`입니다. 16개용과 원리는 같지만, 36은 2의 거듭제곱이 아니어서 트리가 약간 비대칭입니다:

```

Level 1: 36개를 3개씩 묶어 더하기 → 12개의 결과 (12개의 3-input adder)
Level 2: 12개를 3개씩 묶기       → 4개의 결과  (4개의 3-input adder)
Level 3: 4개를 2개씩 묶기        → 2개의 결과  (2개의 2-input adder)
Level 4: 2개를 합치기           → 1개의 최종 결과 (1개의 adder)
총 4단계, 4클럭 레이턴시

```

비트 폭도 레벨마다 적절히 늘어납니다. 초기 16비트(곱셈 결과)에서 시작해서 최종 22비트까지 확장됩니다.

9-8. mac_kern.v — 누적기(Accumulator) 포함

왜 누적이 더 필요한가?

mac_stack은 한 사이클에 36개의 곱-합을 처리합니다. 하지만 합성곱의 전체 연산은 **채널 방향으로 반복**됩니다.

예를 들어 Layer 0을 봅시다: 3×3 필터 $\times$ 3채널. mac_stack이 한 번에 $3 \times 3 \times 4 = 36$ 개를 처리하는데, 3채널이면 36개로 충분합니다($3 \times 3 \times 3 = 27$, 나머지는 0 패딩). 하지만 Layer 8은 3×3 필터 $\times$ 128채널 = **1,152개**의 곱셈이 필요합니다. 36개씩 처리하면 **32번 반복**해야 합니다.

이 "반복해서 누적하는" 것을 담당하는 것이 **Partial Sum (부분합) 누적기**입니다.

Partial Sum의 동작

`r_len = 32` 라고 하면 ($128\text{채널} \div 4\text{채널씩} = 32\text{번 반복}$):

```
Cycle 0 (acc_len == 0):  psum = acc      ← 첫 결과는 그냥 저장
Cycle 1 (acc_len == 1):  psum += acc     ← 두 번째 결과를 더함
Cycle 2 (acc_len == 2):  psum += acc     ← 계속 누적
...
Cycle 31 (acc_len == 31): psum += acc   ← 마지막! → 출력
```

즉, `r_len` 번 반복 누적인 뒤에야 비로소 하나의 완전한 출력 픽셀 값이 됩니다. 이 뒤에 bias 덧셈, de-scaling, ReLU를 거쳐야 최종 UINT8 출력이 됩니다.

mac_kern.v의 포트 인터페이스

```
module mac_kern(
    input          clk,
    input          rstn,
    input          i_start,      // 시작 신호
    input          i_mode,      // 모드: CONV3x3 vs CONV1x1
    input          i_vld,       // 입력 유효
    input [7:0]    i_len,       // 누적 길이 (채널/4)
    input [36*8-1:0] i_wgt,     // 가중치 36개
    input [64*8-1:0] i_ifm,     // 입력 64개 (4x4x4)
    output         o_req,       // bias/scale 요청 신호
```

```

    output [4*32-1:0] o_ofm,          // 출력 4개 × 32비트
    output reg      o_vld            // 출력 유효
);

```

여기서 `i_mode` (CONV3x3 vs CONV1x1)가 등장합니다. 3차시에서 배웠듯이 우리 네트워크에는 3x3 Conv와 1x1 Conv 두 종류가 있습니다. 1x1 Conv은 3x3 중 가운데 하나만 쓰는 것이므로, 같은 하드웨어로 모드만 바꿔서 두 가지 연산을 모두 지원합니다.

왜 입력이 64개인가? (i_ifm이 64x8비트)

mac_stack에 4개의 입력 세트(ifm_00~11)가 있고, 각각 36개가 아니라 **4x4x4 = 64개의 픽셀** 중에서 필요한 부분을 선택합니다. 3x3 Conv 모드에서는 3x3 윈도우 위치를 4개 잡아서 4개의 출력 픽셀을 동시에 계산합니다.

9-9. 전체 계층 정리 — Bottom to Top

지금까지 배운 모듈들을 계층적으로 정리하면:

```

DSP48E (FPGA 내장 하드웨어)
└─ mul.v / mul_dual.v (곱셈기 wrapper)
    └─ mac_stack.v (36곱셈기 × 4세트 + 4 Adder Tree = 144 MAC)
        └─ mac_kern.v (mac_stack + Accumulator + 모드 제어)

```

이것이 6차시에서 배운 가속기 구조도의 **"PE (Processing Elements)"** 부분에 해당합니다. 가속기 전체 구조를 다시 떠올려 봅시다:

```

YOLO Accelerator
├─ Main Controller (FSM)          ← 11차시에서 배울 것
├─ DMA (Read/Write/Ctrl)         ← 12차시
├─ Buffers (Global/Weight/FMap/BiasScale) ← 10차시
└─ PEs (MAC + AdderTree + ReLU)  ← 지금 배우고 있는 것!

```

설계 옵션에 대한 참고

강의 자료에서는 144 MAC의 배분 방법에 두 가지 옵션이 있다고 소개합니다:

- **Option A:** $144 = (3 \times 3 \times 4) \times (2 \times 2 \times 1) \rightarrow$ 한 필터의 36개 곱셈을 완전히 처리, $2 \times 2 = 4$ 개의 출력 픽셀 동시 생성
- **Option B:** $144 = (3 \times 3 \times 1) \times (2 \times 2 \times 4) \rightarrow$ 채널 1개분의 9개 곱셈만 처리, 대신 $4 \times 4 = 16$ 개의 출력 픽셀 동시 생성

각각 장단점이 있고, 성능과 버퍼 크기의 트레이드오프가 달라집니다. 이것은 나중에 시스템 통합(12차시)에서 더 깊이 다룹니다.

9-10. SW 코드와의 대응 관계

5차시에서 배운 `yolo_v2_forward_network_quantized.c`의 `gemm_nn_int8_int32` 함수를 다시 보면:

```
for (i = 0; i < M; ++i) {           // 출력 채널 루프
    for (k = 0; k < K; ++k) {       // 입력 채널 루프
        A_PART = ALPHA * A[i*lda+k];
        for (j = 0; j < N; ++j) {   // 출력 픽셀 루프
            c_tmp[j] += A_PART * B[k*ldb+j];
        }
    }
}
```

이 3중 루프를 하드웨어에 매핑하면:

- **j 루프(출력 픽셀)** $\rightarrow$ `mac_stack`의 4개 입력 세트(`ifm_00~11`)로 **공간적 병렬화**. 4개의 출력 픽셀을 동시에 계산.
- **k 루프(입력 채널)** $\rightarrow$ `mac_kern`의 `i_len` 파라미터로 **시간적 반복(누적)**. 매 사이클마다 채널 4개씩 처리하고 partial sum에 누적.
- **i 루프(출력 채널)** $\rightarrow$ 컨트롤러(11차시)가 가중치를 교체하며 반복.

SW에서 수천수만 사이클이 걸리는 연산을, HW에서는 곱셈의 병렬 처리 + 파이프라인으로 수십수백 배 빠르게 처리합니다. 이것이 FPGA 가속의 핵심 원리입니다.

9-11. 이번 차시 핵심 정리

DSP48E: FPGA에 내장된 전용 곱셈기(25×18비트). Nexys A7에 240개. LUT 곱셈 대비 자원 절약 23%.

mul.v: DSP wrapper. 8비트 입력 → 부호 확장 → DSP 곱셈 → 16비트 출력. 4클럭 파이프라인 레이턴시.

Adder Tree: 곱셈 결과들을 트리 구조로 더하기. 16개면 4단계($\log_2 16=4$), 36개도 4단계. 순차 덧셈 대비 훨씬 빠름.

Valid 신호 지연: 파이프라인 레이턴시만큼 시프트 레지스터로 지연시켜야 데이터와 제어 신호가 동기화됨.

Dual Multiplier: 하나의 DSP로 가중치를 재사용해 두 번 곱셈. DSP 효율 2배.

mac_stack: 36곱셈기 × 4세트 = 144 MAC. 한 사이클에 4개의 출력 픽셀 동시 계산.

mac_kern: mac_stack + 누적기. 채널 방향 반복 누적(i_len), CONV3x3/1x1 모드 지원. 누적 완료 후 bias/scale 요청.

SW↔HW 대응: C의 3중 루프 → HW의 공간적 병렬화(j루프) + 시간적 반복(k루프) + 가중치 교체(i루프).

10차시로 넘어가겠습니다. 강의 자료를 확인하세요. 충분한 자료를 확인했습니다. 시작하겠습니다.

10차시: BRAM 버퍼 설계 (온칩 메모리)

10-1. 이번 차시의 위치: MAC이 일하려면 데이터가 필요하다

9차시에서 MAC 유닛(곱셈-누산기)을 만들었습니다. 하지만 MAC이 아무리 빨라도, 데이터를 제때 공급하지 못하면 놀게 됩니다. MAC에 매 클럭마다 가중치와 입력 데이터를 넣어줘야 하는데, DRAM에서 직접 가져오면 너무 느립니다.

6차시에서 배운 **3단계 데이터 이동**을 떠올려 봅시다:

- ① Host → DRAM (UART, 매우 느림, 한 번만)
- ② DRAM → Global Buffer (DMA, 빠름)
- ③ Global Buffer → L1 Buffer → PE (내부, 가장 빈번하고 빨라야 함)

③번, 즉 MAC에 가장 가까운 "L1 버퍼"가 바로 오늘 배울 **BRAM(Block RAM)**으로 만든 온칩 메모리입니다. BRAM은 FPGA 칩 안에 내장된 전용 메모리 블록으로, DRAM에 비해 **레이턴시가 매우 낮고(1~2클럭)**, **대역폭이 높습니다**.

비유하자면: DRAM이 **창고**라면, BRAM은 MAC 바로 옆에 놓인 **작업대 서랍**입니다. 창고에서 한 번에 많이 가져와서 서랍에 넣어 두고, 작업할 때는 서랍에서 바로 꺼내 쓰는 구조입니다.

10-2. BRAM이란? — FPGA 내장 메모리

기본 사양

우리 보드 Nexys A7-100T의 BRAM 사양:

- **135개의 Block RAM** (각 36Kbit = 36,864비트)
- 총 용량: $135 \times 36\text{Kbit} = \text{약 } 607\text{KB}$ (4,860Kbit)
- **동기식(Synchronous)**: 클럭 엣지에 맞춰 읽기/쓰기
- 단일 포트(Single-port) 또는 듀얼 포트(Dual-port) 구성 가능

607KB라는 것은 우리 네트워크의 전체 가중치(~3.4MB)를 다 담기엔 부족합니다. 그래서 **필요한 레이어의 가중치만 일부 올리고, 나머지는 DRAM에 두었다가 필요할 때 가져오는 전략**이 필요합니다.

BRAM의 설정 가능한 구조 (Port Aspect Ratios)

하나의 36Kbit BRAM 블록을 다양한 형태로 쓸 수 있습니다:

```
512 워드 × 72비트    (넓은 워드, 적은 수)
1024 워드 × 36비트
2048 워드 × 18비트
4096 워드 × 9비트
8192 워드 × 4비트
16384 워드 × 2비트   (좁은 워드, 많은 수)
```

비유하면: 같은 크기의 서랍장이라도, 칸을 크게 나누면(72비트) 적은 칸이 되고, 칸을 작게 나누면(2비트) 많은 칸이 됩니다. 용도에 맞게 선택합니다.

10-3. 두 가지 메모리 타입: Single-port vs Dual-port

Single-port RAM (spram)

포트가 하나입니다. 읽기든 쓰기든 하나의 포트를 통해서만 가능합니다.

```
// spram_wrapper의 포트들
input      clk,          // 클럭
input      addr,         // 주소
input      we,           // Write Enable (1이면 쓰기, 0이면 읽기)
input      cs,           // Chip Select (이 메모리를 선택)
input      wdata,        // 쓰기 데이터
output     rdata         // 읽기 데이터
```

동작 원리는 간단합니다:

- `we=1` 이면: `mem[addr] ← wdata` (쓰기)
- `we=0` 이면: `rdata ← mem[addr]` (읽기)

하지만 한 클럭에 읽기 또는 쓰기 중 하나만 가능합니다. 동시에 못 합니다.

Dual-port RAM (dpram)

포트가 두 개입니다. 가장 많이 쓰는 형태는 **Simple Dual Port RAM**입니다:

Port A: Write Only (쓰기 전용)
- clka, ena, wea, addra, dina

Port B: Read Only (읽기 전용)
- clk_b, enb, addrb, doutb

핵심 장점: **같은 클럭에 읽기와 쓰기를 동시에** 할 수 있습니다!

왜 이게 중요한가? 합성곱을 계산하는 동안에는 가중치 버퍼에서 **읽기**(Port B로 MAC에 데이터 공급)를 계속해야 하는데, 동시에 DMA가 다음 레이어의 가중치를 **쓰기**(Port A로 업데이트)할 수 있습니다. 만약 Single-port였다면, DMA가 쓰는 동안 MAC은 멈춰서 기다려야 합니다.

이것은 6차시에서 배운 "데이터 이동 3단계"의 ②번(DRAM→Global Buffer)과 ③번(Buffer→PE)이 **동시에** 일어날 수 있다는 의미입니다. 이를 통해 성능을 크게 향상시킬 수 있습니다.

10-4. 읽기/쓰기 비트폭이 다른 설정 — 대역폭 최적화

BRAM의 매우 유용한 기능 중 하나는 **Write Width**와 **Read Width**를 다르게 설정할 수 있다는 것입니다.

왜 필요한가?

매뉴얼의 설명을 풀어보면: 가중치 버퍼에 데이터를 쓸 때(DMA에서 전송)는 한 번만 하면 됩니다. 하지만 읽을 때(MAC이 사용할 때)는 **매 출력 픽셀마다 반복해서** 읽어야 합니다. 읽기가 쓰기보다 훨씬 빈번한 거죠.

예를 들어:

```
Write Width = 8비트      (DMA가 한 번에 8비트씩 씀 - 느려도 됨)
Read Width = 128비트     (MAC이 한 번에 128비트 = 16개 픽셀 읽음 - 빨라야 함)
```

이렇게 하면 **메모리 인스턴스 수를 늘리지 않고도 읽기 대역폭을 높일** 수 있습니다. 물리적으로는 같은 BRAM 블록인데, 쓸 때는 좁게, 읽을 때는 넓게 접근하는 겁니다.

우리 프로젝트의 **Weight Buffer**가 정확히 이 기법을 사용합니다:

```
Port A (Write): 72비트 × 4096 워드 → 주소 12비트
Port B (Read):  288비트 × 1024 워드 → 주소 10비트
```

Write는 72비트 단위로 쓰고, Read는 288비트(= 72×4) 단위로 읽습니다. 한 번 읽으면 4개 워드를 동시에 가져오는 셈이니까 읽기 대역폭이 4배입니다. 288비트는 mac_stack의 가중치 입력($36 \times 8 \text{비트} = 288 \text{비트}$)과 정확히 일치합니다. 한 번 읽으면 MAC이 필요로 하는 36개의 가중치를 한 클럭에 공급할 수 있는 거죠.

10-5. Wrapper 패턴 — FPGA 모드 vs 시뮬레이션 모드

강의 자료에서 소개하는 `spram_wrapper.v`와 `dpram_wrapper.v`는 매우 실용적인 설계 패턴입니다.

왜 Wrapper가 필요한가?

FPGA에 올릴 때는 Vivado IP Generator로 만든 **실제 BRAM IP**를 사용해야 합니다. 하지만 시뮬레이션할 때 (Vivado의 Behavioral Simulation)는 BRAM IP 없이도 **reg 배열로 메모리를 모델링**할 수 있습니다.

Wrapper는 이 두 모드를 **하나의 모듈 안에서 전환**할 수 있게 해 줍니다:

```
`define FPGA 1      // 이 줄이 있으면 FPGA 모드 → 실제 BRAM IP 사용

module spram_wrapper (
    clk, addr, we, cs, wdata, rdata
);
    parameter DW = 16;      // Data Width: 워드당 비트수
```

```

parameter AW = 13;      // Address Width: 주소 비트수
parameter DEPTH = 6240; // 총 워드 수

`ifdef FPGA
    // FPGA 모드: 실제 BRAM IP 인스턴스화
    spram_6240x16 u_spram_6240x16 (
        .clka(clk), .ena(cs), .wea(we),
        .addra(addr), .dina(wdata), .douta(rdata)
    );
`else
    // 시뮬레이션 모드: reg 배열로 메모리 모델링
    reg [DW-1:0] mem [0:DEPTH-1];

    always @(posedge clk) begin
        if (cs) begin
            if (we) mem[addr] <= wdata;
            rdata <= mem[addr];
        end
    end
`endif

endmodule

```

이 패턴의 장점:

1. 시뮬레이션할 때는 ``define FPGA`를 주석 처리하면, BRAM IP 없이도 시뮬레이션 가능
2. FPGA에 올릴 때는 ``define FPGA 1`을 활성화하면, 실제 BRAM IP 사용
3. 상위 모듈(`mac_kern`, `cnn_ctrl` 등)은 코드를 바꿀 필요가 없음 — wrapper의 포트가 동일하니까

이것은 7차시에서 배운 "테스트벤치에서 SRAM 모델로 DRAM을 대체한다"는 개념과 일맥상통합니다. 시뮬레이션과 실제 FPGA 구현을 유연하게 전환하는 설계 기법입니다.

Parameter 설정

Wrapper를 사용할 때는 세 가지 파라미터를 설정합니다:

DW (Data Width) - 한 워드의 비트 수
 AW (Address Width) - 주소의 비트 수 ($= \log_2(\text{DEPTH})$)

예를 들어 DW=4, AW=3, DEPTH=8 이면: 4비트짜리 워드가 8개, 주소는 3비트(000~111)인 작은 메모리입니다.

10-6. COE 파일로 메모리 초기화

BRAM을 만들 때, 초기값을 넣어줄 수 있습니다. 이때 사용하는 것이 **COE(Coefficient) 파일**입니다.

COE 파일 형식

```
memory_initialization_radix=16;  
memory_initialization_vector=  
ebfe17d6ef3dfaf513,  
fdfc0be0f230fbf411,  
fbff06e1ef3100f010,  
0000000000000000,  
...
```

첫 줄은 진법(16진수), 둘째 줄부터 각 주소의 초기값입니다.

언제 초기화하는가?

우리 프로젝트에서 작은 레이어(Layer 0, 2, 4)의 가중치는 COE 파일로 미리 초기화합니다. 이 레이어들의 가중치 크기를 보면:

```
Layer 0: 3×3×3×16 = 432 바이트 → 작음  
Layer 2: 3×3×16×32 = 4,608 바이트 → 중간  
Layer 4: 3×3×32×64 = 18,432 바이트 → 제법 큼, 하지만 BRAM에 들어감
```

이 세 레이어는 가중치 전체가 BRAM에 들어가므로 "ROM처럼" 사용 가능합니다. Bitstream에 초기값이 포함되어 FPGA 프로그래밍 시 자동으로 로드됩니다.

반면 Layer 6 이후의 가중치($3 \times 3 \times 64 \times 128 = 73,728$ 바이트 등)는 너무 커서 BRAM에 다 안 들어갑니다. 이런 레이어의 가중치는 **DRAM에 저장해 두었다가 DMA로 필요할 때 BRAM에 일부씩 로드합니다.**

10-7. 가속기의 버퍼 구성 — 전체 그림

이제 우리 가속기에 필요한 버퍼들을 전체적으로 살펴봅시다. 강의 자료의 시스템 구조도에 따르면:

DNN Accelerator 내부 버퍼:

- └ L2 Global Buffer (256KB) - DRAM과 L1 사이의 중간 버퍼
- └ L1 Weight Buffer (36KB) - 가중치 저장
- └ L1 Feature Map Buffer (2KB × 2개) - 입력/출력 피쳐맵 (더블 버퍼링)
- └ L1 Bias/Scale Buffer (10KB) - Bias와 Scale 값 저장

각각의 역할과 설계를 하나씩 봅시다.

(1) Global Buffer (256KB)

DRAM에서 가져온 데이터를 임시 저장하는 큰 버퍼입니다. $256 \times 256 \times 4 = 256\text{KB}$ 로, Layer 0의 입력 이미지나 중간 레이어의 출력(예: $128 \times 128 \times 16$)을 통째로 담을 수 있습니다.

DMA가 DRAM에서 데이터를 읽어와 Global Buffer에 쓰고, 컨트롤러가 Global Buffer에서 L1 버퍼로 데이터를 옮깁니다. Global Buffer는 DRAM 접근의 레이턴시를 숨기는 **캐시** 역할을 합니다.

대역폭:

DRAM ↔ Global Buffer: 128비트 (DMA를 통해)
Global Buffer ↔ L1 Buffers: 512비트 (내부 버스)

(2) Weight Buffer (4096 × 72비트 = 36KB)

용도: MAC에 공급할 가중치(필터 계수)를 저장

구성: Simple Dual Port RAM (dpram_4096x72)

Port A (Write): 72비트 × 4096 워드, 주소 12비트

- DMA에서 가중치를 쓸 때 사용

Port B (Read): 288비트 × 1024 워드, 주소 10비트

- MAC이 가중치를 읽을 때 사용
- 한 번 읽으면 288비트 = 36개 × 8비트 가중치

워드 크기 72비트의 의미: 3×3 Conv에서 한 채널의 필터는 3×3×1 = 9개의 8비트 값 = 72비트입니다. 즉 하나의 워드가 정확히 **한 채널분의 3×3 필터**에 해당합니다.

한 워드 (72비트):

w0	w1	w2	← 3×3 필터의 첫 행
w3	w4	w5	← 둘째 행
w6	w7	w8	← 셋째 행

각 w는 8비트, 총 9×8 = 72비트

1×1 Conv의 경우는? 1×1 Conv는 필터가 1×1×1 = 1개의 값만 필요합니다. 하지만 72비트 워드를 쓰므로 1×1×8 = 8개를 한 워드에 묶습니다. 즉 **8개의 1×1 필터를 하나의 워드에 패킹**합니다.

메모리 �핑:

Layer 0 가중치: 주소 0 ~ 63 (3×16 = 48 워드 + 패딩 = 64)

Layer 2 가중치: 주소 512 ~ 1023 (16×32 = 512 워드)

Layer 4 가중치: 주소 2048 ~ 4095 (32×64 = 2048 워드)

Layer 0, 2, 4는 COE로 초기화(ROM처럼 사용). Layer 6 이후는 DMA가 Port A를 통해 런타임에 업데이트합니다.

BRAM 자원: 이 하나의 Weight Buffer가 **36Kb BRAM 8개**를 소모합니다. 전체 135개 중 8개이니 약 6%입니다.

(3) Feature Map Buffer (2KB × 2개)

용도: 현재 처리 중인 입력 피쳐맵(IFM)의 일부를 저장

크기: 각 2KB 정도, 2개를 **더블 버퍼링**으로 운용

더블 버퍼링(Double Buffering)이 뭔가?

이것은 매우 중요한 하드웨어 설계 기법입니다. 버퍼를 두 개 준비해서:

시점 1: Buffer A에서 읽기(MAC 연산) + Buffer B에 쓰기(다음 데이터 로드)
시점 2: Buffer B에서 읽기(MAC 연산) + Buffer A에 쓰기(그 다음 데이터 로드)
시점 3: 다시 A에서 읽기 + B에 쓰기 ...

이렇게 교대로 사용하면, **데이터 로드와 연산이 동시에** 일어날 수 있습니다. 만약 버퍼가 하나뿐이면, 데이터를 로드하는 동안 MAC이 멈춰야 합니다. 더블 버퍼링을 쓰면 MAC이 쉬지 않고 계속 일할 수 있습니다.

비유하면: 셰프가 도마 두 개를 쓰는 것과 같습니다. 한 도마 위의 재료로 요리하는 동안, 보조가 다른 도마에 다음 재료를 준비해 둡니다.

(4) Bias/Scale Buffer (2560 × 32비트 = 10KB)

용도: 각 출력 채널의 bias 값과 de-scaling을 위한 scale 값을 저장

한 워드 (32비트) :

Bias (16비트)	Scale (16비트)
-------------	--------------

5차시에서 배웠듯이 bias는 INT16, scale은 UINT16이고, 이 둘을 하나의 32비트 워드로 합쳐서 저장합니다. MAC이 누적 결과를 완성하면, 해당 채널의 bias를 더하고 scale로 나누어 UINT8 출력을 생성합니다.

2560 워드인 이유: 모든 레이어의 bias/scale을 한꺼번에 저장하기에 충분한 크기입니다. 전체 레이어의 출력 채널 수를 합하면 $16+32+64+128+256+512+256+512+195+128+195 = 2,294$ 개이므로, 2560 워드면 넉넉합니다.

이 버퍼는 COE 파일로 미리 초기화해 두는 것이 일반적입니다. bias와 scale 값은 양자화 과정에서 결정된 상수이므로, 이미지가 바뀌어도 변하지 않습니다.

구성: Single Port RAM (spram_2560x32)으로 충분합니다. DMA로 쓰는 것은 초기 한 번뿐이고, 이후에는 읽기만 하니깐요. 강의 자료의 gbuff_param.v에서는 Bias/Scale 버퍼의 wea를 1'b0으로 고정(항상 읽기 모드)하고 있습니다.

10-8. gbuff_param.v — 파라미터 버퍼 모듈

강의 자료에서는 Weight Buffer와 Bias/Scale Buffer를 하나의 모듈 gbuff_param.v로 묶어서 설계합니다.

```
module gbuff_param(
    input                clk,
    // DMA 인터페이스 (Weight 쓰기)
    input                dma_wea,           // Write Enable
    input  [11:0]        dma_addra,        // Write 주소 (12비트 → 4096 워드)
    input  [71:0]        dma_dina,        // Write 데이터 (72비트)

    // MAC 인터페이스 (Weight 읽기)
    input  [9:0]         wgt_addrb,        // Read 주소 (10비트 → 1024 워드)
    output [287:0]       wgt_doutb,        // Read 데이터 (288비트 = 4×72)

    // MAC 인터페이스 (Bias/Scale 읽기)
    input  [11:0]        bsc_addra,        // Read 주소
    output [31:0]        bsc_douta        // Read 데이터 (32비트 = bias+scale)
);
```

내부에서 두 개의 메모리를 인스턴스화합니다:

```
// Weight Buffer: Dual-port RAM
dpram_4096x72 u_buffer_weight(
```

```

.clka(clk),      .wea(dma_wea),
.addra(dma_addra), .dina(dma_dina),          // Port A: DMA가 쓰기
.clkb(clk),      .addrb(wgt_addrb),
.doutb(wgt_doutb)                             // Port B: MAC이 읽기
);

// Bias/Scale Buffer: Single-port RAM (읽기 전용으로 사용)
spram_2560x32 u_buffer_bias_scale(
    .clka(clk),      .wea(1'b0),              // Write 안 함
    .addra(bsc_addra), .dina(32'd0),
    .douta(bsc_douta)                             // 읽기만
);

```

이 모듈의 구조를 보면, 9차시에서 배운 `mac_kern`이 연산 결과를 완성한 뒤 `o_req` 신호로 bias/scale을 요청하면, 컨트롤러가 `bsc_addra`에 해당 채널 주소를 넣어서 `bsc_douta`로 bias와 scale 값을 받아옵니다.

10-9. BRAM의 읽기 레이턴시 주의

BRAM은 동기식 메모리이므로, 주소를 넣으면 **다음 클럭에** 데이터가 나옵니다 (1사이클 레이턴시). 이것은 `assign` 처럼 즉시 나오는 조합 회로가 아닙니다.

```

클럭 1: addr = 0x005 입력
클럭 2: rdata = mem[0x005] 출력 ← 1클럭 지연!

```

Vivado IP Generator에서 "Primitives Output Register"를 체크하면 **2사이클**이 됩니다. 우리 프로젝트에서는 이것을 **체크 해제**(unchecked)해서 1사이클 레이턴시로 사용합니다.

이 1클럭 지연은 컨트롤러가 데이터를 요청하는 타이밍을 설계할 때 반드시 고려해야 합니다. MAC이 "지금" 데이터를 원하면, 컨트롤러는 "1클럭 전에" 주소를 보내야 합니다.

10-10. BRAM 자원 계산 — 설계 시 고려사항

BRAM은 유한한 자원이므로, 각 버퍼가 얼마나 쓰는지 계산해야 합니다.

Weight Buffer: $4096 \times 72\text{비트} = 294,912\text{비트}$

→ 36Kb BRAM 8개 필요 ($294,912 \div 36,864 \approx 8$)

Bias/Scale Buffer: $2560 \times 32\text{비트} = 81,920\text{비트}$

→ 36Kb BRAM 약 3개 필요

Feature Map Buffer: $2\text{KB} \times 2 = 32,768\text{비트} \times 2 = 65,536\text{비트}$

→ 36Kb BRAM 약 2개 필요

Global Buffer: $256\text{KB} = 2,097,152\text{비트}$

→ 36Kb BRAM 약 57개 필요

합계: $8 + 3 + 2 + 57 =$ 약 70개 BRAM

전체 135개 중 약 52% 사용

나머지 50%는 시스템의 다른 부분(MicroBlaze 캐시, AXI 인터커넥트 버퍼 등)에 할당됩니다. BRAM 자원이 부족하면 Global Buffer 크기를 줄이거나, 가중치를 더 자주 DRAM에서 가져와야 해서 성능이 떨어질 수 있습니다. 이것이 FPGA 설계에서의 핵심 트레이드오프입니다.

10-11. 전체 가속기에서 버퍼의 위치 — 데이터 흐름 정리

지금까지 배운 모든 것을 종합하면, 하나의 합성곱 레이어를 처리하는 데이터 흐름은 이렇습니다:

[1단계: 데이터 로드]

DRAM → DMA → Global Buffer → Weight Buffer (Port A에 쓰기)

→ Feature Map Buffer (피쳐맵 로드)

[2단계: 연산]

Weight Buffer (Port B에서 288비트 읽기) → mac_stack의 wgt 입력

Feature Map Buffer (읽기) → mac_stack의 ifm 입력

→ mac_stack에서 곱셈 + Adder Tree

→ mac_kern에서 채널 방향 누적 (partial sum)

[3단계: 후처리]

Bias/Scale Buffer (읽기) → bias 덧셈 + scale 나눗셈

→ ReLU 적용 (MSB 체크)

→ UINT8 출력

[4단계: 결과 저장]

UINT8 출력 → Feature Map Buffer (다른 쪽에 쓰기, 더블 버퍼링)

→ 또는 Global Buffer → DMA → DRAM

이 과정이 22개 레이어에 대해 반복됩니다. 4차시에서 배운 SW 코드의 `forward_convolutional_layer_q()` 함수가 하는 일을, 하드웨어가 BRAM 버퍼와 MAC을 이용해 수행하는 것입니다.

10-12. 이번 차시 핵심 정리

BRAM (Block RAM): FPGA 내장 전용 메모리. Nexys A7에 135개(607KB). 동기식, 12클럭 레이턴시.

Single-port vs Dual-port: Single-port는 읽기/쓰기 중 하나만, Dual-port는 동시 가능. 성능상 Dual-port가 유리.

읽기/쓰기 비트폭 분리: Write Width ≠ Read Width 가능. 읽기 대역폭을 높여 MAC에 빠르게 데이터 공급.

Wrapper 패턴: FPGA 모드(실제 BRAM IP)와 시뮬레이션 모드(reg 배열)를 `ifdef` 로 전환. 상위 모듈 수정 불필요.

COE 파일: 메모리 초기화용. 작은 레이어의 가중치를 미리 로드. hex 형식.

4가지 버퍼: Global Buffer(256KB, L2), Weight Buffer(36KB, dpram), Feature Map Buffer(2KB×2, 더블 버퍼링), Bias/Scale Buffer(10KB, spram).

Weight Buffer: 72비트 워드 = 3×3×1 필터 한 채널분. Read 288비트 = mac_stack 입력과 일치. 8개 36Kb BRAM 소모.

더블 버퍼링: 버퍼 2개를 교대 사용. 로드와 연산을 동시 수행. MAC이 쉬지 않게 함.

BRAM 자원 관리: 총 135개 BRAM 중 약 70개 사용(~52%). 나머지는 시스템용. 자원 부족 시 성능 트레이드오프.

11차시: CNN 컨트롤러 — 슬라이딩 윈도우와 FSM

지난 차시까지 우리는 MAC 유닛(9차시)과 BRAM 버퍼(10차시)라는 "손과 팔"을 만들었습니다. 이번 차시에서는 그 손과 팔에 "어디를, 언제, 어떤 순서로 움직여라"라고 명령하는 두뇌 — **CNN 컨트롤러**를 설계합니다. 강의자료 `07_System_Integration_Controller.pdf`와 `08_System_integration_Top_Module.pdf`를 기반으로 합니다.

11-1. 컨트롤러가 필요한 이유

9차시에서 만든 `mac_kern`은 입력을 받으면 곱셈-누산을 잘 수행합니다. 하지만 "누가" 데이터를 넣어주는지는 아직 정하지 않았습니다. 소프트웨어의 6중 for 루프를 떠올려 보세요:

```
for (k=0; k<K; k++)      // 출력 채널 (필터)
  for (p=0; p<P; p++)    // 행
    for (q=0; q<Q; q++)  // 열
      for (r=0; r<R; r++) // 필터 행
        for (s=0; s<S; s++) // 필터 열
          for (c=0; c<C; c++) // 입력 채널
```

소프트웨어에서는 CPU가 이 루프를 순서대로 돌면서 알아서 데이터를 가져옵니다. 하지만 하드웨어에는 "for 루프"라는 개념이 없습니다. 대신 **카운터(counter)**와 **유한 상태 기계(FSM, Finite State Machine)**를 조합해서 루프를 구현합니다.

컨트롤러의 핵심 역할을 한 마디로 하면: "지금 몇 번째 행, 몇 번째 열, 몇 번째 채널을 처리하고 있는지를 추적하면서, **MAC에 데이터를 넣어줄 타이밍과 주소를 생성하는 것**"입니다.

11-2. 슬라이딩 윈도우: SW의 루프를 HW로 바꾸기

합성곱 연산의 본질을 다시 한번 정리합시다. Layer 0을 예로 들면:

입력 256×256×3 위에 3×3×3 필터를 한 픽셀씩 슬라이딩시킵니다. 출력 위치 (p, q)에서의 합성곱은:

```
result = 0
for r in [0,1,2]:      // 필터 세로
  for s in [0,1,2]:    // 필터 가로
    for c in [0,1,2]:  // 채널
      h = p + r - 1    // pad=1이므로 -1
      w = q + s - 1
      result += input[c][h][w] × filter[k][c][r][s]
```

하드웨어에서는 이것을 "현재 출력 위치 (row, col)를 카운터로 추적하고, 해당 위치에서 3×3 윈도우를 메모리에서 읽어와서 MAC에 넣는다"로 구현합니다.

여기서 중요한 것이 패딩 처리입니다. row=0일 때(첫 번째 행), 3×3 윈도우의 윗줄은 입력 영역 밖입니다. 이 위치의 값은 0으로 처리해야 합니다. 마찬가지로 col=0(첫 번째 열)이면 왼쪽이 영역 밖이고, row=height-1(마지막 행)이면 아랫줄, col=width-1(마지막 열)이면 오른쪽이 영역 밖입니다.

테스트벤치 코드(conv_kern_tb.v)에서 이 패딩을 처리하는 방식을 봅시다:

```
wire is_first_row = (row == 0) ? 1'b1 : 1'b0;
wire is_last_row  = (row == IFM_HEIGHT-1) ? 1'b1 : 1'b0;
wire is_first_col = (col == 0) ? 1'b1 : 1'b0;
wire is_last_col  = (col == IFM_WIDTH-1) ? 1'b1 : 1'b0;
```

그리고 3×3 윈도우의 9개 위치를 채울 때, 경계 밖이면 8'd0(0)을, 안이면 in_img[주소]를 넣습니다:

```
din[ 7: 0] = (is_first_row || is_first_col) ? 8'd0 : in_img[(row-1)*IFM_WIDTH
+ (col-1)];
din[15: 8] = (is_first_row          ) ? 8'd0 : in_img[(row-1)*IFM_WIDTH
+ col  ];
```

```

din[23:16] = (is_first_row || is_last_col ) ? 8'd0 : in_img[(row-1)*IFM_WIDTH
+ (col+1)];
din[31:24] = (
is_first_col ) ? 8'd0 : in_img[ row *IFM_WIDTH
+ (col-1)];
din[39:32] =
in_img[ row *IFM_WIDTH
+ col ];
din[47:40] = (
is_last_col ) ? 8'd0 : in_img[ row *IFM_WIDTH
+ (col+1)];
din[55:48] = (is_last_row || is_first_col ) ? 8'd0 : in_img[(row+1)*IFM_WIDTH
+ (col-1)];
din[63:56] = (is_last_row
) ? 8'd0 : in_img[(row+1)*IFM_WIDTH +
col ];
din[71:64] = (is_last_row || is_last_col ) ? 8'd0 : in_img[(row+1)*IFM_WIDTH
+ (col+1)];

```

이 코드를 눈여겨보세요. 3×3 윈도우의 9개 픽셀을 **72비트(9×8비트)**에 패킹하고 있습니다. 이것이 바로 Weight Buffer의 워드 크기가 72비트인 이유와 정확히 맞아떨어집니다. 같은 사이클에 9개 가중치와 9개 입력을 읽어서 한꺼번에 MAC에 넣는 것이죠.

11-3. FSM(유한 상태 기계)의 4가지 상태

cnn_ctrl.v는 **4개의 상태**를 가진 FSM으로 구성됩니다:

```
ST_IDLE → ST_VSYNC → ST_HSYNC → ST_DATA → ST_HSYNC → ... → ST_IDLE
```

각 상태의 역할을 하나씩 보겠습니다.

ST_IDLE (대기 상태)

FSM은 리셋 후 ST_IDLE에서 시작합니다. 아무것도 하지 않고 기다립니다. **q_start 신호가 HIGH**가 되면 ST_VSYNC로 전이합니다.

이 상태에서는 외부(MicroBlaze 또는 상위 컨트롤러)가 레이어 파라미터(**q_width**, **q_height**, **q_frame_size**

등)를 설정하고, 필요한 데이터를 버퍼에 로드하는 준비 시간입니다.

ST_VSYNC (프레임/레이어 동기화)

"시작 전 준비 구간"입니다. `ctrl_vsync_cnt` 라는 카운터가 매 클럭마다 1씩 증가하고, `q_vsync_delay` 에 도달하면 ST_HSYNC로 전이합니다.

이 딜레이 동안 무엇을 할 수 있을까요? 필터 가중치를 DRAM에서 Weight Buffer로 사전 로딩하거나, 입력의 첫 몇 줄을 Feature Map Buffer에 미리 채워놓을 수 있습니다. 이것은 **파이프라인 워밍업** 구간이라고 생각하면 됩니다.

왜 "VSYNC"라는 이름인가요? 디스플레이 컨트롤러에서 빌려온 용어입니다. TV나 모니터에서 VSYNC(수직 동기)는 "새 프레임 시작 전 대기"이고, HSYNC(수평 동기)는 "새 줄 시작 전 대기"입니다. CNN 가속기에서도 비슷한 개념으로 사용합니다.

ST_HSYNC (행/줄 동기화)

한 행의 연산을 시작하기 전 잠깐 대기하는 구간입니다. `ctrl_hsync_cnt` 가 `q_hsync_delay` 에 도달하면 ST_DATA로 전이합니다.

모든 행의 시작 전에 이 상태를 거칩니다. 왜 행마다 대기가 필요할까요? 새 행의 입력 데이터를 버퍼에 로드하거나, 이전 행의 출력을 메모리에 기록하는 시간이 필요할 수 있기 때문입니다. 또한 MAC의 파이프라인이 새 행의 첫 번째 데이터를 받을 준비를 하는 시간이기도 합니다.

ST_DATA (연산 상태)

실제 합성곱 계산이 수행되는 핵심 상태입니다. 이 상태에 있는 동안 `ctrl_data_run` 신호가 HIGH가 되고, 매 클럭마다 (row, col) 위치의 3x3 윈도우 데이터를 MAC에 공급합니다.

한 행이 끝나면(`col == q_width - 1`) ST_HSYNC로 돌아가서 다음 행을 준비합니다. 전체 프레임이 끝나면(`end_frame`) ST_IDLE로 돌아갑니다.

Verilog 코드로 보면 이렇습니다:

```
always @(*) begin
    case(cstate)
```

```

    ST_IDLE:  nstate = (q_start) ? ST_VSYNC : ST_IDLE;
    ST_VSYNC: nstate = (ctrl_vsync_cnt == q_vsync_delay) ? ST_HSYNC :
ST_VSYNC;
    ST_HSYNC: nstate = (ctrl_hsync_cnt == q_hsync_delay) ? ST_DATA :
ST_HSYNC;
    ST_DATA:  begin
        if (end_frame)          nstate = ST_IDLE;
        else if (col == q_width-1) nstate = ST_HSYNC;
        else                    nstate = ST_DATA;
    end
    default: nstate = ST_IDLE;
endcase
end

```

이 FSM은 `always @(*)` 로 기술된 **조합 로직**으로, 다음 상태를 결정합니다. 현재 상태의 갱신은 별도의 `always @(posedge clk)` 블록에서 합니다:

```

always @(posedge clk, negedge rstn) begin
    if (~rstn)  cstate <= ST_IDLE;
    else       cstate <= nstate;
end

```

8차시에서 배운 Verilog 기초를 떠올려 보세요. 조합 로직에서 `=` (블로킹 대입), 순차 로직에서 `<=` (비블로킹 대입)를 쓰는 것이 정석입니다.

11-4. Row, Col, Data_count 카운터

FSM이 "지금 어느 상태인지"를 관리한다면, 카운터들은 "지금 어느 위치를 처리하고 있는지"를 추적합니다.

Column 카운터 (col): ST_DATA 상태에서 매 클럭마다 1씩 증가. $0 \rightarrow 1 \rightarrow \dots \rightarrow q_width-1 \rightarrow 0$ (다시 리셋). 가장 빠르게 도는 내부 루프입니다.

Row 카운터 (row): col이 $q_width-1$ 에 도달할 때(한 행 끝)마다 1씩 증가. $0 \rightarrow 1 \rightarrow \dots \rightarrow q_height-1 \rightarrow 0$. 바깥 루프입니다.

Pixel 카운터 (data_count): ST_DATA에서 매 클럭마다 1씩 증가. $0 \rightarrow 1 \rightarrow \dots \rightarrow q_frame_size-1$. 한 프레임(레이어)에서 총 처리한 픽셀 수를 추적합니다.

end_frame 플래그: data_count가 q_frame_size-1 에 도달하면 HIGH. 이것은 "이 레이어의 모든 출력 픽셀을 다 계산했다"는 신호입니다. FSM은 이 신호를 보고 ST_IDLE로 돌아갑니다.

이 구조가 SW의 for 루프와 정확히 대응됩니다:

```
// SW                      // HW
for (row=0; row<HEIGHT; row++) // row 카운터
    for (col=0; col<WIDTH; col++) // col 카운터
        compute(row, col);      // ST_DATA 상태
```

11-5. cnn_ctrl.v의 입출력 정리

모듈의 인터페이스를 정리해 봅시다:

입력 (외부에서 설정):

- `clk`, `rstn`: 클럭과 리셋
- `q_width`, `q_height`: 출력 feature map의 가로/세로 크기
- `q_frame_size`: 한 레이어의 총 출력 픽셀 수 (= width × height)
- `q_vsync_delay`: 프레임 시작 전 대기 클럭 수
- `q_hsync_delay`: 행 시작 전 대기 클럭 수
- `q_start`: 레이어 처리 시작 트리거

출력 (다른 모듈에 전달):

- `o_ctrl_vsync_run`, `o_ctrl_hsync_run`, `o_ctrl_data_run`: 현재 어느 상태인지를 알려주는 플래그
- `o_ctrl_vsync_cnt`, `o_ctrl_hsync_cnt`: 동기화 카운터 현재 값
- `o_row`, `o_col`: 현재 처리 중인 행/열 좌표
- `o_data_count`: 현재까지 처리한 픽셀 수

- `o_end_frame`: 레이어 완료 플래그

핵심은 이 모듈이 "언제 연산하라" (`ctrl_data_run`)와 "어디를 연산하라" (`row, col`)를 동시에 알려준다는 것입니다. 다른 모듈들은 이 신호를 받아서 알맞은 데이터를 MAC에 공급합니다.

11-6. 컨트롤러가 MAC와 버퍼를 어떻게 연결하는가

이제 전체 데이터 흐름을 따라가 봅시다. `ctrl_data_run`이 HIGH가 되면:

- ① **주소 생성**: 컨트롤러의 (`row, col`) 출력을 이용해서 Feature Map Buffer에서 3x3 윈도우 데이터의 주소를 계산합니다. 동시에 Weight Buffer에서 해당 필터 가중치의 주소도 계산합니다.
- ② **데이터 읽기**: Feature Map Buffer에서 72비트(9x8비트) 입력 윈도우를 읽고, Weight Buffer에서 288비트(9x8비트 x 4필터) 가중치를 읽습니다. BRAM은 동기식이라 주소를 넣은 다음 사이클에 데이터가 나옵니다.
- ③ **MAC 연산**: 읽은 데이터를 `mac_kern`에 공급합니다. `vld_i=1`로 설정하고, `din`에 입력 윈도우, `win`에 필터 가중치를 넣습니다. `mac_stack`은 한 사이클에 36개 곱셈(9곱셈 x 4필터)을 수행합니다.
- ④ **부분합 누적**: 3x3x3 필터라면 한 출력 채널당 3번의 MAC 연산이 필요합니다(채널 3개). `mac_kern`의 `i_len`을 3으로 설정하면, 내부 누적기가 3사이클 동안의 MAC 결과를 자동으로 합산합니다.
- ⑤ **후처리**: 누적이 완료되면(`vld_o` 출력) bias 덧셈, ReLU, de-scaling을 거쳐 최종 UINT8 출력을 생성합니다.
- ⑥ **출력 저장**: 결과를 Feature Map Buffer(또는 Global Buffer)에 기록합니다.

이 과정이 한 출력 픽셀에 대한 것이고, `col` 카운터가 증가하면서 다음 픽셀로 이동합니다.

11-7. 채널 방향 반복: `conv_layer_tb`에서 보는 확장

`conv_kern_tb.v`에서는 한 채널만 처리했습니다 (Layer 0의 입력 채널 3개 중 하나). `conv_layer_tb.v`에서는 이것을 채널 방향으로 확장합니다.

차이는 루프가 하나 더 추가된다는 것입니다:

```
// conv_kern_tb: 2중 루프 (row, col만)
for (row = 0; row < HEIGHT; row++)
    for (col = 0; col < WIDTH; col++)
        // MAC 계산

// conv_layer_tb: 3중 루프 (row, col, chn)
for (row = 0; row < HEIGHT; row++)
    for (col = 0; col < WIDTH; col++)
        for (chn = 0; chn < IN_CHANNELS; chn++)
            // MAC 계산 + 부분합 누적
```

채널 루프가 돌 때마다 같은 (row, col) 위치에서 다른 채널의 입력 데이터와 해당 채널의 필터를 MAC에 넣습니다. `mac_kern`의 누적기가 이 결과들을 순차적으로 더합니다.

예를 들어 Layer 0 (입력 채널 3):

- chn=0: R 채널의 3×3 윈도우 × R 채널 필터 → 부분합에 저장
- chn=1: G 채널의 3×3 윈도우 × G 채널 필터 → 부분합에 누적
- chn=2: B 채널의 3×3 윈도우 × B 채널 필터 → 부분합에 누적 → 최종 결과 출력

이것이 `mac_kern`의 `i_len` 파라미터입니다. Layer 0은 `i_len=3` (채널 3개를 3번에 걸쳐 누적). Layer 2는 `i_len=16` (채널 16개를 16번 누적). 깊은 레이어일수록 입력 채널이 많아서 `i_len`이 커지고, 한 출력 픽셀을 만드는 데 더 많은 사이클이 걸립니다.

11-8. VSYNC/HSYNC 딜레이의 실전적 의미

"왜 일부러 대기 시간을 넣는 거지?"라고 궁금할 수 있습니다. 이 딜레이들은 단순한 낭비가 아니라 **데이터 준비 시간**입니다.

VSYNC 딜레이 (프레임 시작 전): 새 레이어를 시작하기 전에 DRAM에서 가중치를 Weight Buffer로 로딩하거나, 입력 feature map의 첫 부분을 Global Buffer에서 Feature Map Buffer로 전송하는 시간입니다. 특히 Layer 6 이후는 가중치가 커서 미리 BRAM에 담아둘 수 없으니, 이 시간에 DMA로 전송합니다.

HSYNC 딜레이 (행 시작 전): 새 행의 입력 데이터를 준비하거나, 이전 행의 출력을 메모리에 기록하는 시간입니다. 더

블 버퍼링을 사용할 경우, 한 버퍼에서 현재 행을 연산하는 동안 다른 버퍼에 다음 행을 로드하면 HSYNC 딜레이를 최소화할 수 있습니다.

이 딜레이 값들은 **설계자가 조절 가능합니다**. 메모리 로딩이 빠르면 딜레이를 줄이고, 느리면 늘리면 됩니다. 성능 최적화의 핵심 포인트 중 하나입니다. 딜레이가 길면 안정적이지만 fps가 떨어지고, 짧으면 데이터가 제때 준비되지 않을 위험이 있습니다.

11-9. 멀티 레이어 확장: 한 레이어가 끝나면?

지금까지는 한 레이어만 처리하는 것을 봤습니다. 그런데 이 네트워크에는 합성곱 레이어가 11개나 있습니다. 어떻게 여러 레이어를 순차적으로 처리할까요?

`cnn_ctrl_multi_layer_tb.v` 에서 그 방법을 보여줍니다. 핵심 아이디어는 간단합니다:

1. **Layer 0의 파라미터를 설정**한다: `q_width=256, q_height=256, q_frame_size=65536`
2. **q_start를 HIGH**로 올린다 → FSM이 Layer 0을 처리
3. **end_frame이 HIGH**가 될 때까지 기다린다
4. **Layer 2의 파라미터로 교체**한다: `q_width=128, q_height=128, q_frame_size=16384`
5. 다시 **q_start를 HIGH**로 올린다 → FSM이 Layer 2를 처리
6. 이를 반복한다

같은 FSM 하드웨어를 **재구성(reconfiguration)**해서 여러 레이어를 순차적으로 처리하는 것입니다. 하드웨어는 변하지 않고, 입력 파라미터만 바뀝니다. 이것이 이 설계의 핵심적인 아름다움입니다 — **하나의 가속기로 모든 합성곱 레이어를 처리**할 수 있습니다.

물론 레이어마다 바뀌는 것은 width/height만이 아닙니다. 필터 개수(출력 채널), 입력 채널 수(→ `i_len`), 필터 크기 (3×3 vs 1×1 → `i_mode`), 가중치 버퍼 시작 주소 등도 모두 레이어별로 다시 설정해야 합니다. 이 재구성을 관리하는 것이 상위 레벨 컨트롤러(나중에 12차시에서 다룰 탑 모듈)의 역할입니다.

11-10. 1×1 합성곱 처리: `i_mode`의 역할

9차시에서 `mac_kern`에 `i_mode` 입력이 있다고 배웠습니다. 이것은 3×3 합성곱과 1×1 합성곱을 구분하는 신호입니다.

3×3 모드 (`i_mode = CONV3x3`): 한 사이클에 9개 곱셈(3×3 윈도우) 수행. 입력 72비트, 가중치 72비트.

1×1 모드 (`i_mode = CONV1x1`): 한 사이클에 1개 곱셈만 의미있음(1×1 필터이므로). 하지만 실제로는 8개 채널을 동시에 처리하는 등 DSP 활용도를 높이는 방식으로 구현할 수 있습니다. Weight Buffer의 72비트 중 64비트(8×8비트)를 사용합니다.

컨트롤러는 현재 레이어가 3×3인지 1×1인지에 따라 `i_mode`를 설정하고, 그에 맞게 주소 생성 패턴도 달라집니다. 1×1 합성곱에서는 슬라이딩 윈도우가 불필요하고(윈도우 크기가 1이니까), 패딩도 필요 없습니다.

11-11. 테스트벤치 검증 전략

이 단계의 검증은 `conv_kern_tb.v`와 `conv_layer_tb.v` 두 가지로 이루어집니다.

`conv_kern_tb.v` — 단일 채널 검증:

- 입력 feature map과 필터를 SW에서 생성한 hex 파일로 로드
- `cnn_ctrl`의 FSM을 이용해 (row, col)을 순회하면서 MAC에 데이터 공급
- 출력을 BMP 이미지로 저장해서 **시각적 검증** (결과가 합리적인 이미지인지 확인)
- 시뮬레이션 시간: 약 1ms ($256 \times 256 = 65,536$ 사이클)

`conv_layer_tb.v` — 다채널 검증:

- `conv_kern_tb`에 채널 루프(channels) 추가
- 부분합(psum) 누적기 포함
- Layer 0의 전체 합성곱 결과를 SW 테스트 벡터(hex 파일)와 **바이트 비교**
- 이것이 "bit-exact 일치"를 확인하는 핵심 검증

7차시에서 배운 단계적 검증 전략을 떠올려 보세요: `mac_tb` → `conv_kern_tb` → `conv_layer_tb` → `cnn_ctrl_multi_layer_tb` → `yolo_engine_tb`. 지금 우리는 이 체인의 가운데 단계에 있습니다.

11-12. 흔한 실수와 디버깅 팁

컨트롤러 설계에서 자주 발생하는 실수들을 미리 알아두면 좋습니다.

데이터 순서 불일치: SW에서 테스트 벡터를 [Channel, Width, Height] 순서로 저장합니다. HW에서 메모리를 읽을 때 이 순서를 틀리면 결과가 완전히 달라집니다.

패딩 경계 처리 누락: 특히 `is_first_row`와 `is_first_col`이 동시에 참인 모서리(corner)에서 실수하기 쉽습니다. 3x3 윈도우의 4개 모서리, 4개 번, 1개 내부 = 총 9가지 케이스를 모두 처리해야 합니다.

카운터 리셋 타이밍: col 카운터가 `q_width-1`에서 0으로 리셋되는 타이밍과, row가 1 증가하는 타이밍이 정확히 맞아야 합니다. 한 클럭이라도 어긋나면 출력이 한 칸씩 밀립니다.

파이프라인 레이턴시 무시: MAC의 레이턴시가 8클럭입니다. `vld_i`를 넣고 8클럭 후에 `vld_o`가 나옵니다. 출력을 저장하는 로직이 이 지연을 고려하지 않으면 잘못된 주소에 데이터를 쓰게 됩니다. `vld` 신호를 시프트 레지스터로 지연시켜서 출력과 동기를 맞춰야 합니다.

signed/unsigned 혼동: 입력은 `UINT8(0~255)`, 가중치는 `INT8(-128~127)`, 누적 결과는 `INT32`입니다. Verilog에서 기본은 `unsigned`이므로, `signed` 연산이 필요한 곳에 `$signed()`를 명시적으로 써야 합니다.

11-13. 큰 그림에서의 위치

지금까지 만든 것들을 조립 과정으로 비유하면:

- **9차시 (MAC):** 엔진을 만들었습니다
- **10차시 (BRAM Buffer):** 연료 탱크를 만들었습니다
- **11차시 (컨트롤러):** 운전대와 계기판을 만들었습니다

다음 12차시에서는 이 모든 것을 하나의 차체(탑 모듈, `yolo_engine.v`)로 통합합니다. 엔진에 연료 탱크를 연결하고, 운전대로 제어하는 전체 자동차를 완성하는 것이죠. DMA를 통해 DRAM과 연결하고, AXI 인터페이스로 MicroBlaze와 통신하는 전체 시스템을 다룹니다.

11차시 핵심 정리:

1. **컨트롤러** = SW의 for 루프를 카운터+FSM으로 구현한 것
2. **FSM 4상태** = ST_IDLE(대기) → ST_VSYNC(프레임 준비) → ST_HSYNC(행 준비) → ST_DATA(연산)
3. **카운터 3개** = col(열, 최내부 루프), row(행), data_count(전체 픽셀)
4. **패딩 처리** = is_first_row/last_row/first_col/last_col 플래그로 경계 밖은 0
5. **멀티 레이어** = end_frame 후 파라미터 재설정 → 같은 HW로 다음 레이어 처리
6. **VSYNC/HSYNC 딜레이** = 데이터 로딩/준비 시간. 성능과 안정성의 트레이드오프

12차시 강의자료를 먼저 확인하겠습니다.충분한 자료를 확인했습니다. 12차시를 시작하겠습니다.

12차시: 탑 모듈 통합 — 전체 가속기 엔진 (yolo_engine)

지난 차시까지 엔진(MAC), 연료 탱크(BRAM Buffer), 운전대(CNN Controller)를 각각 만들었습니다. 이번 차시에 서는 이 모든 것을 하나의 완성된 자동차로 조립합니다.

12-1. 탑 모듈의 전체 블록도

yolo_engine (또는 yolo_engine_ip)은 이 프로젝트 하드웨어의 최상위 모듈입니다. 내부에 6가지 주요 서브모듈이 들어 있습니다:

yolo_engine_ip	
├ AXI Slave (yolo_engine_axi)	← MicroBlaze와 통신
├ Config Registers	← 설정값 저장
├ MAIN CONTROLLER	← 전체 흐름 제어
├ DMA_CTRL (axi_dma_ctrl)	← DMA 요청 생성
│ └ DMA_READ (axi_dma_rd)	← DRAM에서 읽기
│ └ DMA_WRITE (axi_dma_wr)	← DRAM에 쓰기
├ Buffers	← Weight, Feature Map, Bias/Scale
└ DNN Compute Engine	← mac_kern + 후처리

이것을 자동차에 비유하면: AXI Slave는 **핸들**(외부와의 인터페이스), Config Registers는 **계기판**(현재 설정), Main

Controller는 **ECU**(전자 제어 장치), DMA는 **연료 펌프**(데이터를 퍼 나르는 것), Buffers는 **연료 탱크**, DNN Compute Engine은 **엔진** 자체입니다.

12-2. AXI Slave: MicroBlaze와의 대화 창구

`yolo_engine_axi.v`는 MicroBlaze CPU와 YOLO 가속기 사이의 **통신 인터페이스**입니다. AXI4-Lite 프로토콜을 사용합니다.

MicroBlaze가 이 모듈에 **쓰기(Write)**를 하면, 가속기의 설정 레지스터가 업데이트됩니다. MicroBlaze가 **읽기(Read)**를 하면, 가속기의 상태(예: 연산 완료 여부)를 돌려받습니다.

구체적으로 동작을 살펴봅시다:

설정 쓰기 (MicroBlaze → 가속기): MicroBlaze가 특정 주소에 32비트 값을 쓰면, AXI Slave가 그 주소를 디코딩해서 해당 설정 레지스터(`ctrl_reg0`, `ctrl_reg1` 등)를 업데이트합니다. 예를 들어 `ctrl_reg0`에 1을 쓰면 "연산 시작" 명령이 됩니다.

상태 읽기 (가속기 → MicroBlaze): MicroBlaze가 특정 주소를 읽으면, AXI Slave가 가속기의 현재 상태를 반환합니다. 핵심은 `network_done` 신호입니다. MicroBlaze는 이것을 **폴링(polling)**해서 연산이 끝났는지 확인합니다:

```
// MicroBlaze 펌웨어 코드 (의사 코드)
while(1) {
    readout = get_config(YOLO_BASE_ADDR);
    if ((readout >> 1) & 0x00000001)
        break; // 연산 완료!
}
```

`>> 1`로 비트 시프트하는 이유는 `network_done`이 레지스터의 비트 1에 매핑되어 있기 때문입니다.

12-3. Config Registers: 가속기의 설정판

yolo_engine.v에는 4개의 설정 레지스터가 있습니다:

i_ctrl_reg0: 가속기 시작/정지 제어. 비트 0이 network_start입니다. 1을 쓰면 연산 시작, 0으로 돌리면 정지. 디버깅용 비트들도 포함될 수 있습니다.

i_ctrl_reg1: DRAM 읽기 시작 주소. 입력 feature map이 DRAM의 어디서 시작하는지를 지정합니다.

i_ctrl_reg2: DRAM 쓰기 시작 주소. 출력 feature map을 DRAM의 어디에 저장할지를 지정합니다.

i_ctrl_reg3: 예비용 레지스터.

탑 모듈 내부에서 이 레지스터들의 값을 파싱해서 DMA 컨트롤러에 전달합니다:

```
// yolo_engine.v 내부 (개념적 코드)
wire network_start = i_ctrl_reg0[0];
wire [31:0] dram_base_addr_rd = i_ctrl_reg1;
wire [31:0] dram_base_addr_wr = i_ctrl_reg2;
```

이 구조의 핵심은 **하드웨어가 범용적**이라는 것입니다. 같은 하드웨어로 다른 레이어를 처리할 때, 소프트웨어 (MicroBlaze)가 레지스터 값만 바꿔주면 됩니다. 읽기 주소를 Layer 0의 입력 주소로 설정하면 Layer 0을, Layer 2의 입력 주소로 바꾸면 Layer 2를 처리합니다.

12-4. DMA 시스템: DRAM과 버퍼 사이의 데이터 고속도로

DMA(Direct Memory Access)는 CPU의 개입 없이 메모리와 하드웨어 사이에 데이터를 직접 전송하는 메커니즘입니다. 이 프로젝트에서 DMA는 세 개의 모듈로 구성됩니다.

DMA Controller (axi_dma_ctrl.v)

DMA 컨트롤러는 "총 몇 번의 전송 요청을 보낼지"를 관리하는 **요청 생성기**입니다.

핵심 입력:

- `dram_base_addr_rd`: DRAM 읽기 시작 주소
- `dram_base_addr_wr`: DRAM 쓰기 시작 주소
- `num_trans`: 한 번의 AXI 버스트에서 전송할 워드 수 (이 프로젝트에서는 **16으로 고정**)
- `max_req_blk_idx`: 총 블록 수. 예를 들어 256×256 이미지를 읽으려면 $(256 \times 256) / 16 = 4096$ 블록

핵심 출력:

- `ctrl_read`, `ctrl_write`: DMA Read/Write 모듈에 "전송 시작" 신호
- `read_addr`, `write_addr`: 현재 읽기/쓰기할 DRAM 주소

DMA Read FSM

DMA 읽기의 핵심 흐름을 따라가 봅시다:

- ① Main Controller가 "DRAM에서 데이터를 읽어라" 명령
- ② DMA Controller가 `read_addr = Base + req_blk_idx_rd × 64` 를 계산
(64 = 16워드 × 4바이트)
- ③ DMA Read가 AXI 버스를 통해 DRAM에 읽기 요청 전송
- ④ DRAM이 16개의 32비트 데이터를 버스트로 반환
- ⑤ DMA Read가 데이터를 Buffer에 기록
- ⑥ 완료 후 `read_done` 신호 → DMA Controller가 다음 블록 주소 계산
- ⑦ 모든 블록이 끝날 때까지 ②~⑥ 반복

주소 계산 코드를 보면:

```
assign read_addr = dram_base_addr_rd + {req_blk_idx_rd, 6'b0};
```

`{req_blk_idx_rd, 6'b0}`은 `req_blk_idx_rd × 64`와 같습니다. 6비트를 아래에 붙이는 것이 64를 곱하는 것과 동일하거든요. 이것은 하드웨어에서 곱셈 대신 비트 연결(concatenation)로 시프트를 구현하는 전형적인 기법입니다.

Read FSM 자체도 간단한 3상태 FSM입니다:

```
IDLE → DMA_WAIT → DONE → IDLE
      ↑ (SYNC with Write)
```

IDLE에서 `i_start` 신호를 받으면 DMA_WAIT로 전이하고, AXI 읽기를 수행합니다. `read_done` 이 오면 DONE으로 가고, 필요하면 Write와 동기화(SYNC)한 후 다시 IDLE로 돌아갑니다.

DMA Write FSM

쓰기도 비슷하지만 주소 계산이 약간 다릅니다:

```
assign write_addr = dram_base_addr_wr + {req_blk_idx_wr, 6'b0} +  
{write_data_cnt, 2'b0};
```

블록 오프셋(`req_blk_idx_wr × 64`)에 데이터 오프셋(`write_data_cnt × 4`)을 더합니다. 쓰기는 한 번에 한 워드(32비트)씩 보내면서 `write_data_cnt`를 증가시킵니다.

DMA Write는 AXI 프로토콜의 **Write Response Channel**도 처리합니다. DRAM이 "잘 받았다"고 응답(BRESP)을 보내야 다음 전송을 시작합니다. `fail_check` 신호로 쓰기 실패를 감지할 수 있습니다.

12-5. 버퍼 시스템 통합

10차시에서 개별적으로 설계했던 버퍼들이 탑 모듈에서 어떻게 연결되는지 봅시다.

연결 관계

```
DMA_READ  —(32bit data)—> Global Buffer (256KB)  
Global Buffer —> Feature Map Buffer (2KB × 2, 더블 버퍼)  
                —> Weight Buffer (36KB, dpram_4096×72)  
                —> Bias/Scale Buffer (10KB, spram_2560×32)  
  
Feature Map Buffer —(72bit din)—> mac_kern  
Weight Buffer —(288bit wgt)—> mac_kern  
Bias/Scale Buffer —(32bit bsc)—> 후처리 로직
```

```
mac_kern —(출력)—> 후처리 —> Feature Map Buffer (출력 쪽)
Feature Map Buffer —> Global Buffer —> DMA_WRITE —> DRAM
```

10차시에서 배운 `gbuff_param.v`를 떠올려 보세요. 이 모듈은 Weight Buffer와 Bias/Scale Buffer를 하나로 묶은 것이었습니다. DMA에서 쓸 때는 `dma_wea`, `dma_addra`, `dma_dina` 포트를 사용하고, MAC에서 읽을 때는 `wgt_addrb`와 `bsc_addra` 포트를 사용합니다. 이처럼 각 버퍼가 **두 가지 접근 경로**(DMA 쓰기 + 연산 읽기)를 가지고 있어서, 데이터 로딩과 연산을 가능한 한 병렬로 수행할 수 있습니다.

더블 버퍼링의 실제 동작

Feature Map Buffer가 2개(2KB × 2)인 이유는 **더블 버퍼링** 때문입니다. 동작을 시간순으로 보면:

```
시점 1: Buffer A에서 현재 타일 읽기(연산) + Buffer B에 다음 타일 쓰기(DMA 로딩)
시점 2: Buffer B에서 현재 타일 읽기(연산) + Buffer A에 다음 타일 쓰기(DMA 로딩)
시점 3: 역할 교대, 반복...
```

이렇게 하면 **데이터 로딩 지연을 연산 시간으로 숨길** 수 있습니다. 한 버퍼에서 연산하는 동안 다른 버퍼에 미리 데이터를 채워놓으니까요. 이것은 성능 최적화의 핵심 기법 중 하나입니다.

12-6. DNN Compute Engine: 연산의 심장

DNN Compute Engine은 11차시의 CNN 컨트롤러와 9차시의 MAC 유닛이 결합된 것입니다. 핵심 데이터 흐름을 한 출력 픽셀에 대해 따라가 봅시다:

- ① `cnn_ctrl`이 (row, col) 좌표 생성 → `ctrl_data_run` = HIGH
- ② 주소 생성 로직이 Feature Map Buffer 주소와 Weight Buffer 주소 계산
- ③ Buffer에서 데이터 읽기 (1사이클 레이턴시)
- ④ `mac_kern`에 `vld_i`, `din`(입력 윈도우), `win`(필터 가중치) 공급
- ⑤ `mac_stack`이 36개 곱셈 수행 (파이프라인: 4사이클)
- ⑥ `adder_tree`가 합산 (4사이클)
- ⑦ `mac_kern` 내부 누적기가 채널 방향 부분합 누적 (`i_len` 사이클)

- ⑧ 누적 완료 → bsc_req(bias/scale 요청) 신호 발생
- ⑨ Bias/Scale Buffer에서 bias(16bit)와 scale(16bit) 읽기
- ⑩ 후처리: $\text{psum} + \text{bias} \rightarrow \text{ReLU} \rightarrow \text{de-scaling}(\div \text{scale}) \rightarrow \text{UINT8}$ 출력
- ⑪ 출력을 Feature Map Buffer에 기록

⑩의 de-scaling을 잠깐 더 살펴봅시다. 5차시에서 배운 대로, de-scaling은 누적합을

$(\text{input_quant_multiplier} \times \text{weights_quant_multiplier}) / \text{next_input_quant_multiplier}$ 로 나누는 것입니다. 하드웨어에서 나눗셈은 비싸므로, 이 값을 미리 계산해서 scale로 Bias/Scale Buffer에 저장해 두고, 정수 나눗셈으로 처리합니다.

12-7. yolo_engine.v의 입출력 인터페이스

탑 모듈의 포트를 정리하면:

제어 입력 (MicroBlaze에서):

- i_ctrl_reg0: 시작/정지/디버그 제어
- i_ctrl_reg1: DRAM 읽기 주소
- i_ctrl_reg2: DRAM 쓰기 주소
- i_ctrl_reg3: 예비

AXI Master 인터페이스 (DRAM과 통신):

- 읽기 채널: M_ARVALID, M_ARADDR, M_ARLEN, M_RDATA, M_RVALID, M_RLAST 등
- 쓰기 채널: M_AWVALID, M_AWADDR, M_WDATA, M_WVALID, M_WLAST, M_BRESP 등

상태 출력:

- network_done: 전체 네트워크 추론 완료 플래그
- network_done_led: FPGA 보드의 LED에 연결해서 시각적 확인
- read_data_vld, read_data: 디버깅용 데이터 출력

AXI 신호가 매우 많아 보이지만, 대부분은 AXI 프로토콜의 정해진 형식입니다. 핵심만 이해하면 됩니다: **주소 채널 (Address Channel)**로 "어디를 읽을/쓸 건지" 알려주고, **데이터 채널 (Data Channel)**로 실제 데이터를 주고받고, **응답 채널 (Response Channel, 쓰기 전용)**로 "성공적으로 썼다"를 확인합니다.

12-8. 테스트벤치: yolo_engine_tb.v 해부

이것이 가장 상위 레벨의 테스트벤치입니다. 실제 FPGA 보드 대신 **시뮬레이션 환경에서 전체 시스템을 테스트**합니다.

구성 요소

```
yolo_engine_tb
├─ yolo_engine (DUT - Design Under Test)
├─ axi_sram_if (AXI ↔ SRAM 인터페이스)
├─ sram (외부 메모리 모델)
└─ bmp_image_writer × 4 (디버깅용 이미지 저장)
```

외부 메모리 모델 (sram)

실제 DDR2 DRAM 대신, **Verilog로 구현된 간단한 SRAM 모델**을 사용합니다. 이 모델은 hex 파일에서 초기 데이터를 읽어옵니다:

```
sram #(
    .FILE_NAME(IFM_FILE),    // hex 파일 경로
    .SIZE(2**MEM_ADDRW),     // 메모리 크기
    .WL_ADDR(MEM_ADDRW),     // 주소 비트폭 (22비트)
    .WL_DATA(MEM_DW))        // 데이터 비트폭 (16비트)
u_ext_mem_input (
    .clk(clk), .rst(rstn),
    .addr(mem_addr), .wdata(mem_di),
    .rdata(mem_do),
    .ena(1'b0)               // Read only
);
```

중요한 제약: 이 메모리 모델은 **16비트 데이터만 지원**합니다. 따라서 SW에서 생성한 8비트 테스트 벡터를 16비트로 패킹해야 합니다. 예를 들어 8비트 데이터 {64, 70, 70, 00}을 16비트 메모리에 넣으려면 {7064, 0070}으로 2개의 16비트 워드로 분할합니다.

AXI SRAM Interface (axi_sram_if)

이 모듈은 yolo_engine의 AXI Master 포트와 SRAM 모델을 변환해줍니다. AXI 프로토콜의 핸드셰이크 (VALID/READY)를 처리하고, 버스트 전송을 개별 메모리 접근으로 변환합니다. 실제 FPGA에서는 MIG(Memory Interface Generator)가 이 역할을 하지만, 시뮬레이션에서는 이 간단한 모델로 대체합니다.

테스트 시퀀스

yolo_engine_tb.v의 initial 블록을 따라가 봅시다:

```
initial begin
    rstn = 1'b0;                // ① 리셋
    i_0 = 0;                    // 설정 초기화
    i_1 = 0;                    // 읽기 주소 = 0
    i_2 = (4*256*256)*4;        // 쓰기 주소 = 입력 뒤

    #(4*CLK_PERIOD) rstn = 1'b1; // ② 리셋 해제
    #(100*CLK_PERIOD)
        @(posedge clk) i_0 = 32'd1; // ③ 연산 시작 (network_start = 1)
    #(100*CLK_PERIOD)
        @(posedge clk) i_0 = 32'd0; // ④ 시작 펄스 해제

    while(!network_done) begin // ⑤ 완료 대기 (폴링)
        #(128*CLK_PERIOD) @(posedge clk);
    end

    #(100*CLK_PERIOD)
        @(posedge clk) $stop; // ⑥ 시뮬레이션 종료
end
```

이 시퀀스는 실제 FPGA에서 MicroBlaze가 하는 일과 거의 동일합니다: 리셋 → 주소 설정 → 시작 → 완료 대기 → 종료. 시뮬레이션에서 이것이 잘 동작하면, 실제 FPGA에서도 동작할 것이라고 기대할 수 있습니다.

BMP Image Writer로 시각적 검증

테스트벤치에 4개의 bmp_image_writer가 있습니다. DMA를 통해 읽거나 쓴 32비트 데이터를 바이트 단위로 분리해서 각각 BMP 이미지 파일로 저장합니다:

```
// DMA Write 데이터를 4개 채널로 분리해서 이미지 저장
din = i_WDATA[7:0]    // 채널 0
din = i_WDATA[15:8]   // 채널 1
din = i_WDATA[23:16]  // 채널 2
din = i_WDATA[31:24]  // 채널 3
```

이 BMP 파일을 열어보면 출력 feature map이 시각적으로 보입니다. 에지가 검출되는 필터의 출력은 윤곽선이, 색상 필터의 출력은 특정 색이 강조된 이미지가 나옵니다. 이것은 **정량적 검증(hex 비교)**과 함께 사용하는 **정성적 검증** 방법입니다.

단, 강의자료에서 강조하듯 BMP 이미지는 **시각적 확인용에 불과**합니다. 최종 검증은 반드시 SW에서 생성한 hex 파일과의 바이트 단위 비교로 해야 합니다.

12-9. 전체 시뮬레이션 흐름

yolo_engine_tb.v의 시뮬레이션이 실행되면 내부적으로 이런 일이 벌어집니다:

시뮬레이션 시작

↓

① 리셋 (4 클럭)

↓

② SRAM에 hex 파일의 입력 이미지 데이터가 초기화됨

↓

③ network_start = 1 (100 클럭 대기 후)

↓

④ Main Controller가 DMA_CTRL에 "DRAM에서 입력 읽어라" 명령

↓

⑤ DMA_READ가 AXI를 통해 SRAM에서 16워드씩 버스트 읽기
→ Global Buffer에 저장

↓

⑥ Global Buffer에서 Feature Map Buffer로 타일 전송
(+ Weight Buffer에서 필터 준비)

↓

⑦ cnn_ctrl FSM 시작: ST_IDLE → ST_VSYNC → ST_HSYNC → ST_DATA

```

↓
⑧ ST_DATA: 매 클럭마다 (row,col)에서 3×3 윈도우 생성 → MAC 연산
  → 채널 누적 → bias/ReLU/de-scaling → 출력 저장
↓
⑨ end_frame → 레이어 완료
↓
⑩ 출력을 DMA_WRITE로 SRAM에 기록
↓
⑪ 다음 레이어 파라미터 재설정 → ④부터 반복
↓
⑫ 모든 레이어 완료 → network_done = 1
↓
⑬ 테스트벤치가 $stop → 시뮬레이션 종료

```

시뮬레이션 시간은 4ms 정도로 설정합니다. 100MHz 클럭(10ns 주기)에서 4ms면 400,000 클럭 사이클입니다. Layer 0 하나만 해도 $256 \times 256 = 65,536$ 출력 픽셀이고, 채널 3번 반복 + 파이프라인 딜레이를 합치면 약 20만 사이클 정도 소요됩니다.

12-10. 파일/디렉토리 구조

코드가 어떻게 정리되어 있는지 알아두면 파일을 찾을 때 편합니다:

```

yolohw/
├─ src/                                # 소스 코드
│   ├── yolo_engine.v                 # 탑 모듈
│   ├── yolo_engine_axi.v             # AXI Slave 인터페이스
│   ├── axi_dma_read.v                # DMA 읽기 모듈
│   ├── axi_dma_write.v               # DMA 쓰기 모듈
│   ├── axi_dma_ctrl.v                # DMA 컨트롤러
│   ├── cnn_ctrl.v                    # CNN 컨트롤러 (FSM)
│   ├── mac_kern.v                    # MAC 커널
│   ├── mac_stack.v                   # MAC 스택
│   ├── gbuff_param.v                 # 파라미터 버퍼
│   └─ user_param_h.v                 # 사용자 파라미터 정의
├─ sim/                                # 시뮬레이션
│   └─ yolo_engine_tb.v               # 탑 레벨 테스트벤치

```

```

|   |─ conv_layer_tb.v           # 레이어 단위 테스트
|   |─ cnn_ctrl_multi_layer_tb.v # 멀티 레이어 컨트롤러 테스트
|   |─ inout_data_sw/
|   |   |─ log_feamap/           # SW에서 생성한 feature map hex
|   |   |─ log_param/            # SW에서 생성한 파라미터 hex
|   |─ inout_data_hw/           # HW 시뮬레이션 출력
|   |─ sim_dram_model/          # 외부 메모리 모델
|─ arxiv/                        # 결과 스크린샷

```

`user_param_h.v`는 시뮬레이션과 합성에서 공통으로 사용하는 파라미터를 정의합니다. `yolo_engine_tb.v`의 맨 위에서 `include` 하는 것을 확인할 수 있습니다.

12-11. 데이터 패킹: 8비트를 16비트/32비트로

실전에서 자주 헷갈리는 부분이 데이터 패킹입니다.

SW에서 생성한 테스트 벡터는 **8비트 단위**입니다 (UINT8 또는 INT8). 하지만 DRAM은 **32비트 단위**로 읽고, 시뮬레이션 메모리 모델은 **16비트 단위**입니다.

따라서 데이터를 패킹할 때 **바이트 순서(endianness)**를 잘 맞춰야 합니다. 강의자료의 예시를 보면:

```

SW에서 생성한 8비트 데이터:  64, 70, 70, 00
32비트로 패킹:                0x00707064 (= {00, 70, 70, 64})
16비트 메모리에 저장:        줄 1: 7064 (하위 16비트)
                               줄 2: 0070 (상위 16비트)

```

채널 방향으로 패킹한다면, 같은 픽셀 위치의 R, G, B 값을 하나의 32비트 워드에 넣습니다: {padding, B, G, R}. 이 순서는 설계자가 결정하는 것이고, SW의 hex 파일 생성 코드와 HW의 언패킹 로직이 일치해야 합니다.

12-12. 디버깅 전략: 어디서부터 문제를 찾을 것인가

탐 모듈 통합 후에 시뮬레이션이 안 될 때, 문제를 찾는 순서입니다:

1단계: DMA Read 확인. DMA가 SRAM에서 데이터를 제대로 읽어오는가? `read_data_vld`와 `read_data`를 파형(waveform)에서 확인합니다. BMP Writer가 입력 이미지를 정상적으로 출력하는지 봅니다.

2단계: Buffer 확인. 읽어온 데이터가 Feature Map Buffer와 Weight Buffer에 올바른 주소로 저장되었는가? BRAM의 내용을 시뮬레이션에서 직접 확인합니다.

3단계: CNN Controller 확인. FSM이 정상적으로 상태 전이하는가? row, col 카운터가 올바르게 증가하는가? `ctrl_data_run`이 적절한 타이밍에 HIGH가 되는가?

4단계: MAC 출력 확인. `mac_kern`의 `vld_o`와 `acc_o`를 확인합니다. 첫 번째 출력 픽셀의 값이 SW 테스트 벡터의 값과 일치하는가?

5단계: DMA Write 확인. 최종 출력이 SRAM에 올바르게 기록되는가? 기록된 hex 파일을 SW 출력과 비교합니다.

문제가 어느 단계에서 발생하는지 찾으면 해당 서브모듈만 집중적으로 디버깅하면 됩니다. 이것이 7차시에서 배운 **단계적 검증**의 실제 적용입니다.

12-13. 이번 차시의 위치와 다음 단계

지금까지의 여정을 정리하면:

차시	무엇을 만들었나	비유
9	MAC 유닛	엔진
10	BRAM 버퍼	연료 탱크
11	CNN 컨트롤러	운전대
12	탐 모듈 통합	완성된 자동차

자동차는 완성되었지만, 아직 **도로 위에 올려놓지는 않았습니다**. 시뮬레이션이라는 "가상 도로"에서만 테스트한 상태입니다.

다음 13차시에서는 이 자동차를 **실제 도로(FPGA 보드)**에 올려놓습니다. Vivado에서 합성(Synthesis)하고, 비트스트림(Bitstream)을 생성하고, IP로 패키징해서 MicroBlaze 시스템에 연결하는 과정을 다룹니다.

12차시 핵심 정리:

1. **yolo_engine** = AXI Slave + DMA(Read/Write/Ctrl) + Buffers + DNN Compute Engine
2. **AXI Slave** = MicroBlaze ↔ 가속기 통신. 설정 쓰기 + 상태 읽기(폴링)
3. **DMA 시스템** = 16워드 버스트 전송. 주소 = Base + block_idx × 64
4. **테스트벤치** = SRAM 모델(hex 초기화) + AXI 인터페이스 + BMP Writer(시각 검증)
5. **데이터 패킹** = 8비트 → 16비트/32비트. 바이트 순서 일치 필수
6. **디버깅** = DMA Read → Buffer → Controller → MAC → DMA Write 순서로 추적

다음 13차시(FPGA 구현)를 진행할 준비가 되면 말씀해 주세요!